THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that **MS. K KAVINILA (20524UCSC053),** a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions**		

	• **Aggregate Functions** • **String Functions** • **Date Functions**		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

EX. NO:01 Creation of DDL Operations Using MovieCollection Table

DATE:

Aim

To create a MovieCollection table, insert movie records, display all records, and retrieve movies released from the year 2000 onwards.

Software Used

Oracle / MySQL / SQLite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects in a database.

DDL COMMANDS

- Create
- Alter
- Drop
- Truncate

CREATE

Create command is used to create a new table and also use constraints.

Syntax:

```
CREATE TABLE table_name (
   column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure

```
SQL> CREATE TABLE movie_collection (
   id INTEGER PRIMARY KEY,
   name TEXT,
   release_year INTEGER
);
```

Table Created

ALTER

Alter command is used to alter a table or make some changes in a table. It has 3 methods such as ADD, MODIFY, DROP.

ADD

Add command is used to insert a new attribute/column with data type.

Syntax:

ALTER TABLE table_name ADD column_name datatype;

Query:

SQL> ALTER TABLE movie_collection ADD director VARCHAR(100);

Table Altered

MODIFY

Modify command is used to modify or change the data type and to increase the data size in the table.

Syntax:

ALTER TABLE table_name MODIFY column_name datatype;

Query:

SQL> ALTER TABLE movie_collection MODIFY name VARCHAR(150);

OR

ALTER TABLE movie_collection ALTER COLUMN name VARCHAR(150);

DROP

Drop command is used to delete the attribute/column in a table.

Syntax:

ALTER TABLE table_name DROP COLUMN column_name;

Query:

SQL> ALTER TABLE movie_collection DROP COLUMN director;

RENAME

Rename command is used to re-change the table name.

SQL> RENAME TABLE movie_collection TO films;

Table renamed.

OR

ALTER TABLE movie_collection RENAME TO films;

Column Rename

ALTER TABLE movie_collection RENAME COLUMN name TO movie_name;

TRUNCATE

Truncate command is used to delete all values in the table.

Syntax:

TRUNCATE TABLE tablename;

Query:

TRUNCATE TABLE movie_collection;

DROP

Drop command deletes the table and all its data permanently.

Syntax:

DROP TABLE tablename;

Query:

DROP TABLE movie_collection;

Explanation

- Create a table named movie_collection.
- Insert movie details into the table.
- Display all movie records.
- Retrieve movies released in or after the year 2000 and sort them by release year.

Create Table

```
CREATE TABLE movie_collection (
   id INTEGER PRIMARY KEY,
   name TEXT,
   release_year INTEGER
);
```

Sample Data

```
INSERT INTO movie_collection VALUES (1, 'Inception', 2009);
INSERT INTO movie_collection VALUES (2, 'Good Will Hunting', 1997);
INSERT INTO movie_collection VALUES (3, 'Jaws', 1977);
```

INSERT INTO movie_collection VALUES (4, 'The Incredibles', 2004);

INSERT INTO movie_collection VALUES (5, 'Pulp Fiction', 1994);

INSERT INTO movie_collection VALUES (6, 'Coraline', 2009);

SQL Query

Query 1: Display all movie records

SQL> SELECT * FROM movie_collection;

Resultant Query

id	name	release_year
1	Inception	2009
2	Good Will Hunting	1997
3	Jaws	1977
4	The Incredibles	2004
5	Pulp Fiction	1994
6	Coraline	2009

Query 2: Display movies released from the year 2000 onwards

SQL> SELECT *

FROM movie_collection

WHERE release_year >= 2000

ORDER BY release_year;

Resultant Query

id	name	release year
4	The Incredibles	2004
1	Inception	2009
6	Coraline	2009

RESULT

Thus, the MovieCollection table was created successfully, records were inserted, all movie details were displayed, and the movies released from the year 2000 onwards were retrieved successfully.

EX. NO:02 Creation of DML Operations using Student Performance Table

DATE:

Aim

To create a Student Performance table, insert student records, display all records, update student details, delete a student record, and retrieve student records based on their marks.

Software Used

Oracle / MySQL / sqlite3

Description

DML (Data Manipulation Language) is used to insert, modify, delete, and retrieve data stored in database tables.

DML COMMANDS:

- INSERT
- UPDATE
- DELETE
- SELECT

INSERT

INSERT command is used to add new records into a table.

Syntax

INSERT INTO table_name (column1, column2, column3)

VALUES (value1, value2, value3);

Query

SQL> INSERT INTO student_performance VALUES (1, 'Aarav', 82);

1 row inserted.

UPDATE

UPDATE command is used to modify existing records in a table.

Syntax

UPDATE table_name

SET column_name = value

WHERE condition;

Query

SQL> UPDATE student_performance

SET mark = 89

WHERE id = 1;

1 row updated.

DELETE

DELETE command is used to remove existing records from a table.

Syntax

DELETE FROM table_name

WHERE condition;

Query

SQL> DELETE FROM student_performance

WHERE id = 5;

1 row deleted.

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

SELECT * FROM table_name;

Query

SQL> SELECT * FROM student_performance;

Explanation

- Create a table named **student_performance**.
- Insert student details into the table.
- Display all student records.
- Update the marks of a student.
- Delete a student record.
- Retrieve students who scored 80 or above and sort them by marks.

Create Table

```
CREATE TABLE student_performance (
    id INTEGER PRIMARY KEY,
    name TEXT,
    mark INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO student_performance VALUES (1, 'Aarav', 82);
INSERT INTO student_performance VALUES (2, 'Nila', 74);
INSERT INTO student_performance VALUES (3, 'Vihan', 93);
INSERT INTO student_performance VALUES (4, 'Tara', 67);
INSERT INTO student_performance VALUES (5, 'Kabir', 86);
INSERT INTO student_performance VALUES (6, 'Ana', 96);
```

Records Inserted

SQL Query

Query 1: Display all student records

```
SQL> SELECT * FROM student_performance;
```

Resultant Query

id	name	mark
1	Aarav	82
2	Nila	74
3	Vihan	93
4	Tara	67
5	Kabir	86
6	Ana	96

Query 2: Update the marks of Aarav

```
SQL> UPDATE student_performance
SET mark = 89
```

WHERE id = 1;

1 row updated.

Updated Record

id	name	Mark
1	Aarav	89

Query 3: Delete Kabir's record

SQL> DELETE FROM student_performance

WHERE id = 5;

1 row deleted.

Remaining Records

id	name	mark
1	Aarav	89
2	Nila	74
3	Vihan	93
4	Tara	67
6	Ana	96

Query 4: Display students who scored 80 or above

SQL> SELECT *

FROM student_performance

WHERE mark >= 80

ORDER BY mark;

Resultant Query

id	name	mark
1	Aarav	89
3	Vihan	93
6	Ana	96

RESULT

Thus, the **Student Performance** table was created successfully, records were inserted, student details were displayed, the student marks were updated, a student record was deleted, and the students who scored 80 or above were retrieved successfully.

EX. NO:03 Creation of DQL Operations using Employee Scores Table

DATE:

Aim

To create a Employee Scores table, insert employee records, display all records, update employee details, delete a employee record, and retrieve employee_scores based on their marks**.**

Software Used

Oracle / MySQL / sqlite3

Description

DML (Data Manipulation Language) is used to insert, modify, delete, and retrieve data stored in database tables.

DML COMMANDS:

- INSERT
- UPDATE
- DELETE
- SELECT

INSERT

INSERT command is used to add new records into a table.

Syntax

INSERT INTO table_name (column1, column2, column3)

VALUES (value1, value2, value3);

Query

SQL> INSERT INTO employee_scores VALUES (1, 'Ish',

84);

1 row inserted.

UPDATE

UPDATE command is used to modify existing records in a table.

Syntax

UPDATE table_name

SET column_name = value

WHERE condition;

Query

SQL> UPDATE employee_scores

SET mark = 88

WHERE id =

1; 1

rowupdated.

DELETE

DELETE command is used to remove existing records from a table.

Syntax

DELETE FROM table_name

WHERE condition;

Query

SQL> DELETE FROM employee_scores

WHERE id =

5; **1 row**

deleted.

SELECT

SELECT command is used to retrieve and display records from a table.

Syntax

SELECT * FROM table_name;

Query

SQL> SELECT * FROM employee_scores;

Explanation

- Create a table named employee_scores.
- Insert employee details into the table.
- Display all employee records.
- Update the marks of a employee.
- Delete a employee record.
- Retrieve employee_scores who scored 80 or above and sort them by marks.

Create Table

```
CREATE TABLE employee_scores
(     id INTEGER PRIMARY KEY,
    name TEXT,
mark INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO employee_scores VALUES (1, 'Ish', 84);
INSERT INTO employee_scores VALUES (2, 'Diya', 76);
INSERT INTO employee_scores VALUES (3, 'Ravi', 92);
INSERT INTO employee_scores VALUES (4, 'Kavi', 69);
INSERT INTO employee_scores VALUES (5, 'Manu', 87);
INSERT INTO employee_scores VALUES (6, 'Sneha', 94);
```

Records Inserted

SQL Query

Query 1: Display all employee records

```
SQL> SELECT * FROM employee_scores;
```

Resultant Query

id	name	mark
1	Ish	84
2	Diya	76
3	Ravi	92
4	Kavi	69
5	Manu	87
6	Sneha	94

Query 2: Update the marks of Ish

SQL> UPDATE employee_scores

SET mark = 88

WHERE id =

1; 1 row

updated.

Updated Record

id	name	mark
1	Ish	88

Query 3: Delete Manu's record

SQL> DELETE FROM employee_scores WHERE id = 5;

1 row deleted.

Remaining Records

id	name	mark
1	Ish	88
2	Diya	76
3	Ravi	92
4	Kavi	69
6	Sneha	94

Query 4: Display employee_scores who scored 80 or above

```
SQL> SELECT *
FROM employee_scores
WHERE mark >= 80
ORDER BY mark;
```

Resultant Query

id	name	mark	
1	Ish	88	
3	Ravi	92	
6	Sneha	94	

RESULT:

Thus, the Employee Scores table was created successfully, records were inserted, employee details were displayed, the employee marks were updated, a employee record was deleted, and the employee_scores who scored 80 or above were retrieved successfully.

EX. NO:04 Creation of Transaction Operations using Client Accounts Table

DATE:

Aim

To create an Client Accounts table, insert client records, perform transaction operations using COMMIT, ROLLBACK, and SAVEPOINT, and verify the changes made to the database.

Software Used

Oracle / MySQL / sqlite3

Description

Transaction Control Language (TCL) is used to manage transactions in a database. It helps to save or undo the changes made to the database.

TRANSACTION COMMANDS:

- COMMIT
- ROLLBACK
- SAVEPOINT

COMMIT

COMMIT command is used to permanently save all the changes made during a transaction.

Syntax

COMMIT;

Query

SQL> COMMIT;

Transaction committed.

ROLLBACK

ROLLBACK command is used to undo the changes made during a transaction before the changes are committed.

Syntax

ROLLBACK;

Query

SQL> ROLLBACK;

Transaction rolled back.

SAVEPOINT

SAVEPOINT command is used to create a temporary point within a transaction. The transaction can be rolled back to that specific point.

Syntax

SAVEPOINT savepoint_name;

Query

SQL> SAVEPOINT sp1;

Savepoint created.

ROLLBACK TO SAVEPOINT

ROLLBACK TO SAVEPOINT is used to undo the changes made after a particular savepoint without undoing the entire transaction.

Syntax

ROLLBACK TO SAVEPOINT savepoint_name;

Query

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

Explanation

- Create a table named client_clients.
- Insert client details into the table.
- Use COMMIT to permanently save the inserted records.
- Update an client balance and create a SAVEPOINT.
- Make another change after the savepoint.
- Use ROLLBACK TO SAVEPOINT to undo the changes made after the savepoint.

- Use ROLLBACK to undo uncommitted changes.

Create Table

```
CREATE TABLE client_clients (
client_id INTEGER PRIMARY KEY,
   client_holder TEXT,
balance INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO client_clients VALUES (101, 'Adi', 28000);
INSERT INTO client_clients VALUES (102, 'Hari', 31000);
INSERT INTO client_clients VALUES (103, 'Naren', 21000);
INSERT INTO client_clients VALUES (104, 'Kavi', 37000);
INSERT INTO client_clients VALUES (105, 'Sanju', 42000);
```

Records Inserted

SQL Query

Query 1: Display all client records

```
SQL> SELECT * FROM client_clients;
```

Resultant Query

client_id	client_holder	balance
101	Adi	28000
102	Hari	31000
103	Naren	21000
104	Kavi	37000
105	Sanju	42000

Query 2: Commit the inserted records

SQL> COMMIT;

Transaction committed.

The inserted client records are now permanently saved.

Query 3: Update Adi's client balance

SQL> UPDATE client_clients

SET balance = 31000

WHERE client_id =

101; 1 row updated.

Updated Record

client_id	client_holder	balance
101	Adi	31000

Query 4: Create a SAVEPOINT

SQL> SAVEPOINT sp1;

Savepoint created.

Query 5: Update Hari's client balance

SQL> UPDATE client_clients

SET balance = 37000

WHERE client_id =

102; 1 row updated.

Updated Record

client_id	client_holder	balance
102	Hari	37000

Query 6: Rollback to SAVEPOINT

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

The changes made after SAVEPOINT sp1 are undone.

Resultant Query

client_id	client_holder	balance
101	Adi	31000
102	Hari	31000

Query 7: Rollback an uncommitted change

SQL> UPDATE client_clients

SET balance = 16000

WHERE client_id =

103; **1 row updated**.

SQL> ROLLBACK;

Transaction rolled back.

The balance of Naren is restored to its previous value.

Resultant Query client

_id	**client_holder**	**balance**
103	**Naren**	**21000:**

TRUTH-TOIL-TRIUMPH AUTONOMOUS

RESULT: Thus, the Client Accounts table was created successfully, client records were inserted, and the transaction operations COMMIT, ROLLBACK, SAVEPOINT, and ROLLBACK TO SAVEPOINT were performed successfully.

EX. NO:05 Creation of a join operations using Clients and Sales Table

DATE:

AIM

To create Clients and Sales tables, insert records, and retrieve related data using different types of joins such as Inner Join, Left Join, Right Join, Full Join, and Self Join.

Software Used

Oracle / MySQL / sqlite3

Description

Joins are used to combine records from two or more tables based on a related column between them.

JOIN COMMANDS:

- Inner Join
- Left Join
- Right Join
- Full Join
- Self Join

INNER JOIN

Inner Join returns only the records that have matching values in both tables.

Syntax:

SELECT columns

FROM table1

INNER JOIN table2

ON table1.column = table2.column;

LEFT JOIN

Left Join returns all records from the left table and the matching records from the right table.

Syntax:

SELECT columns

FROM table1

LEFT JOIN table2

ON table1.column = table2.column;

RIGHT JOIN

Right Join returns all records from the right table and the matching records from the left table.

Syntax:

SELECT columns

FROM table1

RIGHT JOIN table2

ON table1.column = table2.column;

FULL JOIN

Full Join returns all records from both tables. It includes matching and non-matching records.

Syntax:

SELECT columns

FROM table1

FULL JOIN table2

ON table1.column = table2.column;

SELF JOIN

Self Join is used to join a table with itself. It is useful when records in the same table are related to each other.

Syntax:

SELECT A.column, B.column

FROM table_name A

JOIN table_name B

ON A.column = B.column;

Explanation

- Create Clients and Sales tables.
- Insert customer and order details into the tables.
- Display matching customer and order records using Inner Join.
- Display all clients and their matching sales using Left Join.
- Display all sales and their matching clients using Right Join.
- Display all matching and non-matching records using Full Join.
- Use Self Join to display staff and their managers.

Create Tables

```
CREATE TABLE clients (
client_id INTEGER PRIMARY
KEY,
    client_name TEXT,
city TEXT
);
CREATE TABLE sales (     order_id
INTEGER PRIMARY KEY,
    client_id
INTEGER,
product TEXT,
amount INTEGER
);
```

Tables Created

Sample Data

```
INSERT INTO clients VALUES (1, 'Riya', 'Coimbatore');
```

INSERT INTO clients VALUES (2, 'Yuvan', 'Madurai');

INSERT INTO clients VALUES (3, 'Sanju', 'Tirunelveli');

INSERT INTO clients VALUES (4, 'Mahi', 'Erode');

INSERT INTO clients VALUES (5, 'Aadhi', 'Nagercoil');

INSERT INTO sales VALUES (101, 1, 'Tablet', 48000);

INSERT INTO sales VALUES (102, 2, 'Camera', 21800);

INSERT INTO sales VALUES (103, 1, 'Earbuds', 2500);

INSERT INTO sales VALUES (104, 3, 'Monitor', 1800);

INSERT INTO sales VALUES (105, 6, 'Cable', 900);

Records Inserted

SQL Query

Query 1: Inner Join

SQL> SELECT clients.customer_name, sales.product, sales.amount

FROM clients

INNER JOIN sales

ON clients.customer_id = sales.customer_id;

Resultant Query

client_name	product	amount
Riya	Tablet	48000
Yuvan	Camera	21800
Riya	Earbuds	2500
Sanju	Monitor	1800

Query 2: Left Join

SQL> SELECT clients.customer_name, sales.product, sales.amount

FROM clients

LEFT JOIN sales

ON clients.customer_id = sales.customer_id;

Resultant Query

client_name	product	amount
Riya	Tablet	48000
Yuvan	Camera	21800
Sanju	Monitor	1800
Mahi	NULL	NULL
Aadhi	NULL	NULL
Riya	Earbuds	2500

Query 3: Right Join

SQL> SELECT clients.customer_name, sales.product, sales.amount

FROM clients

RIGHT JOIN sales

ON clients.customer_id = sales.customer_id;

Resultant Query

client_name	product	amount
Riya	Tablet	48000
Yuvan	Camera	21800
Riya	Earbuds	2500
Sanju	Monitor	1800
NULL	Cable	900

Query 4: Full Join

```
SQL> SELECT clients.customer_name, sales.product, sales.amount
FROM clients
FULL JOIN sales
ON clients.customer_id = sales.customer_id;
```

Resultant Query

client_name	product	amount
Riya	Tablet	48000
Yuvan	Camera	21800
Riya	Earbuds	2500
Sanju	Monitor	1800
Mahi	NULL	NULL
Aadhi	NULL	NULL
NULL	Cable	900

Query 5: Self Join

```
Create Staff Table CREATE
TABLE staff (     staff_id
INTEGER PRIMARY KEY,
   staff_name TEXT,
manager_id INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO staff VALUES (1, 'Riya', NULL);
```

INSERT INTO staff VALUES (2, 'Yuvan', 1);

INSERT INTO staff VALUES (3, 'Sanju', 1);

INSERT INTO staff VALUES (4, 'Mahi', 2);

INSERT INTO staff VALUES (5, 'Aadhi', 2); **Records Inserted**

SQL> SELECT A.staff_name_name AS Employee,

B.staff_name_name AS Manager

FROM staff A

LEFT JOIN staff B

ON A.manager_id = B.staff_name_id;

Resultant Query

Employee	**Manager**
Riya	NULL
Yuvan	Riya
Sanju	Riya
Mahi	Yuvan
Aadhi	Yuvan

RESULT:

Thus, the Clients and Sales tables were created successfully, and the required records were retrieved successfully using Inner Join, Left Join, Right Join, Full Join, and Self Join operations.

EX. NO:06 Creation of a SQL functions using Inventory Items Table

DATE:

Aim

To create a Inventory Items table, insert inventory item records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / sqlite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL FUNCTION TYPES:

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS

Aggregate functions are used to perform calculations on a group of rows and return a single result.

Common aggregate functions are:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

Syntax

SELECT aggregate_function(column_name)

FROM table_name;

STRING FUNCTIONS

String functions are used to perform operations on text or character values.

Common string functions are:

- UPPER()
- LOWER()
- LENGTH()

Syntax

SELECT string_function(column_name)

FROM table_name;

DATE FUNCTIONS

Date functions are used to perform operations on date values.

Common date functions are:

- CURRENT_DATE
- YEAR()
- MONTH()
- DAY()

Syntax

SELECT date_function(date_column)

FROM table_name;

Explanation

- Create a table named inventory_items.
- Insert product details into the table.
- Display all inventory item records.

- Calculate the total number of inventory_items using COUNT().
- Calculate the total and average price using SUM() and AVG().
- Find the highest and lowest price using MAX() and MIN().
- Display inventory item names in uppercase and lowercase using UPPER() and LOWER().
- Find the length of item names using LENGTH().
- Retrieve product details based on the date.

Create Table

```
CREATE TABLE inventory_items
(     item_id INTEGER PRIMARY
KEY,     item_name TEXT,
category TEXT,     price INTEGER,
purchase_date DATE
);
```

Table Created

Sample Data

```
INSERT INTO inventory_items VALUES (1, 'Tablet', 'Gadgets', 42000, '2026-01-12');
INSERT INTO inventory_items VALUES (2, 'Smartwatch', 'Gadgets', 9000, '2026-02-18');
INSERT INTO inventory_items VALUES (3, 'Webcam', 'Peripherals', 3500, '2026-03-22');
INSERT INTO inventory_items VALUES (4, 'Speaker', 'Peripherals', 2800, '2026-04-09');
INSERT INTO inventory_items VALUES (5, 'Router', 'Gadgets', 12000, '2026-05-17');
INSERT INTO inventory_items VALUES (6, 'SSD', 'Peripherals', 6500, '2026-06-25');
```

Records Inserted

SQL Query

Query 1: Display all inventory item records

```
SQL> SELECT * FROM inventory_items;
```

Resultant Query

item_id	item_name	category	price	purchase_date
1	Tablet	Gadgets	42000	2026-01-12
2	Smartwatch	Gadgets	9000	2026-02-18
3	Webcam	Peripherals	3500	2026-03-22
4	Speaker	Peripherals	2800	2026-04-09
5	Router	Gadgets	12000	2026-05-17
6	SSD	Peripherals	6500	2026-06-25

AGGREGATE FUNCTIONS

Query 2: Count the total number of inventory_items

SQL> SELECT COUNT(*) AS Total_Items

FROM inventory_items;

Resultant Query

Total_Items
6

Query 3: Calculate the total price of all inventory_items

SQL> SELECT SUM(price) AS Total_Cost

FROM inventory_items;

Resultant Query

Total_Cost

75800

Query 4: Calculate the average price of inventory_items

SQL> SELECT AVG(price) AS Average_Cost

FROM inventory_items;

Resultant Query

Average_Cost
12633.33

Query 5: Find the highest and lowest item price

SQL> SELECT MAX(price) AS Highest_Cost,

MIN(price) AS Lowest_Cost

FROM inventory_items;

Resultant Query

Highest_Cost	Lowest_Cost
42000	2800

STRING FUNCTIONS

Query 6: Display inventory item names in uppercase

SQL> SELECT item_name, UPPER(item_name) AS Upper_Item_Name

FROM inventory_items;

Resultant Query

item_name	Upper_Item_Name
Tablet	TABLET
Smartwatch	SMARTWATCH
Webcam	WEBCAM
Speaker	SPEAKER
Router	ROUTER
SSD	SSD

Query 7: Display inventory item names in lowercase

SQL> SELECT item_name, LOWER(item_name) AS Lower_Item_Name FROM inventory_items;

Resultant Query

item_name	Lower_Item_Name
Tablet	tablet
Smartwatch	smartwatch
Webcam	webcam
Speaker	speaker
Router	router
SSD	ssd

Query 8: Display the length of each item name

```
SQL> SELECT item_name, LENGTH(item_name) AS Item_Name_Length
FROM inventory_items;
```

Resultant Query

item_name	Item_Name_ Length
Tablet	6
Smartwatch	10
Webcam	6
Speaker	7
Router	6
SSD	3

DATE FUNCTIONS

Query 9: Display the current date

```
SQL> SELECT CURRENT_DATE AS Current_Date;
```

Resultant Query

Current_Date
Current system date

Query 10: Display inventory_items purchased after March 2026

```
SQL> SELECT item_name, purchase_date
FROM inventory_items
WHERE purchase_date > '2026-03-31';
```

Resultant Query

item_name	purchase_date
Speaker	2026-04-09
Router	2026-05-17
SSD	2026-06-25

RESULT:

Thus, the Inventory Items table was created successfully, inventory item records were inserted, and the required data was retrieved successfully using Aggregate Functions, String Functions, and Date Functions.

EX. NO:07 Creation of Subqueries and Nested Queries using Staff Records

DATE:

Aim

To create an Staff Records table, insert employee records, and retrieve data using Subqueries and Nested Queries.

Software Used

Oracle / MySQL / sqlite3

Description

A Subquery is a query written inside another SQL query. It is used to retrieve intermediate results and use them in the main query.

A Nested Query is a query placed inside another query to perform operations based on the result of the inner query.

SUBQUERY TYPES:

- Single Row Subquery
- Multiple Row Subquery
- Nested Subquery

SINGLE ROW SUBQUERY

A Single Row Subquery returns only one value or one row and is used with comparison operators such as =, >, <, >= and <=.

Syntax:

```
SELECT column_name

FROM table_name

WHERE column_name > (SELECT column_name

                     FROM table_name

                     WHERE condition);
```

MULTIPLE ROW SUBQUERY

A Multiple Row Subquery returns more than one value or row. It is commonly used with IN, ANY, and ALL operators.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name IN (SELECT column_name
        FROM table_name
        WHERE condition);
```

NESTED SUBQUERY

A Nested Subquery contains one query inside another query. The result of the inner query is used by the outer query.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name = (SELECT column_name
        FROM table_name
        WHERE condition);
```

Explanation

- Create a table named staff_records.
- Insert employee details into the table.
- Display all employee records.
- Find staff_records whose salary is greater than the average salary.
- Find staff_records who belong to the IT or HR departments.
- Find the employee who has the highest salary.
- Retrieve staff_records based on the result of another query.

Create Table

```
CREATE TABLE staff_records (
staff_id INTEGER PRIMARY
KEY,     staff_name TEXT,
department TEXT,     salary
INTEGER
);
```

Table Created

Sample Data

INSERT INTO staff_records VALUES (1, 'Ananya', 'IT', 52000); INSERT INTO staff_records VALUES (2, 'Rohan', 'HR', 34000);

INSERT INTO staff_records VALUES (3, 'Vishal', 'Finance', 41000);

INSERT INTO staff_records VALUES (4, 'Sneha', 'IT', 56000);

INSERT INTO staff_records VALUES (5, 'Manoj', 'Sales', 30000);

INSERT INTO staff_records VALUES (6, 'Keerthi', 'HR', 46000);

Records Inserted

SQL Query

Query 1: Display all employee records

```
SQL> SELECT * FROM staff_records;
```

Resultant Query

staff_id	staff_name	department	salary
1	Ananya	IT	52000
2	Rohan	HR	34000
3	Vishal	Finance	41000
4	Sneha	IT	56000
5	Manoj	Sales	30000
6	Keerthi	HR	46000

Query 2: Display staff_records whose salary is greater than the average salary

```
SQL> SELECT staff_name, salary
FROM staff_records
WHERE salary > (SELECT AVG(salary)
        FROM staff_records);
```

Resultant Query

staff_name	salary
Ananya	52000
Sneha	56000
Keerthi	46000

Query 3: Display staff_records who belong to IT or HR department

```
SQL> SELECT staff_name, department
FROM staff_records
WHERE department IN (SELECT department
        FROM staff_records
        WHERE department IN ('IT', 'HR'));
```

Resultant Query

staff_name	department
Ananya	IT
Rohan	HR
Sneha	IT
Keerthi	HR

Query 4: Display the employee with the highest salary

```
SQL> SELECT staff_name, salary
FROM staff_records
WHERE salary = (SELECT MAX(salary)
        FROM staff_records);
```

Resultant Query

staff_name	salary
Sneha	56000

Query 5: Display staff_records whose salary is greater than Rohan's salary

```
SQL> SELECT staff_name, salary
FROM staff_records
WHERE salary > (SELECT salary
        FROM staff_records
        WHERE staff_name = 'Rohan');
```

Resultant Query

staff_name	salary
Ananya	52000
Vishal	41000
Sneha	56000
Keerthi	46000

RESULT:

Thus, the Staff Records table was created successfully, employee records were inserted, and the required employee details were retrieved successfully using Subqueries and Nested Queries.

EX. NO:08 Creation of views, indexes and sequences

DATE:

Aim

To create and perform operations on Views, Indexes, and Sequences using SQL commands.

Software Used

Oracle SQL / SQL Command Line

Description

View:
A View is a virtual table created from one or more tables using a SELECT query. It does not store the actual data separately.

Index:
An Index is used to improve the speed of data retrieval from a table.

Sequence:
A Sequence is a database object used to generate unique numeric values automatically.

1. VIEW

Definition

A View is a virtual table based on the result of a SQL query.

Syntax

CREATE VIEW view_name AS

SELECT column1, column2

FROM table_name

WHERE condition;

Create Table

CREATE TABLE team_members (

member_id NUMBER PRIMARY

KEY, member_name

VARCHAR2(50), department

VARCHAR2(30), salary NUMBER

);

Table Created

Insert Records

INSERT INTO team_members VALUES (201, 'Aarthi', 'IT', 48000);

INSERT INTO team_members VALUES (202, 'Naveen', 'HR', 37000);

INSERT INTO team_members VALUES (203, 'Sanjay', 'Finance', 40000);

INSERT INTO team_members VALUES (204, 'Keerthana', 'IT', 53000);

INSERT INTO team_members VALUES (205, 'Harish', 'HR', 43000);

COMMIT;

Records Inserted

Create View

CREATE VIEW tech_team AS

SELECT member_id, member_name, salary

FROM team_members

WHERE department = 'IT';

View Created

Display View

SELECT * FROM tech_team;

Resultant Query

member_id	member_name	salary
201	Aarthi	48000
204	Keerthana	53000

Explanation

The tech_team view displays only the employees who belong to the IT department.

2. INDEX

Definition

An Index is a database object that improves the speed of searching and retrieving records from a table.

Syntax

CREATE INDEX index_name

ON table_name(column_name);

Create Index

CREATE INDEX idx_member_name

ON team_members(member_name);

Index Created

Use Index in Search

SELECT *

FROM team_members

WHERE member_name = 'Aarthi';

Resultant Query

member_id	member_name	department	salary
201	Aarthi	IT	48000

Explanation

The index idx_member_name is created on the member_name column. It helps improve the performance of searches based on employee names.

3. SEQUENCE

Definition

A Sequence is a database object that automatically generates a series of unique numeric values.

Syntax

CREATE SEQUENCE sequence_name

START WITH initial_value

INCREMENT BY increment_value;

Create Sequence

CREATE SEQUENCE member_seq

START WITH 206

INCREMENT BY 1;

Sequence Created

Insert Record Using Sequence

VALUES (member_seq.NEXTVAL, 'Ritesh', 'Sales', 31000);

VALUES (member_seq.NEXTVAL, 'Ritesh', 'Sales', 31000);

COMMIT;

Record Inserted

Display Sequence Value

SELECT member_seq.CURRVAL AS current_sequence

FROM dual;

Resultant Query

current_sequence
206

Insert Another Record Using Sequence

INSERT INTO team_members

VALUES (member_seq.NEXTVAL, 'Pooja', 'Marketing', 37000);

COMMIT;

Record Inserted

Display All Employee Records

SELECT * FROM team_members

ORDER BY member_id;

Resultant Query

member_id	member_name	department	salary
201	Aarthi	IT	48000
202	Naveen	HR	37000
203	Sanjay	Finance	40000
204	Keerthana	IT	53000
205	Harish	HR	43000
206	Ritesh	Sales	31000
207	Pooja	Marketing	37000

Important Commands

Drop View

DROP VIEW tech_team;

View Dropped

Drop Index

DROP INDEX idx_member_name;

Index Dropped

Drop Sequence

DROP SEQUENCE member_seq;

Sequence Dropped



Result

Thus, the View, Index, and Sequence were created and successfully operated using SQL commands. The View was used to display selected records, the Index was used to improve data retrieval, and the Sequence was used to generate unique numeric values automatically.

EX. NO:09 Creation of stored procedures and functions
DATE:

Aim

To create and execute Stored Procedures and Functions using PL/SQL.

Software Used

Oracle SQL / SQL Command Line

Description

Stored Procedure:
A Stored Procedure is a named PL/SQL program stored in the database. It is used to perform a specific task and may or may not return a value.

Function:
A Function is a named PL/SQL program that performs a specific operation and always returns a single value.

1. STORED PROCEDURE

Definition

A Stored Procedure is a stored PL/SQL block that performs a particular operation. It can accept input parameters and can modify or display data.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name
(parameter_name IN datatype)
IS
BEGIN
   statements;
END;/
```

Create Table

```
CREATE TABLE exam_results (
learner_id NUMBER PRIMARY
KEY,     learner_name
VARCHAR2(50),      mark NUMBER
```

);

Table Created

Insert Records

INSERT INTO exam_results VALUES (101, 'Asha', 88);

INSERT INTO exam_results VALUES (102, 'Naveen', 74);

INSERT INTO exam_results VALUES (103, 'Rohit', 93);

INSERT INTO exam_results VALUES (104, 'Keerthi', 67);

INSERT INTO exam_results VALUES (105, 'Sanjana', 96);

COMMIT;

Records Inserted

Display Student Records

SELECT * FROM exam_results;

Resultant Query

learner_id	learner_name	mark
101	Asha	88
102	Naveen	74
103	Rohit	93
104	Keerthi	67
105	Sanjana	96

Create Stored Procedure

The following procedure displays the details of a student based on the learner ID.

```
CREATE OR REPLACE PROCEDURE show_result (
   p_learner_id IN NUMBER
)
```

```
IS     v_name
exam_results.learner_name%TYPE;
v_mark exam_results.mark%TYPE;
BEGIN
   SELECT learner_name, mark
   INTO v_name, v_mark
   FROM exam_results
   WHERE learner_id = p_learner_id;

   DBMS_OUTPUT.PUT_LINE('Learner Name: ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Mark: ' || v_mark);
END;
/
```

Procedure Created

Execute Stored Procedure

```
SET SERVEROUTPUT ON;
EXEC display_student(103);
```

Resultant Query

Learner Name	Mark
Rohit	93

Output

Learner Name: Rohit

Mark: 93

Explanation

The procedure show_result accepts the learner ID as an input parameter and retrieves the corresponding learner name and mark from the exam_results table.

2. FUNCTION

Definition

A Function is a stored PL/SQL program that performs a specific operation and returns a single value.

Syntax

```
CREATE OR REPLACE FUNCTION function_name
(parameter_name IN datatype)
RETURN datatype
IS
BEGIN
   statements;
   RETURN value;
END;
/
```

Create Function

The following function returns the grade based on the student's mark.

```
CREATE OR REPLACE FUNCTION calculate_grade (
p_mark IN NUMBER
)
RETURN VARCHAR2
IS     v_grade
VARCHAR2(5);
BEGIN
```

```
    IF p_mark >= 90 THEN
        v_grade := 'A+';
    ELSIF p_mark >= 80 THEN
        v_grade := 'A';
    ELSIF p_mark >= 70 THEN
        v_grade := 'B';
    ELSIF p_mark >= 60 THEN
        v_grade := 'C';
    ELSIF p_mark >= 50 THEN
        v_grade := 'D';
ELSE
        v_grade := 'F';
    END IF;

    RETURN v_grade;
END;
/
```

Function Created

Execute Function SELECT

learner_id, learner_name, mark,

calculate_grade(mark) AS grade

FROM exam_results;

Resultant Query

learner_id	learner_name	mark	grade
101	Asha	88	A
102	Naveen	74	B
103	Rohit	93	A+
104	Keerthi	67	C
105	Sanjana	96	A+

Explanation

The function calculate_grade accepts the student's mark as an input and returns the corresponding grade. The function is called inside the SELECT statement for each student.

Important Commands

Display Stored Procedure

```
SELECT object_name, object_type
FROM user_objects
WHERE object_name = 'SHOW_RESULT';
```

Resultant Query

object_name	object_type
SHOW_RESULT	PROCEDURE

Display Stored Function

```
SELECT object_name, object_type
FROM user_objects
WHERE object_name = 'CALCULATE_GRADE';
```

Resultant Query

object_name	object_type
CALCULATE_GRADE	FUNCTION

Drop Procedure

DROP PROCEDURE show_result;

Procedure Dropped

Drop Function

DROP FUNCTION calculate_grade;

Function Dropped

Result

Thus, the Stored Procedure and Function were created and executed successfully using PL/SQL. The Stored Procedure was used to retrieve student details, and the Function was used to calculate the grade based on the student's mark.

EX. NO:10 Creation of Triggers method using warehouse stock table

DATE:

Aim

To create and execute Triggers using PL/SQL to automatically perform an action when an INSERT, UPDATE, or DELETE operation occurs on a table.

Software Used

Oracle SQL / SQL Command Line

Description

A Trigger is a stored PL/SQL program that is automatically executed when a specified event occurs on a table or database.

Triggers can be used to automatically perform operations such as inserting, updating, or maintaining records when a particular event takes place.

1. CREATE TABLE

```
CREATE TABLE warehouse_stock (
   stock_id NUMBER PRIMARY
KEY,    item_name
VARCHAR2(50),    quantity
NUMBER,    price NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO warehouse_stock VALUES (301, 'Tablet', 14, 32000);
INSERT INTO warehouse_stock VALUES (302, 'Smartwatch', 22, 6000);
INSERT INTO warehouse_stock VALUES (303, 'Webcam', 18, 4500);
INSERT INTO warehouse_stock VALUES (304, 'Speaker', 10, 3500);
```

INSERT INTO warehouse_stock VALUES (305, 'Router', 27, 2800);

COMMIT;

Records Inserted

Display Stock Records

SELECT * FROM warehouse_stock;

Resultant Query

stock_id	item_name	quantity	price
301	Tablet	14	32000
302	Smartwatch	22	6000
303	Webcam	18	4500
304	Speaker	10	3500
305	Router	27	2800

2. BEFORE INSERT TRIGGER

Definition

A BEFORE INSERT Trigger is executed automatically before a new record is inserted into a table.

Syntax

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE INSERT ON table_name
FOR EACH ROW
BEGIN
   statements;
END;
/
```

Create Trigger

The following trigger checks whether the quantity is negative before inserting a product record.

```
CREATE OR REPLACE TRIGGER check_stock_quantity
BEFORE INSERT ON warehouse_stock
FOR EACH ROW
BEGIN
   IF :NEW.quantity < 0 THEN
RAISE_APPLICATION_ERROR(-20001, 'Stock quantity cannot be negative');
   END IF;
END;
/
```

Trigger Created

Execute Trigger

```
INSERT INTO warehouse_stock
VALUES (306, 'Scanner', -5, 7000);
```

Output

ORA-20001: Stock quantity cannot be negative

The trigger prevents the insertion of an invalid quantity.

3. BEFORE UPDATE TRIGGER

Definition

A BEFORE UPDATE Trigger is executed automatically before an existing record is updated.

Create Trigger

The following trigger prevents the item price from being updated to a negative value.

```
CREATE OR REPLACE TRIGGER check_stock_price
```

```
BEFORE UPDATE OF price ON warehouse_stock
FOR EACH ROW
BEGIN
   IF :NEW.price < 0 THEN
RAISE_APPLICATION_ERROR(-20002, 'Item price cannot be negative');
   END IF;
END;
/
```

Trigger Created

Execute Trigger

```
UPDATE warehouse_stock
SET price = -500
WHERE stock_id = 303;
```

Output

ORA-20002: Item price cannot be negative

The trigger prevents the invalid price from being updated.

4. AFTER INSERT TRIGGER

Definition

An AFTER INSERT Trigger is executed automatically after a new record is successfully inserted into a table.

Create Audit Table

```
CREATE TABLE
stock_audit (     audit_id
NUMBER,     stock_id
```

NUMBER, action_type

VARCHAR2(22)

);

Table Created

Create Trigger

```
CREATE OR REPLACE TRIGGER stock_insert_audit
AFTER INSERT ON warehouse_stock
FOR EACH ROW
BEGIN
   INSERT INTO stock_audit
   VALUES (stock_audit_seq.NEXTVAL, :NEW.stock_id, 'INSERT');
END;
/
```

Trigger Created

Before executing the above trigger, create the required sequence:

```
CREATE SEQUENCE stock_audit_seq
START WITH 1
INCREMENT BY 1;
```

Sequence Created

Insert New Stock

```
INSERT INTO warehouse_stock
VALUES (306, 'Scanner', 8, 7000);
COMMIT;
```

Record Inserted

Display Audit Records

SELECT * FROM stock_audit;

Resultant Query

audit_id	stock_id	action_type
1	306	INSERT

5. DISPLAY FINAL PRODUCT RECORDS

SELECT * FROM warehouse_stock

ORDER BY stock_id;

Resultant Query

stock_id	item_name	quantity	price
301	Tablet	14	32000
302	Smartwatch	22	6000
303	Webcam	18	4500
304	Speaker	10	3500
305	Router	27	2800
306	Scanner	8	7000

Important Commands

Display Triggers

SELECT trigger_name, status

FROM user_triggers;

Resultant Query

trigger_name	status
CHECK_STOCK_QUANTITY	ENABLED
CHECK_STOCK_PRICE	ENABLED
STOCK_INSERT_AUDIT	ENABLED

Drop Trigger

DROP TRIGGER check_stock_quantity;

Trigger Dropped

DROP TRIGGER check_stock_price;

Trigger Dropped

DROP TRIGGER stock_insert_audit;

Trigger Dropped

Result

Thus, the Triggers were created and executed successfully using PL/SQL. The triggers automatically validated stock quantity and price before modification and recorded product insertion details in the audit table.

EX. NO:11 Creation of cursor implementation using library table

DATE:

Aim

To implement and execute a Cursor using PL/SQL to retrieve and process records from a database table.

Software Used

Oracle SQL / SQL Command Line

Description

A Cursor is a pointer used to process the rows returned by a SQL query one row at a time.

Cursors are useful when each record needs to be processed individually.

There are two types of cursors:

1. Implicit Cursor
2. Explicit Cursor

In this practical, an Explicit Cursor is implemented.

1. CREATE TABLE

```
CREATE TABLE library_books (
library_id NUMBER PRIMARY KEY,
   title VARCHAR2(50),
author VARCHAR2(50),
price NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO library_books VALUES (401, 'C Programming', 'Bjarne', 480);
INSERT INTO library_books VALUES (402, 'Data Analytics', 'Nadia', 720);
INSERT INTO library_books VALUES (403, 'Java Basics', 'Herbert', 430);
```

INSERT INTO library_books VALUES (404, 'Web Design', 'Ravi', 680);

INSERT INTO library_books VALUES (405, 'Operating Systems', 'Silberschatz', 610);

COMMIT;

Records Inserted

Display Book Records

SELECT * FROM library_books;

Resultant Query

library_id	title	author	price
401	C Programming	Bjarne	480
402	Data Analytics	Nadia	720
403	Java Basics	Herbert	430
404	Web Design	Ravi	680
405	Operating Systems	Silberschatz	610

2. EXPLICIT CURSOR

Definition

An Explicit Cursor is a cursor that is declared and controlled by the programmer. It is used to retrieve multiple rows from a table and process them one by one.

Syntax

```
DECLARE
   CURSOR cursor_name IS
      SELECT statement;
BEGIN
   OPEN cursor_name;

   LOOP
      FETCH cursor_name INTO variables;
```

```
        EXIT WHEN cursor_name%NOTFOUND;


        statements;
    END LOOP;
    CLOSE cursor_name;
END;
/
```

Create Explicit Cursor

The following cursor retrieves each title and price from the library_books table.

```
SET SERVEROUTPUT ON;


DECLARE
    CURSOR library_cursor IS
        SELECT title, price
FROM library_books;


    v_title  library_books.title%TYPE;
v_price   library_books.price%TYPE;
BEGIN
    OPEN library_cursor;


    LOOP
        FETCH library_cursor INTO v_title, v_price;


        EXIT WHEN library_cursor%NOTFOUND;


        DBMS_OUTPUT.PUT_LINE(
            'Title: ' || v_title || ' | Price: ' || v_price₹
```

```
        );
    END LOOP;

    CLOSE library_cursor;
END;
/
```

Output

Title: C Programming | Price: 480₹

Title: Data Analytics | Price: 720₹

Title: Java Basics | Price: 430₹

Title: Web Design | Price: 680₹

Title: Operating Systems | Price: 610₹

3. CURSOR WITH CONDITION

The following cursor displays only the library books whose price is greater than 500.₹

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR library_cursor IS
        SELECT library_id, title, price
        FROM library_books
WHERE price > 500;
v_library_id
library_books.library_id%TYP
E;      v_title
library_books.title%TYPE;
```

```
v_price
library_books.price%TYPE;
BEGIN
    OPEN library_cursor;

    LOOP
        FETCH library_cursor INTO v_library_id, v_title, v_price;

        EXIT WHEN library_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Library ID: ' || v_library_id ||
            ' | Title: ' || v_title ||
            ' | Price: ' || v_price₹
        );
    END LOOP;
    CLOSE library_cursor;
END;
/
```

Resultant Query

library_i d	title	price
402	Data Analytics	720
404	Web Design	680
405	Operating Systems	610

4. CURSOR ATTRIBUTES

Cursor attributes provide information about the current status of a cursor.

Important Cursor Attributes

Cursor Attribute	Description
%FOUND	Returns TRUE if the last fetch returned a row
%NOTFOUND	Returns TRUE if the last fetch did not return a row
%ROWCOUNT	Returns the number of rows processed
%ISOPEN	Returns TRUE if the cursor is currently open

5. CURSOR USING %ROWCOUNT

The following program displays the total number of library books processed by the cursor.

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR library_cursor IS
      SELECT title, price
FROM library_books;

   v_title  library_books.title%TYPE;
v_price   library_books.price%TYPE;
BEGIN
   OPEN library_cursor;
   LOOP
      FETCH library_cursor INTO v_title, v_price;

      EXIT WHEN library_cursor%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(
         'Title: ' || v_title ||
         ' | Price: ' || v_price₹
      );
```

```
    END LOOP;

    DBMS_OUTPUT.PUT_LINE(
        'Total Library Books Processed: ' || library_cursor%ROWCOUNT
    );

    CLOSE library_cursor;
END;
/
```

Output

```
Title: C Programming | Price: 480₹
Title: Data Analytics | Price: 720₹
Title: Java Basics | Price: 430₹
Title: Web Design | Price: 680₹
Title: Operating Systems | Price: 610₹
Total Library Books Processed: 5
```

Result

Thus, the Cursor was successfully implemented using PL/SQL. The explicit cursor was used to retrieve and process library book records one by one, apply conditions, and count the number of processed records.

EX. NO:12 Creation of PL/SQL Exception Handling using client accounts table

DATE:

Aim

To implement Exception Handling in PL/SQL and handle runtime errors using predefined and user-defined exceptions.

Software Used

Oracle SQL / SQL Command Line

Description

An Exception is an error that occurs during the execution of a PL/SQL program.

Exception Handling is used to handle runtime errors without terminating the program abnormally.

The basic structure of exception handling is:

```
DECLARE
   declarations;
BEGIN
   statements;
EXCEPTION
   WHEN exception_name THEN
      statements;
END;
/
```

Types of Exceptions

Type	Description
Predefined Exception	Built-in exceptions provided by Oracle
User-Defined Exception	Exception created by the programmer

1. CREATE TABLE

```
CREATE TABLE client_accounts (
client_id NUMBER PRIMARY
KEY,     client_name
VARCHAR2(50),      balance
NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO client_accounts VALUES (501, 'Aarav', 28000);
INSERT INTO client_accounts VALUES (502, 'Bhavana', 34000); INSERT INTO
client_accounts VALUES (503, 'Chetan', 22000);
INSERT INTO client_accounts VALUES (504, 'Deepika', 37000);
INSERT INTO client_accounts VALUES (505, 'Eshan', 45000);
COMMIT;
```

Records Inserted

Display Account Records

```
SELECT * FROM client_accounts;
```

Resultant Query

client_id	client_name	balance
501	Aarav	28000
502	Bhavana	34000
503	Chetan	22000
504	Deepika	37000
505	Eshan	45000

2. NO_DATA_FOUND EXCEPTION

Definition

NO_DATA_FOUND occurs when a SELECT INTO statement does not return any record.

Program

```
SET SERVEROUTPUT ON;
DECLARE
   v_name client_accounts.client_name%TYPE;
v_balance client_accounts.balance%TYPE;
BEGIN
   SELECT client_name, balance
   INTO v_name, v_balance
   FROM client_accounts
   WHERE client_id = 599;


   DBMS_OUTPUT.PUT_LINE('Account Holder: ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Balance: ' || v_balance);₹


EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Exception: Account not found.');
END;
/
```

Output

Exception: Account not found.

Exception Handled Successfully

3. TOO_MANY_ROWS EXCEPTION

Definition

TOO_MANY_ROWS occurs when a SELECT INTO statement returns more than one record.

Program

```
SET SERVEROUTPUT ON;
DECLARE
    v_name client_accounts.client_name%TYPE;
BEGIN
    SELECT client_name
    INTO v_name
    FROM client_accounts
    WHERE balance > 25000;

    DBMS_OUTPUT.PUT_LINE('Account Holder: ' || v_name);

EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: More than one account found.'
        );
```

```
END;
/
```

Output

Exception: More than one account found.

Exception Handled Successfully

4. ZERO_DIVIDE EXCEPTION

Definition

ZERO_DIVIDE occurs when a number is divided by zero.

Program

```
SET SERVEROUTPUT ON;
DECLARE
   v_balance NUMBER :=
28000;     v_divisor NUMBER
:= 0;     v_result NUMBER;
BEGIN
   v_result := v_balance / v_divisor;


   DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);


EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: Cannot divide by zero.'
      );
END;
/
```

Output

Exception: Cannot divide by zero.

Exception Handled Successfully

5. USER-DEFINED EXCEPTION

Definition

A User-Defined Exception is an exception created by the programmer to handle a specific business condition.

In this example, withdrawal is allowed only when sufficient balance is available.

Program

```
SET SERVEROUTPUT ON;
DECLARE
   v_balance client_accounts.balance%TYPE;
v_withdraw NUMBER := 50000;
insufficient_balance EXCEPTION;
BEGIN
   SELECT balance
   INTO v_balance
   FROM client_accounts
   WHERE client_id = 501;

   IF v_withdraw > v_balance THEN
      RAISE insufficient_balance;
   END IF;

   UPDATE client_accounts
   SET balance = balance - v_withdraw
   WHERE client_id = 501;

   COMMIT;
```

```
    DBMS_OUTPUT.PUT_LINE('Withdrawal  Successful.');


EXCEPTION
    WHEN insufficient_balance THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: Insufficient balance.'
        );
END;
/
```

Output

Exception: Insufficient balance.

User-Defined Exception Handled Successfully

6. EXCEPTION HANDLING WITH SUCCESSFUL TRANSACTION

The following program performs a valid withdrawal when sufficient balance is available.

```
SET SERVEROUTPUT ON;
DECLARE
    v_balance client_accounts.balance%TYPE;
v_withdraw NUMBER := 5000;
BEGIN
    SELECT balance
    INTO v_balance
    FROM client_accounts
    WHERE client_id = 501;


    IF v_withdraw > v_balance THEN
        RAISE_APPLICATION_ERROR(
```

```
            -20001,
            'Insufficient balance'
        );
    END IF;

    UPDATE client_accounts
    SET balance = balance - v_withdraw
    WHERE client_id = 501;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Withdrawal Successful.');
    DBMS_OUTPUT.PUT_LINE(
        'Withdrawn Amount: ' || v_withdraw₹
    );

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
END;
/
```

Output

Withdrawal Successful.

Withdrawn Amount: 5000₹

Display Updated Account

```
SELECT * FROM client_accounts
```

WHERE client_id = 501;

Resultant Query

client_id	client_name	balance
501	Aarav	23000

Important Predefined Exceptions

Exception	Description
NO_DATA_FOUND	No record is found
TOO_MANY_ROWS	More than one record is returned
ZERO_DIVIDE	Division by zero occurs
DUP_VAL_ON_INDEX	Duplicate value is inserted into a unique/primary key column
VALUE_ERROR	Invalid value or conversion error occurs

Result

Thus, PL/SQL Exception Handling was successfully implemented. The NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, and user-defined exceptions were handled successfully using appropriate exception-handling mechanisms.

EX. NO:13 Creation of Database Normalization using learner subject table

DATE:

Aim

To study and implement Database Normalization up to Boyce-Codd Normal Form (BCNF) to reduce data redundancy and improve data integrity.

Software Used

Oracle SQL / SQL Command Line

Description

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies.

The major normal forms covered are:

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)

1. UNNORMALIZED TABLE

Consider the following Learner Subject table.

Unnormalized Table

The table contains repeated information. For example, the trainer name Mohan is repeated for students taking the C Programming course.

learner_id	learner_name	subject_id	subject_name	trainer_id	trainer_name
601	Asha	S01	C Programming	T01	Mohan
602	Bharat	S02	Database Systems	T02	Nila
603	Charan	S01	C Programming	T01	Mohan
604	Diya	S03	Data Analytics	T03	Suresh

2. FIRST NORMAL FORM (1NF)

Definition

A relation is in 1NF when:

- Each column contains atomic values.
- There are no repeating groups.
- Each row is uniquely identifiable.

The above table already contains atomic values, so it can be considered as the 1NF table.

1NF Table

learner_id	learner_name	subject_id	subject_name	trainer_id	trainer_name
601	Asha	S01	C Programming	T01	Mohan
602	Bharat	S02	Database Systems	T02	Nila
603	Charan	S01	C Programming	T01	Mohan
604	Diya	S03	Data Analytics	T03	Suresh

Primary Key: learner_id, subject_id

3. SECOND NORMAL FORM (2NF)

Definition

A relation is in 2NF when:

1. It is already in 1NF.
2. There is no partial dependency.
3. Every non-key attribute must depend on the complete primary key.

In the 1NF table, some attributes depend only on part of the composite key.

For example:

- learner_name depends only on learner_id.
- subject_name depends only on subject_id.
- trainer_id depends on subject_id.

Therefore, the table is divided into separate tables.

Learner Table

learner_id	learner_name
601	Asha
602	Bharat
603	Charan
604	Diya

Subject Table

subject_id	subject_name	trainer_id
S01	C Programming	T01
S02	Database Systems	T02
S03	Data Analytics	T03

Enrollment Table

learner_id	subject_id
601	S01
602	S02
603	S01
604	S03

The partial dependencies are removed.

Therefore, the tables are in 2NF.

4. THIRD NORMAL FORM (3NF)

Definition

A relation is in 3NF when:

1. It is already in 2NF.
2. There is no transitive dependency.
3. Non-key attributes must depend only on the primary key.

In the Course table:

subject_id → trainer_id

and

trainer_id → trainer_name

Therefore, trainer_name has a transitive dependency on subject_id.

To remove this dependency, the trainer information is separated into another table.

Subject Table

subject_id	subject_name	trainer_id
S01	C Programming	T01
S02	Database Systems	T02
S03	Data Analytics	T03

Trainer Table

trainer_id	trainer_name
T01	Mohan
T02	Nila
T03	Suresh

Enrollment Table

learner_id	subject_id
601	S01
602	S02
603	S01
604	S03

Therefore, the tables are in 3NF.

5. BOYCE-CODD NORMAL FORM (BCNF)

Definition

A relation is in BCNF when, for every non-trivial functional dependency:

The determinant must be a candidate key.

BCNF is a stronger version of 3NF.

Example

Consider an Instructor_Course table.

trainer_id	subject_id
T01	S01
T01	S02
T02	S03
T03	S04

Assume that each instructor teaches only one course.

Functional dependency:

trainer_id → subject_id

Here, trainer_id determines subject_id.

If trainer_id is not a candidate key in the given relationship, the relation violates BCNF.

Therefore, the relation can be decomposed into separate tables.

Trainer Table

trainer_id	subject_id
T01	S01
T02	S03
T03	S04

Subject Table

subject_id
S01
S02
S03
S04

After decomposition, the determinant is treated as a key in its corresponding relation.

Therefore, the relation is decomposed to satisfy BCNF.

Result Thus, the Database Normalization process was studied and implemented up to 1NF, 2NF, 3NF, and BCNF. The normalization process reduces data redundancy, removes dependency-related anomalies, and improves database consistency and integrity.

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

MINI PROJECT

EX. NO:13 **URBAN TRAFFIC AND CONGESTION ANALYSIS**

DATE:

Aim

To develop an **Urban Traffic and Congestion Analysis system** using SQL and PL/SQL to store, manage, process, and analyze traffic records based on vehicle count, average speed, location, time, and weather conditions.

Software Used

- GitHub Codespaces
- SQLite
- Oracle Database
- SQL
- PL/SQL

Description

The Urban Traffic and Congestion Analysis system is developed to monitor and analyze traffic conditions at different locations. The system stores traffic timestamp, location, vehicle count, average speed, and weather conditions.

SQL queries are used to retrieve and analyze traffic information, while PL/SQL procedures and functions are used to classify traffic conditions, retrieve traffic details, generate congestion reports, and handle exceptions.

Database Table

1. TRAFFIC_DATA

COLUMN_NAME	DATA TYPE	CONSTRAINT
TRAFFIC_ID	NUMBER	PRIMARY KEY
TRAFFIC_TIME	DATE	NOT NULL
LOCATION	VARCHAR2(50)	NOT NULL
VEHICLE_COUNT	NUMBER	NOT NULL
AVERAGE_SPEED	NUMBER(6,2)	NOT NULL
WEATHER	VARCHAR2(20)	NOT NULL

TABLE CREATION

TABLE 1

```
CREATE TABLE traffic_data (
    traffic_id NUMBER PRIMARY KEY,
    traffic_time DATE NOT NULL,
    location VARCHAR2(50) NOT NULL,
    vehicle_count NUMBER NOT NULL,
    average_speed NUMBER(6,2) NOT NULL,
    weather VARCHAR2(20) NOT NULL
);
```

Table Created

The **TRAFFIC_DATA** table was created successfully with primary key and NOT NULL constraints.

DATASET INSERTION

TRAFFIC DATA RECORDS

```
INSERT INTO traffic_data VALUES (1,DATE '2026-01-01','Main Road',850,18.5,'Clear');
INSERT INTO traffic_data VALUES (2,DATE '2026-01-01','Main Road',920,16.2,'Clear');
INSERT INTO traffic_data VALUES (3,DATE '2026-01-01','Main Road',700,22.4,'Cloudy');
INSERT INTO traffic_data VALUES (4,DATE '2026-01-01','Central Road',650,25.5,'Clear');
INSERT INTO traffic_data VALUES (5,DATE '2026-01-01','Central Road',780,21.0,'Rain');
INSERT INTO traffic_data VALUES (6,DATE '2026-01-02','Central Road',1050,14.5,'Rain');
INSERT INTO traffic_data VALUES (7,DATE '2026-01-02','Airport Road',450,35.0,'Clear');
INSERT INTO traffic_data VALUES (8,DATE '2026-01-02','Airport
Road',520,31.5,'Cloudy');
INSERT INTO traffic_data VALUES(9,DATE '2026-01-02','Market Road',1100,12.8,'Rain');
INSERT INTO traffic_data VALUES(10,DATE '2026-01-02','Market
Road',980,17.5,'Cloudy');
INSERT INTO traffic_data VALUES(11,DATE '2026-01-03','Main Road',1150,11.5,'Rain');
INSERT INTO traffic_data VALUES(12,DATE '2026-01-03','Main Road',760,20.5,'Clear');
INSERT INTO traffic_data VALUES(13,DATE '2026-01-03','Central
Road',890,18.0,'Cloudy');
INSERT INTO traffic_data VALUES (14,DATE '2026-01-03','Central
Road',720,23.5,'Clear');
```

```
INSERT INTO traffic_data VALUES (15,DATE '2026-01-03','Airport
Road',380,38.0,'Clear');

INSERT INTO traffic_data VALUES (16,DATE '2026-01-04','Airport Road',600,29.5,'Rain');

INSERT INTO traffic_data VALUES (17,DATE '2026-01-04','Market
Road',1250,10.5,'Rain');

INSERT INTO traffic_data VALUES (18,DATE '2026-01-04','Market
Road',1050,13.8,'Cloudy');

INSERT INTO traffic_data VALUES (19,DATE '2026-01-04','Main Road',680,24.0,'Clear');

INSERT INTO traffic_data VALUES (20,DATE '2026-01-04','Main Road',990,16.8,'Cloudy');

INSERT INTO traffic_data VALUES (21,DATE '2026-01-05','Central
Road',1180,12.5,'Rain');

INSERT INTO traffic_data VALUES (22,DATE '2026-01-05','Central
Road',810,19.5,'Cloudy');

INSERT INTO traffic_data VALUES (23,DATE '2026-01-05','Airport
Road',420,36.5,'Clear');

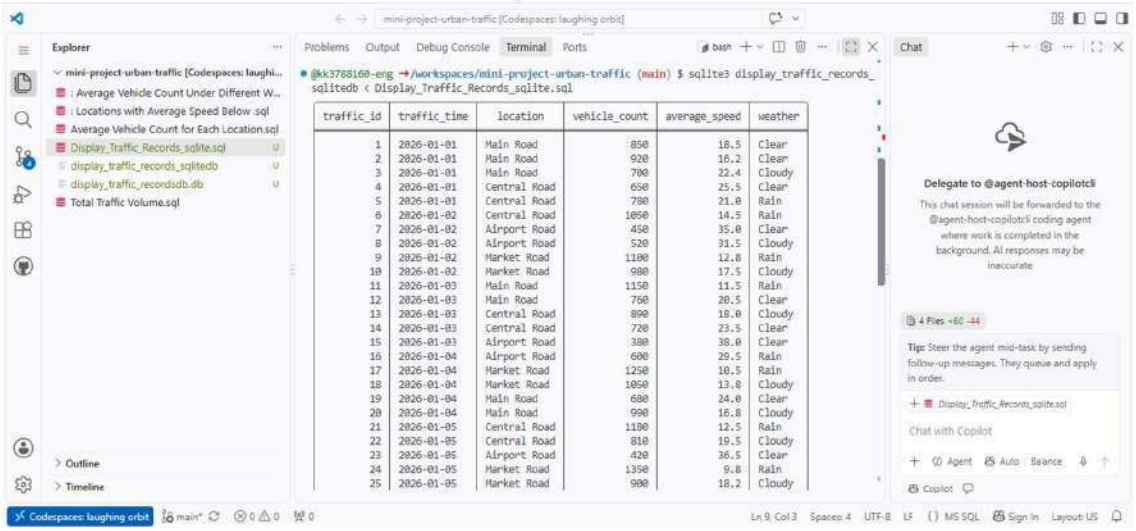
INSERT INTO traffic_data VALUES (24,DATE '2026-01-05','Market Road',1350,9.8,'Rain');

INSERT INTO traffic_data VALUES (25,DATE '2026-01-05','Market
Road',900,18.2,'Cloudy');

COMMIT;
```

Records Inserted

A total of **25 traffic records** were inserted into the **TRAFFIC_DATA** table.



traffic_id	traffic_time	location	vehicle_count	average_speed	weather
1	2026-01-01	Main Road	850	18.5	Clear
2	2026-01-01	Main Road	920	16.2	Clear
3	2026-01-01	Main Road	700	22.4	Cloudy
4	2026-01-01	Central Road	650	25.5	Clear
5	2026-01-01	Central Road	780	21.0	Rain
6	2026-01-02	Central Road	1050	14.5	Rain
7	2026-01-02	Airport Road	450	35.0	Clear
8	2026-01-02	Airport Road	520	31.5	Cloudy
9	2026-01-02	Market Road	1100	12.8	Rain
10	2026-01-02	Market Road	980	17.5	Cloudy
11	2026-01-03	Main Road	1150	11.5	Rain
12	2026-01-03	Main Road	760	20.5	Clear
13	2026-01-03	Central Road	890	18.0	Cloudy
14	2026-01-03	Central Road	720	23.5	Clear
15	2026-01-03	Airport Road	380	38.0	Clear
16	2026-01-04	Airport Road	600	29.5	Rain
17	2026-01-04	Market Road	1250	10.5	Rain
18	2026-01-04	Market Road	1050	13.8	Cloudy
19	2026-01-04	Main Road	680	24.0	Clear
20	2026-01-04	Main Road	990	16.8	Cloudy
21	2026-01-05	Central Road	1180	12.5	Rain
22	2026-01-05	Central Road	810	19.5	Cloudy
23	2026-01-05	Airport Road	420	36.5	Clear
24	2026-01-05	Market Road	1350	9.8	Rain
25	2026-01-05	Market Road	900	18.2	Cloudy

SQL Query 1: Display Traffic Records

```
SELECT location, ROUND(AVG(vehicle_count),2) AS average_vehicle_count
FROM traffic_data
GROUP BY location
ORDER BY location;
```

Output

location	average_vehicle_c...
Airport Road	474.0
Central Road	868.57
Main Road	864.29
Market Road	1105.0

SQL Query 2: Average Vehicle Count for Each Location

```
SELECT location, average_speed
FROM traffic_data
WHERE average_speed < 20
ORDER BY average_speed;
```

Output

location	average_speed
Market Road	9.8
Market Road	10.5
Main Road	11.5
Central Road	12.5
Market Road	12.8
Market Road	13.8
Central Road	14.5
Main Road	16.2
Main Road	16.8
Market Road	17.5
Central Road	18.0
Market Road	18.2
Main Road	18.5
Central Road	19.5

SQL Query 3: Locations with Average Speed Below 20 km/h

```
SELECT location, SUM(vehicle_count) AS total_traffic_volume
```

FROM traffic_data

GROUP BY location

ORDER BY total_traffic_volume DESC;

Output

location	total_traffic_volume
Market Road	6630
Central Road	6080
Main Road	6050
Airport Road	2370

SQL Query 4: Total Traffic Volume

SELECT weather, ROUND(AVG(vehicle_count),2) AS average_vehicle_count

FROM traffic_data

GROUP BY weather

ORDER BY weather**;**

Output

weather	average_vehicle_c...
Clear	647.78
Cloudy	855.0
Rain	1057.5

SQL Query 5: Average Vehicle Count Under Different Weather Conditions

SELECT COUNT(*) AS total_records

FROM traffic_data;

Output

total_records
25

PL/SQL Procedure 1: Classify Traffic Condition

Traffic Classification

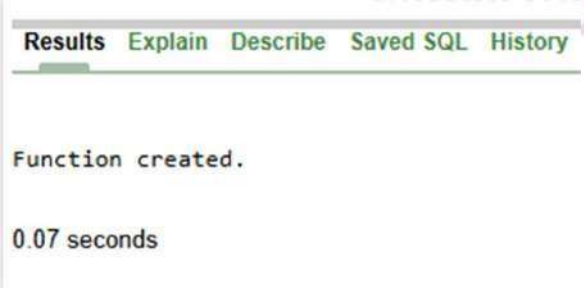
- Below 500 → **Low**
- 500 to 1000 → **Moderate**
- Above 1000 → **Heavy**

```
CREATE OR REPLACE FUNCTION classify_traffic (
   p_vehicle_count IN NUMBER
)
RETURN VARCHAR2
IS
BEGIN
   IF p_vehicle_count < 500 THEN
      RETURN 'Low';

   ELSIF p_vehicle_count <= 1000 THEN
      RETURN 'Moderate';

   ELSE
      RETURN 'Heavy';
   END IF;
END;
/
```

Output



Execution

```
SELECT classify_traffic(350) AS traffic_condition
FROM dual;
```

Output



Another execution:

```
SELECT classify_traffic(850) AS traffic_condition
FROM dual;
```

Output



PL/SQL Procedure 2: Retrieve Traffic Details

```
CREATE OR REPLACE PROCEDURE get_traffic_details (
    p_location IN VARCHAR2,
    p_traffic_date IN DATE
)
IS
BEGIN
    FOR r IN (
        SELECT traffic_id,
               location,
               vehicle_count,
               average_speed,
               weather
        FROM traffic_data
        WHERE location = p_location
        AND traffic_time = p_traffic_date
    )
```

```
    LOOP
        DBMS_OUTPUT.PUT_LINE(
            'ID: ' || r.traffic_id ||
            ' Location: ' || r.location ||
            ' Vehicles: ' || r.vehicle_count ||
            ' Speed: ' || r.average_speed ||
            ' Weather: ' || r.weather
        );
    END LOOP;
END;
/
```

OUTPUT




```
Procedure created.

0.03 seconds
```

Execution

```
BEGIN
    get_traffic_details(
        'Main Road',
        DATE '2026-01-01'
    );
END;
/
```

Output

Results Explain Describe Saved SQL History

```
ID: 1 Location: Main Road Vehicles: 850 Speed: 18.5 Weather: Clear
ID: 2 Location: Main Road Vehicles: 920 Speed: 16.2 Weather: Clear
ID: 3 Location: Main Road Vehicles: 700 Speed: 22.4 Weather: Cloudy

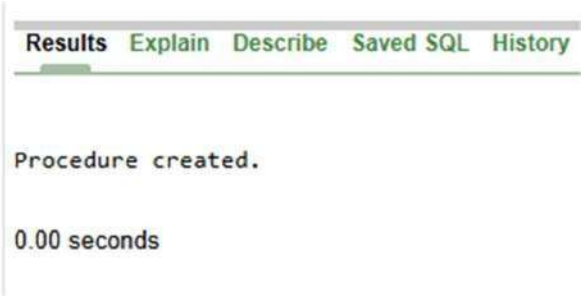
Statement processed.

0.00 seconds
```

PL/SQL Procedure 3: Traffic Congestion Report

```
CREATE OR REPLACE PROCEDURE traffic_congestion_report
IS
BEGIN
   FOR r IN (
      SELECT location,
           ROUND(AVG(vehicle_count),2) AS avg_vehicle_count,
           ROUND(AVG(average_speed),2) AS avg_speed
      FROM traffic_data
      GROUP BY location
      ORDER BY location
   )
   LOOP
      DBMS_OUTPUT.PUT_LINE(
         'Location: ' || r.location ||
         ' Average Vehicles: ' || r.avg_vehicle_count ||
         ' Average Speed: ' || r.avg_speed
      );
   END LOOP;
END;
/
```

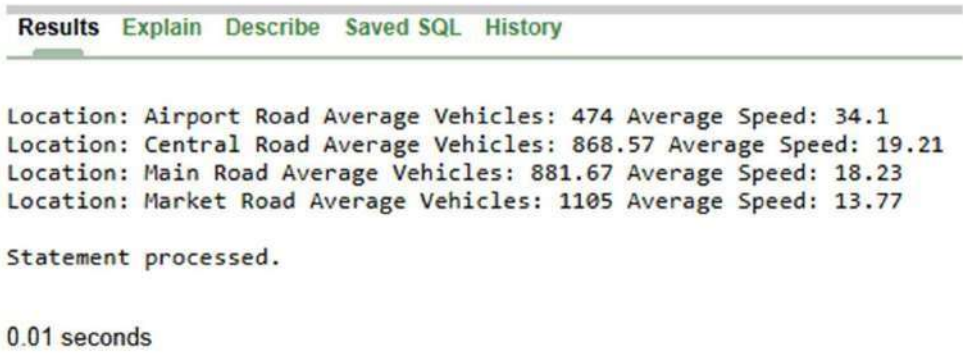
Output



Execution

```
BEGIN
   traffic_congestion_report;
END;
/
```

Output



Traffic Congestion Report

```
SELECT location,
      COUNT(*) AS total_records,
      SUM(vehicle_count) AS total_vehicles,
      ROUND(AVG(average_speed),2) AS average_speed
FROM traffic_data
GROUP BY location
ORDER BY location;
```

Output

Results Explain Describe Saved SQL History

LOCATION	TOTAL_RECORDS	TOTAL_VEHICLES	AVERAGE_SPEED
Airport Road	5	2370	34.1
Central Road	7	6080	19.21
Main Road	6	5290	18.23
Market Road	6	6630	13.77

4 rows returned in 0.00 seconds CSV Export

Exception Handling

Test Case 1 – Invalid Location

```
BEGIN
   get_traffic_details(
      'Unknown Road',
      DATE '2026-01-01'
   );
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Traffic record not found.');
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Statement processed.

0.02 seconds
```

Test Case 2 – Invalid Vehicle Count

```
SELECT classify_traffic(-10)
FROM dual;
```

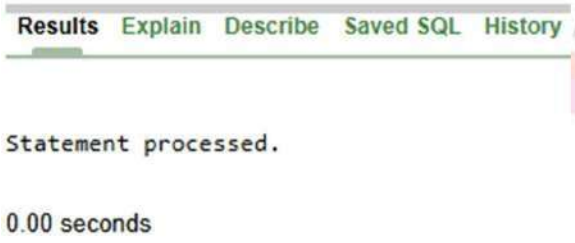
Output



Test Case 3 – Invalid Location for Report

```
BEGIN
   get_traffic_details(
      'Unknown Location',
      DATE '2026-05-01'
   );
END;
/
```

Output



Dataset Insights

- The dataset contains **25 traffic records**.
- The traffic data covers **Main Road, Central Road, Airport Road, and Market Road**.
- Traffic conditions vary based on **vehicle count and average speed**.
- **Market Road** has the highest total vehicle volume.
- **Airport Road** has comparatively higher average speed.
- Rainy weather shows a higher average vehicle count in the dataset.
- Locations with high vehicle count and low average speed indicate **heavy congestion**.
- SQL aggregation helps analyze traffic volume by location and weather.

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

Result

Thus, the **Urban Traffic and Congestion Analysis system** was successfully implemented using SQL and PL/SQL. The traffic table was created, **25 records were inserted**, SQL queries were executed successfully, PL/SQL functions and procedures were implemented, traffic congestion reports were generated, and exception handling was tested successfully.

KHAN ACADEMY

1.BOX OFFICE HITS DATABASE

PROGRAM

```
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009);

INSERT INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);

INSERT INTO movies VALUES (4, "Shrek 2", 2004);

INSERT INTO movies VALUES (5, "The Lion King", 1994);

INSERT INTO movies VALUES (6, "Disney's Up", 2009);

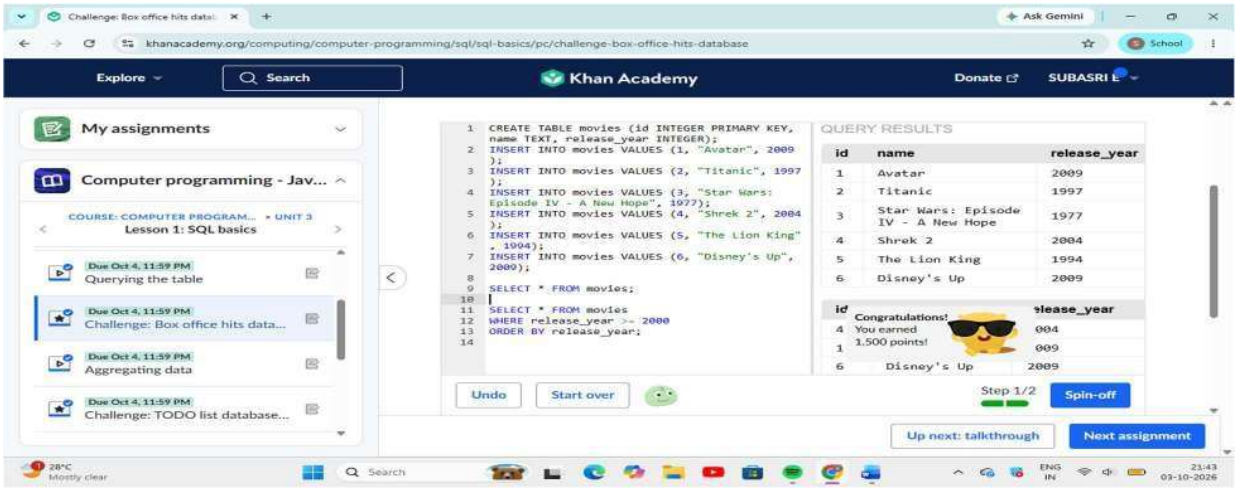
 SELECT * FROM movies;


SELECT * FROM movies

WHERE release_year >= 2000

ORDER BY release_year;
```

OUTPUT

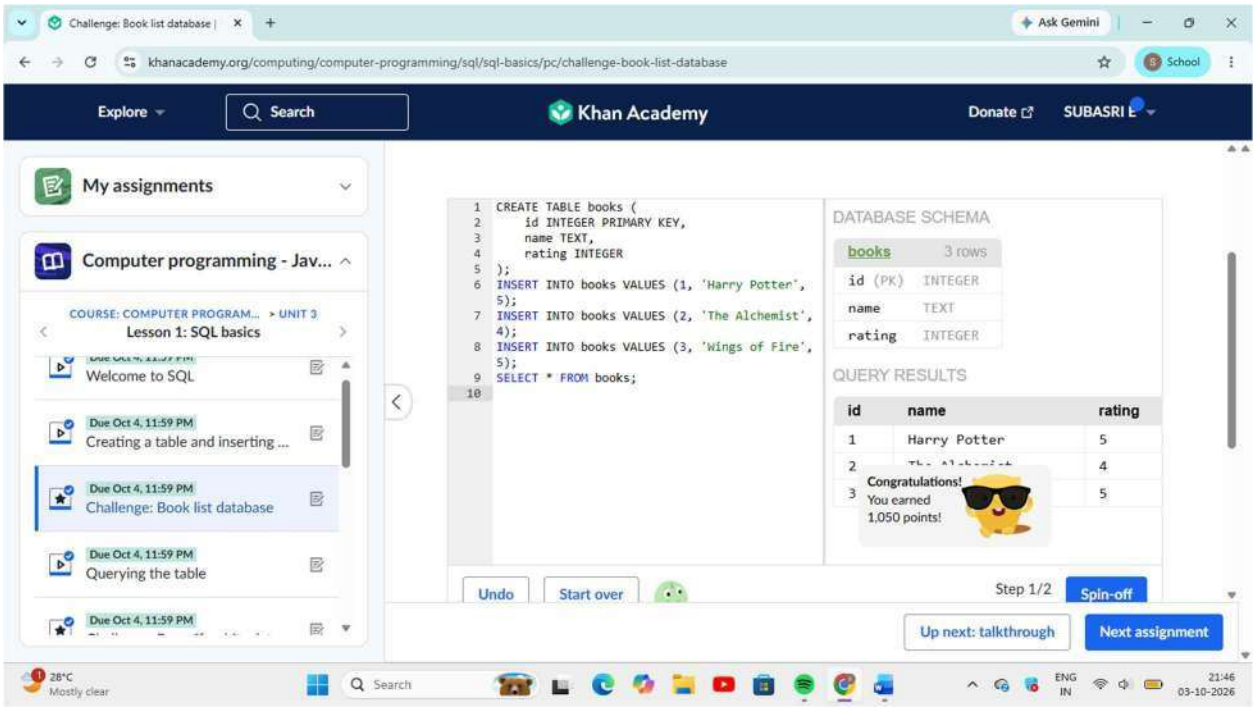


2.BOOK LIST DATABASE

PROGRAM

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    name TEXT,
    rating INTEGER
);
INSERT INTO books VALUES (1, 'Harry Potter', 5);
INSERT INTO books VALUES (2, 'The Alchemist', 4);
INSERT INTO books VALUES (3, 'Wings of Fire', 5);
SELECT * FROM books;
```

OUTPUT



3.TODO LIST DATABASE STATS

PROGRAM

CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

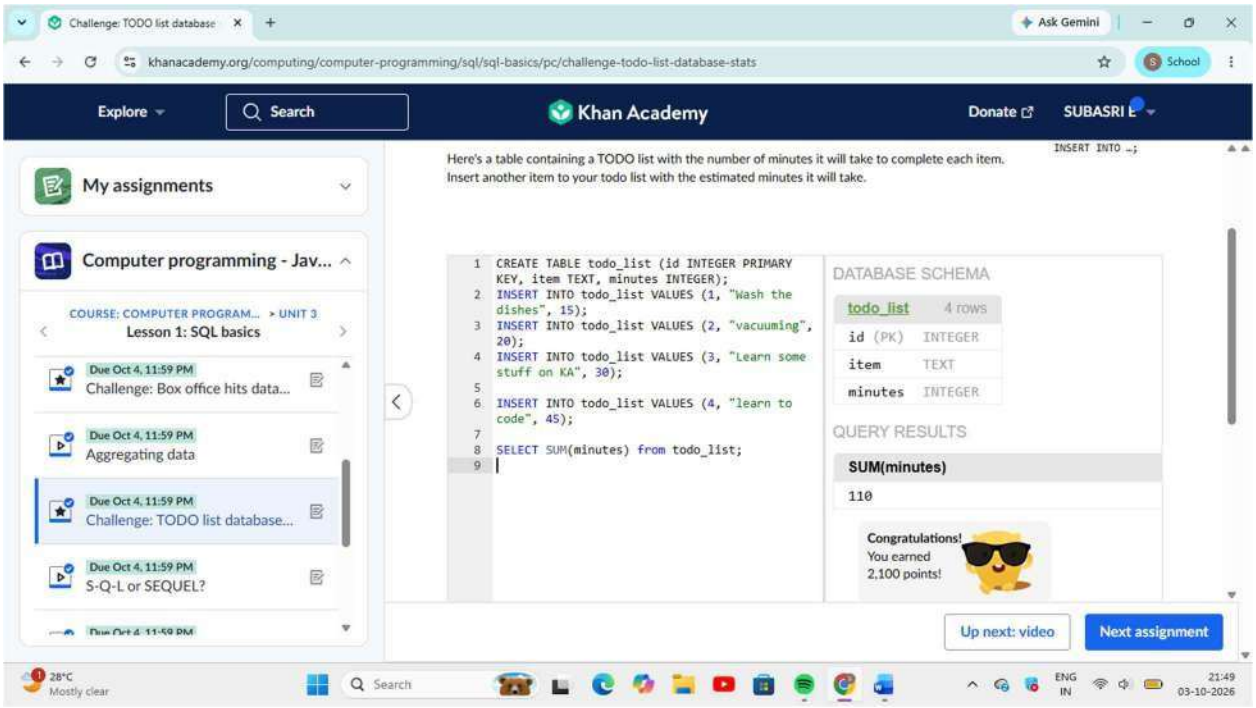
INSERT INTO todo_list VALUES (2, "vacuuming", 20);

INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4, "learn to code", 45);

SELECT SUM(minutes) from todo_list;

OUTPUT



4.KARAOKE SONG SELECTOR

PROGRAM

```
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
    mood TEXT,
    duration INTEGER,
    released INTEGER);


INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);
INSERT INTO songs (title, artist, mood, duration, released)
```

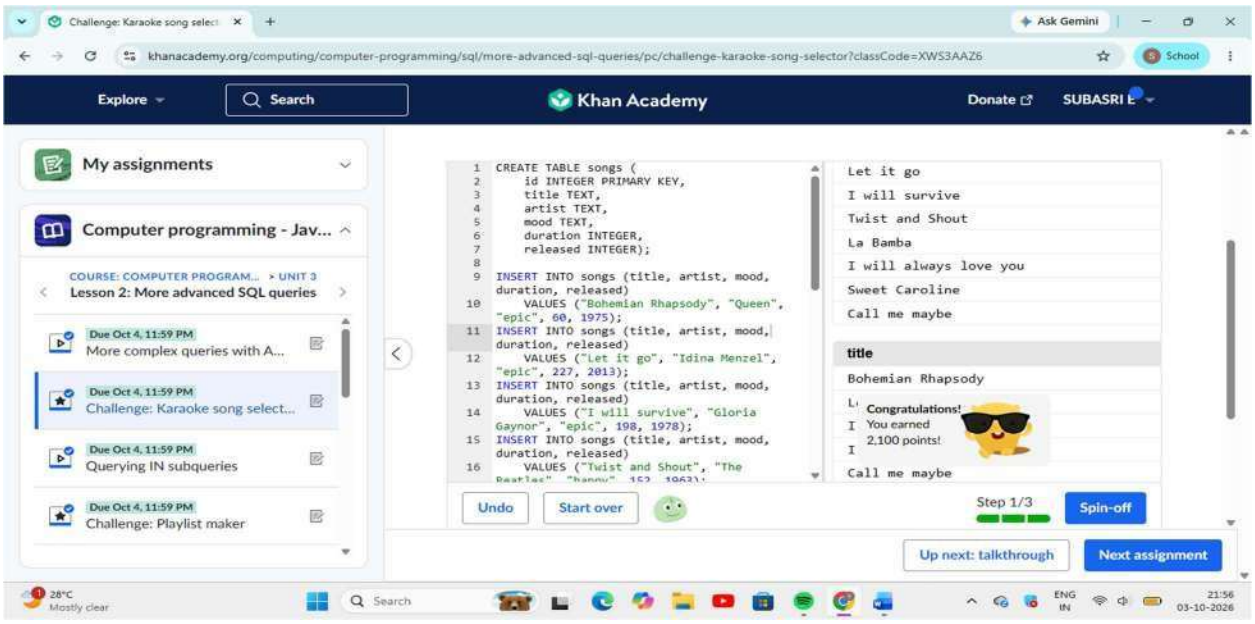
```
    VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);


    SELECT title
FROM songs;


SELECT title
FROM songs
WHERE mood = 'epic'
   OR released > 1990;


   SELECT title
FROM songs
WHERE mood = 'epic'
  AND released > 1990
  AND duration < 240;
```

OUTPUT



5.PLAYLIST MAKER

PROGRAM

```
CREATE TABLE artists (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    country TEXT,
    genre TEXT);


INSERT INTO artists (name, country, genre)
   VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
    VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
```

```
    VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Guns N' Roses", "US", "Hard rock");
 INSERT INTO artists (name, country, genre)
    VALUES ("Gloria Estefan", "US", "Pop");
 INSERT INTO artists (name, country, genre)
    VALUES ("Bob Marley", "Jamaica", "Reggae");

CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT,
    title TEXT);

INSERT INTO songs (artist, title)
    VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
    VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
```

```
    VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
    VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
    VALUES ("Guns N' Roses", "Don't cry");



SELECT title
FROM songs
WHERE artist = 'Queen';

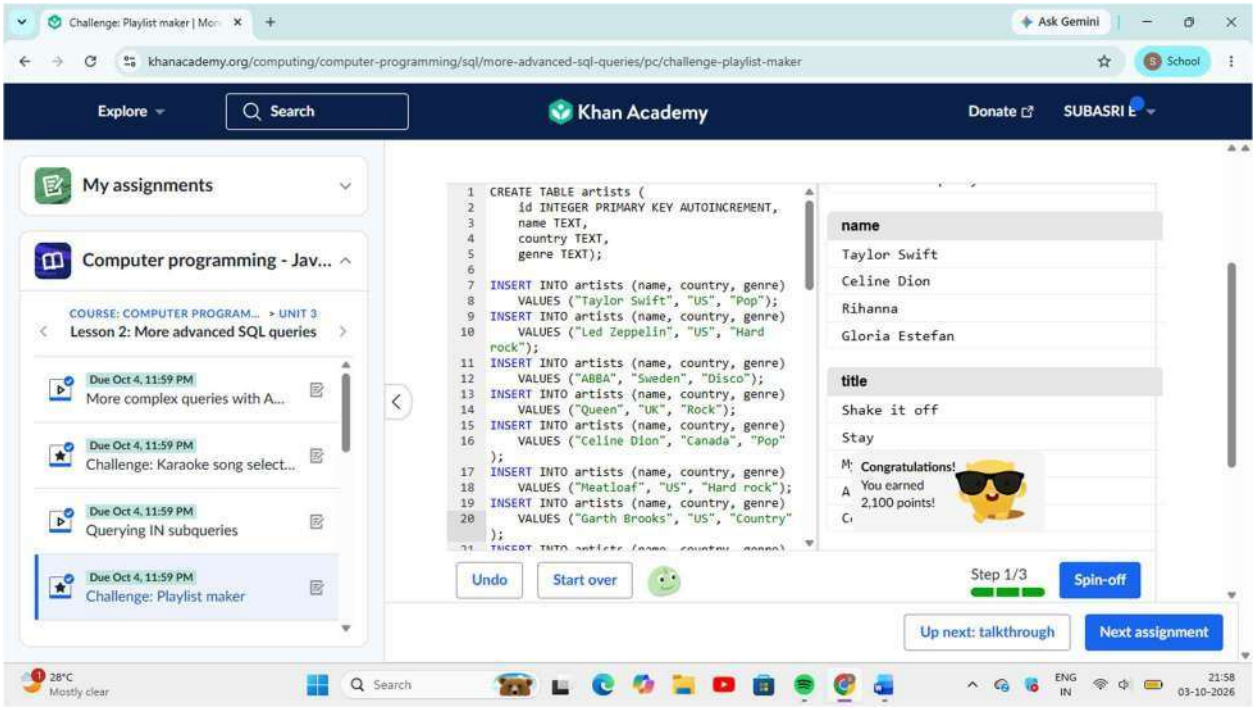


SELECT name
FROM artists
WHERE genre = 'Pop';

SELECT title
FROM songs
WHERE artist IN (
    SELECT name
    FROM artists
    WHERE genre = 'Pop'
```

);

OUTPUT


Challenge: Playlist maker | Mo
Ask Gemini
khanacademy.org/computing/computer-programming/sql/more-advanced-sql-queries/pc/challenge-playlist-maker
School
Explore
Search
Khan Academy
Donate
SUBASRI E
My assignments
Computer programming - Jav...
COURSE: COMPUTER PROGRAM... > UNIT 3
Lesson 2: More advanced SQL queries
Due Oct 4, 11:59 PM
More complex queries with A...
Due Oct 4, 11:59 PM
Challenge: Karaoke song select...
Due Oct 4, 11:59 PM
Querying IN subqueries
Due Oct 4, 11:59 PM
Challenge: Playlist maker
CREATE TABLE artists (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,
country TEXT,
genre TEXT);
INSERT INTO artists (name, country, genre)
VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
VALUES ("Garth Brooks", "US", "Country");
name
Taylor Swift
Celine Dion
Rihanna
Gloria Estefan
title
Shake it off
Stay
Congratulations!
You earned
2,100 points!
Undo
Start over
Step 1/3
Spin-off
Up next: talkthrough
Next assignment
28°C
Mostly clear
Search
ENG
IN
21:58
03-10-2026

6. THE WORDIEST AUTHOR

PROGRAM

```
CREATE TABLE books (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   author TEXT,
   title TEXT,
   words INTEGER);


INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
   VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);


INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Twilight", 118501);
```

```
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);

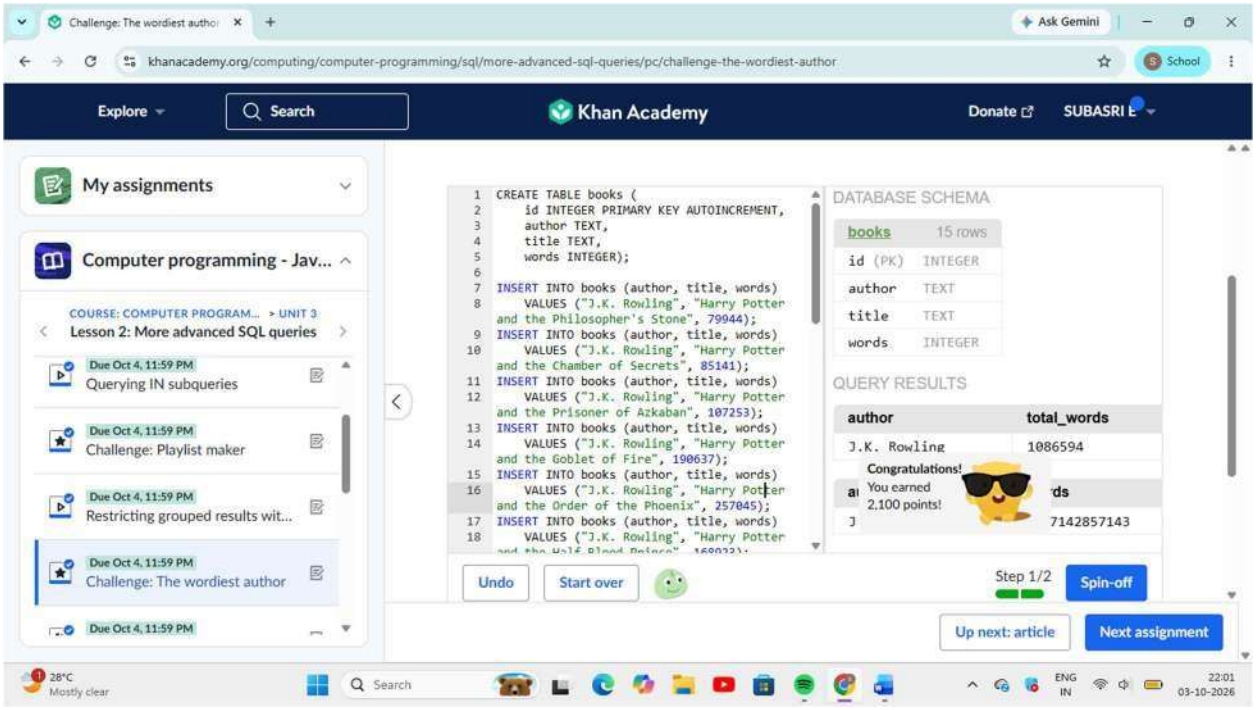
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Return of the King", 134462);

 SELECT author,
      SUM(words) AS total_words
FROM books
GROUP BY author
HAVING SUM(words) > 1000000;

SELECT author,
      AVG(words) AS avg_words
FROM books
GROUP BY author
```

HAVING AVG(words) > 150000;

OUTPUT



7.GRADEBOOK

PROGRAM

```
CREATE TABLE student_grades (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT,
   number_grade INTEGER,
   fraction_completed REAL);

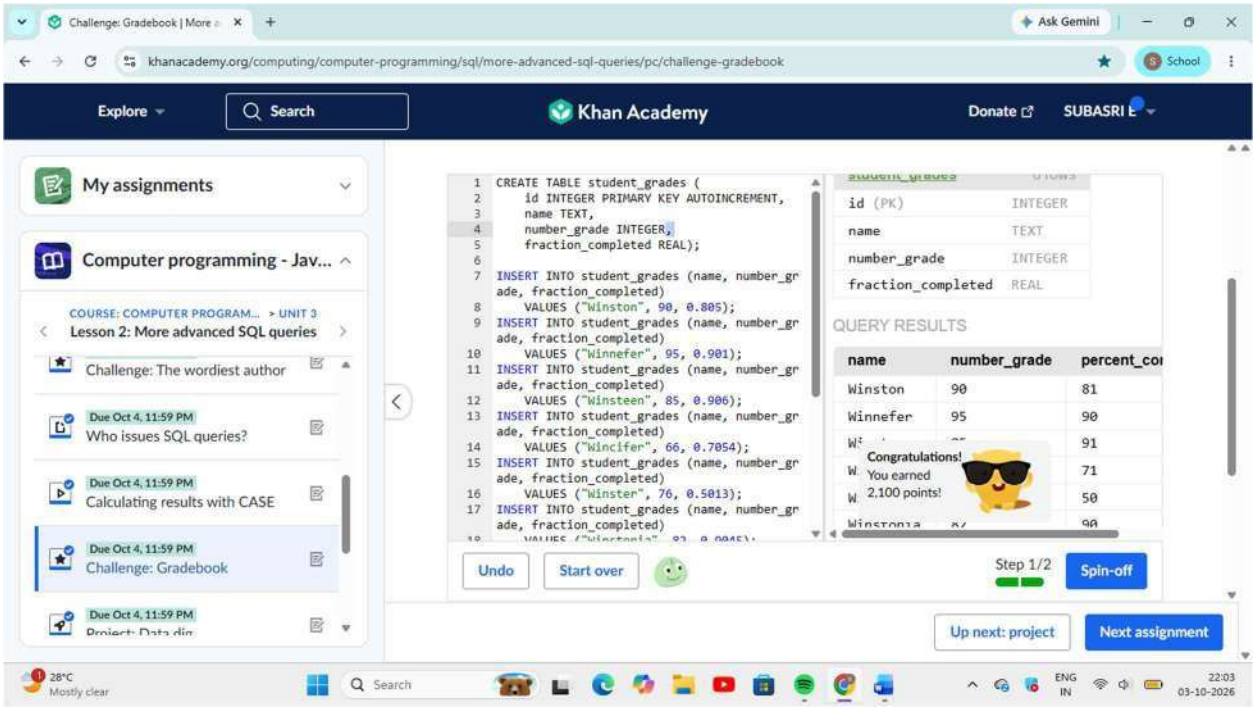

INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
   VALUES ("Winstonia", 82, 0.9045);


SELECT name, number_grade, ROUND(100 * fraction_completed) AS percent_completed
FROM student_grades;


SELECT COUNT(*) AS number_of_students,
```

```
CASE
    WHEN number_grade > 90 THEN "A"
    WHEN number_grade > 80 THEN "B"
    WHEN number_grade > 70 THEN "C"
    ELSE "F"
END AS letter_grade
FROM student_grades
GROUP BY letter_grade;
```

OUTPUT



8. BOBBY'S HOBBIES

PROGRAM

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER
);


-- Insert people
INSERT INTO persons (name, age) VALUES ('Bobby McBobbyFace', 12);
INSERT INTO persons (name, age) VALUES ('Lucy BoBucie', 25);
INSERT INTO persons (name, age) VALUES ('Banana FoFanna', 14);
INSERT INTO persons (name, age) VALUES ('Shish Kabob', 20);
INSERT INTO persons (name, age) VALUES ('Fluffy Sparkles', 8);


-- Create hobbies table
CREATE TABLE hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT
);


-- Insert hobbies
INSERT INTO hobbies (person_id, name) VALUES (1, 'drawing');
INSERT INTO hobbies (person_id, name) VALUES (1, 'coding');
```

```
INSERT INTO hobbies (person_id, name) VALUES (2, 'dancing');
INSERT INTO hobbies (person_id, name) VALUES (2, 'coding');
INSERT INTO hobbies (person_id, name) VALUES (3, 'skating');
INSERT INTO hobbies (person_id, name) VALUES (3, 'rowing');
INSERT INTO hobbies (person_id, name) VALUES (3, 'drawing');
INSERT INTO hobbies (person_id, name) VALUES (4, 'coding');
INSERT INTO hobbies (person_id, name) VALUES (4, 'dilly-dallying');
INSERT INTO hobbies (person_id, name) VALUES (4, 'meowing');

-- Add one more person
INSERT INTO persons (name, age)
VALUES ('John Doe', 30);

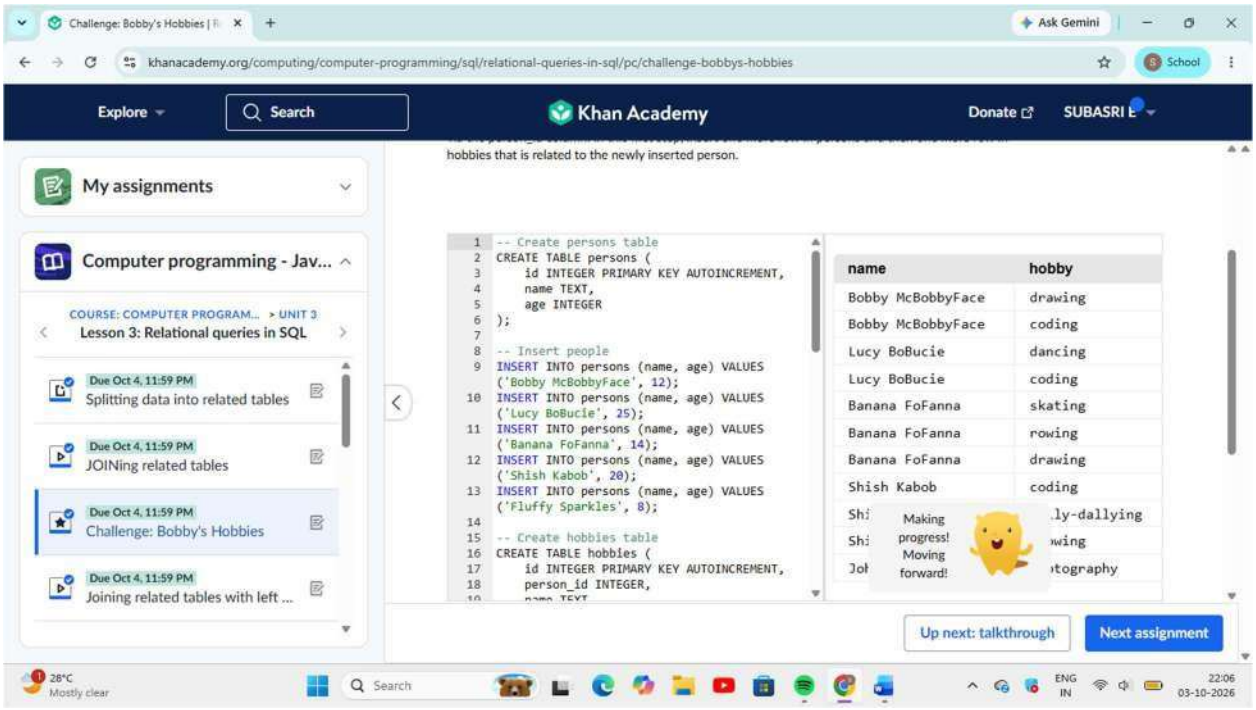
-- Add a hobby for the new person (John Doe gets id = 6)
INSERT INTO hobbies (person_id, name)
VALUES (6, 'Photography');

-- Display each person with their hobby
SELECT
   persons.name,
   persons.age,
   hobbies.name AS hobby
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT
```

```
    persons.name,
    hobbies.name AS hobby
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
```


```
SELECT
    persons.name,
    hobbies.name AS hobby
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
```


OUTPUT



9.CUSTOMER'S ORDERS

PROGRAM

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT);

INSERT INTO customers (name, email) VALUES ("Doctor Who", "doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome", "captain@awesome.com");

CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER,
    item TEXT,
    price REAL);

INSERT INTO orders (customer_id, item, price)
    VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "TARDIS", 1000000.00);

SELECT customers.name, customers.email, orders.item, orders.price
```

```
FROM customers

LEFT OUTER JOIN orders

ON customers.id = orders.customer_id;


SELECT customers.name, customers.email, SUM(orders.price) AS total_spent

FROM customers

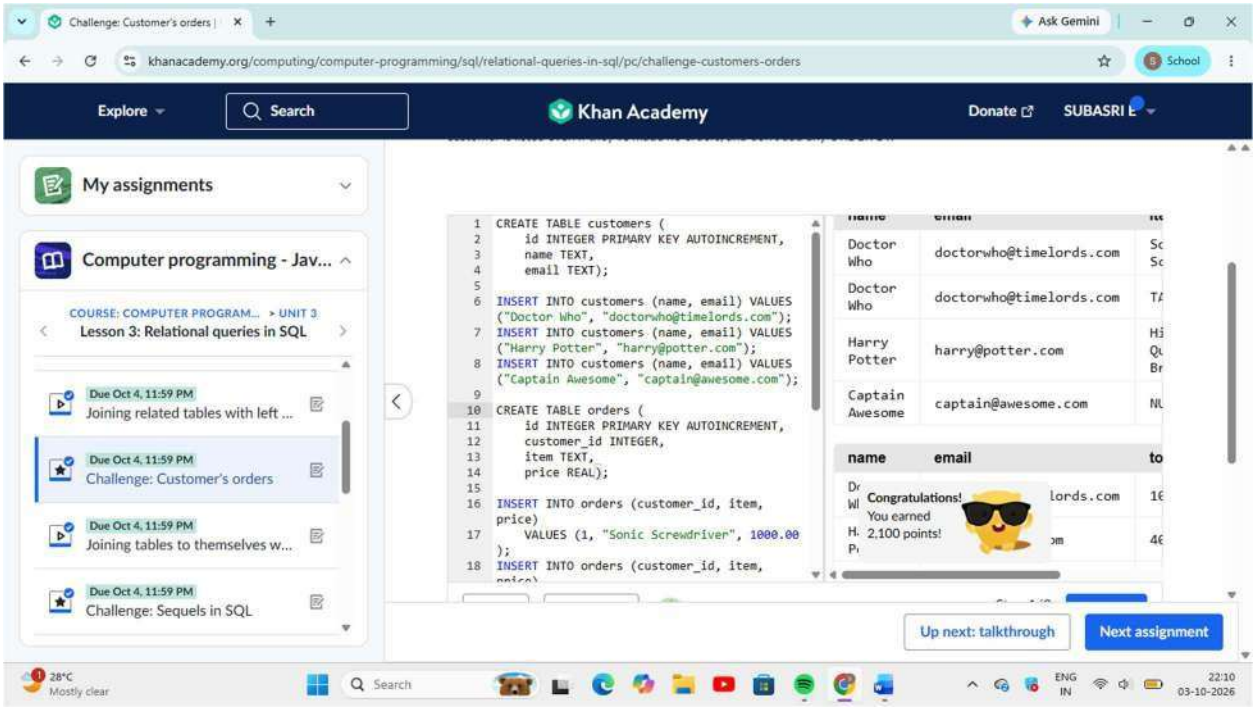
LEFT OUTER JOIN orders

ON customers.id = orders.customer_id

GROUP BY customers.id, customers.name, customers.email

ORDER BY total_spent DESC;
```

OUTPUT



10. SEQUELS IN SQL

PROGRAM

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT,
   released INTEGER,
   sequel_id INTEGER);


INSERT INTO movies
   VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
   VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
   VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
   VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
   VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
INSERT INTO movies
   VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
   VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
   VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);

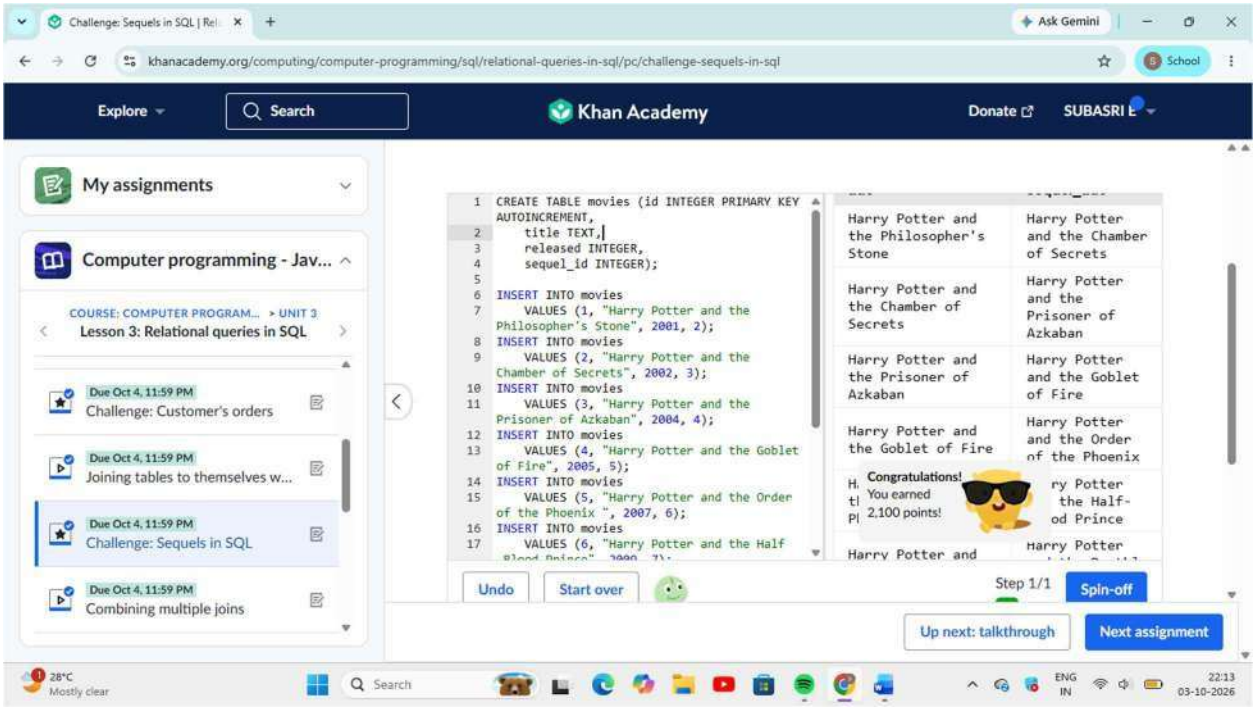

SELECT movies.title, sequel.title AS sequel_title
```

```
FROM movies

LEFT OUTER JOIN movies sequel

ON movies.sequel_id = sequel.id;
```

OUTPUT



11. DYNAMIC DOCUMENTS

PROGRAM

```
CREATE table documents (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT,
   content TEXT,
   author TEXT);


INSERT INTO documents (author, title, content)
   VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
   VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
   VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
INSERT INTO documents (author, title, content)
   VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post.");
INSERT INTO documents (author, title, content)
   VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates,
heights.");


SELECT * FROM documents;


UPDATE documents
SET author = "Jackie Draper"
WHERE author = "Jackie Paper";
```

SELECT *

FROM documents;

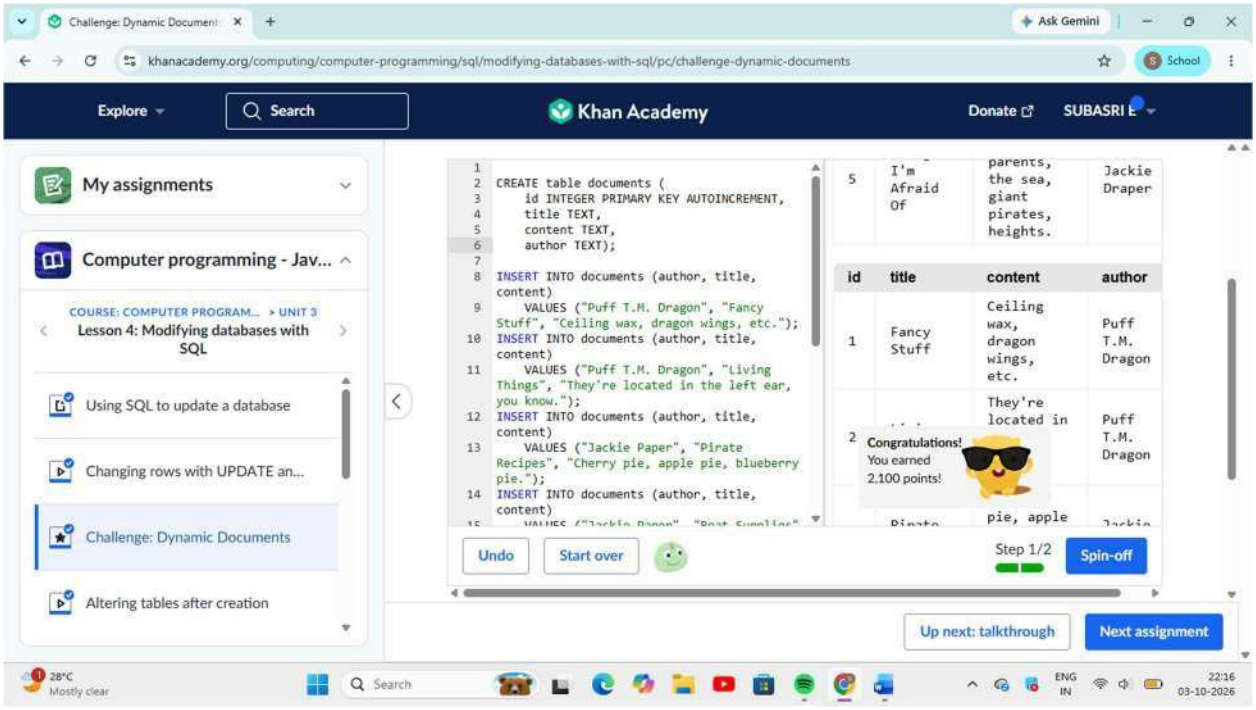
DELETE FROM documents

WHERE title = "Things I'm Afraid Of";

SELECT *

FROM documents;

OUTPUT



12. CLOTHING ALTERATIONS

PROGRAM

```
CREATE TABLE clothes (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT,
   design TEXT
);


INSERT INTO clothes (type, design)
VALUES ("dress", "pink polka dots");


INSERT INTO clothes (type, design)
VALUES ("pants", "rainbow tie-dye");


INSERT INTO clothes (type, design)
VALUES ("blazer", "black sequin");


ALTER TABLE clothes
ADD price INTEGER;


SELECT *
FROM clothes;


UPDATE clothes
SET price = 10
```

```
WHERE id = 1;

UPDATE clothes
SET price = 20
WHERE id = 2;

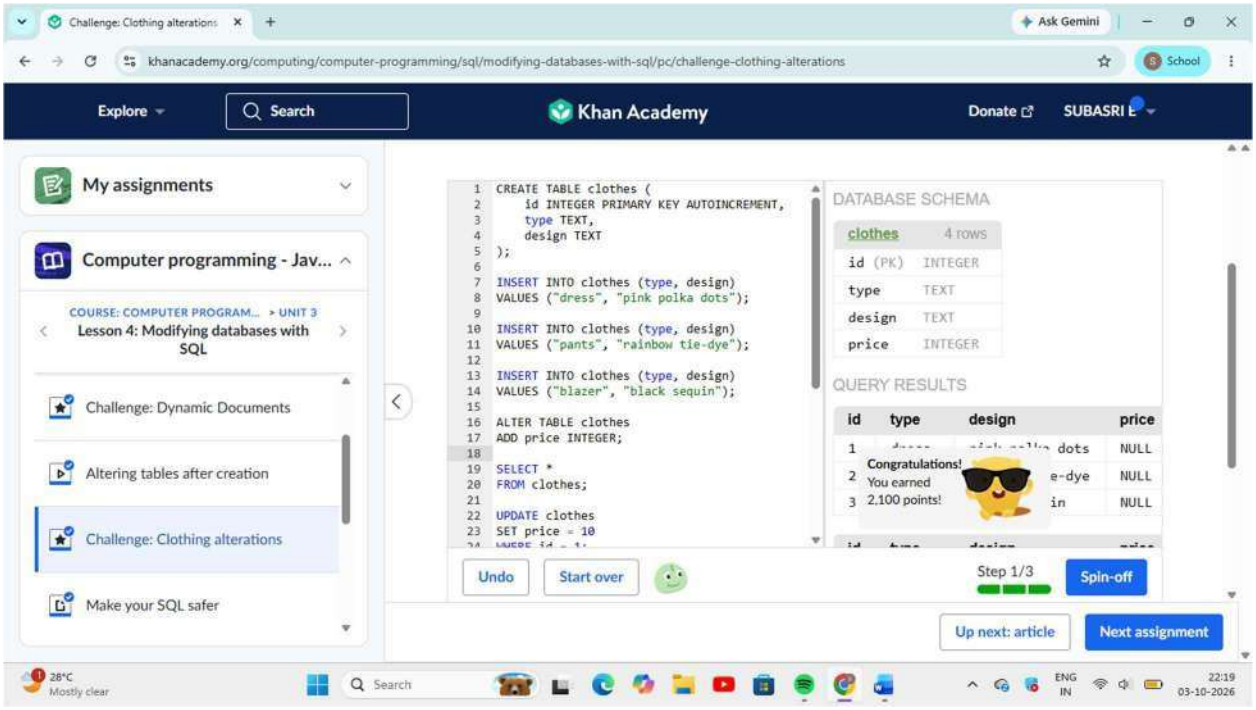
UPDATE clothes
SET price = 30
WHERE id = 3;

SELECT *
FROM clothes;

INSERT INTO clothes (type, design, price)
VALUES ("Hat", "plain", 40);

SELECT *
FROM clothes;
```

OUTPUT


Challenge: Clothing alterations
Ask Gemini
khanacademy.org/computing/computer-programming/sql/modifying-databases-with-sql/pc/challenge-clothing-alterations
School
Explore
Search
Khan Academy
Donate
SUBASRI E
My assignments
Computer programming - Jav...
COURSE: COMPUTER PROGRAM... > UNIT 3
Lesson 4: Modifying databases with SQL
Challenge: Dynamic Documents
Altering tables after creation
Challenge: Clothing alterations
Make your SQL safer
CREATE TABLE clothes (
id INTEGER PRIMARY KEY AUTOINCREMENT,
type TEXT,
design TEXT
);
INSERT INTO clothes (type, design)
VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
VALUES ("blazer", "black sequin");
ALTER TABLE clothes
ADD price INTEGER;
SELECT *
FROM clothes;
UPDATE clothes
SET price = 10
DATABASE SCHEMA
clothes 4 rows
id (PK) INTEGER
type TEXT
design TEXT
price INTEGER
QUERY RESULTS
id type design price
1 dots NULL
2 e-dye NULL
3 in NULL
Congratulations!
You earned
2,100 points!
Undo
Start over
Step 1/3
Spin-off
Up next: article
Next assignment
28°C
Mostly clear
Search
ENG
IN
22:19
03-10-2026

13. CREATING A TABLE AND INSERTING DATA

PROGRAM

CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER);

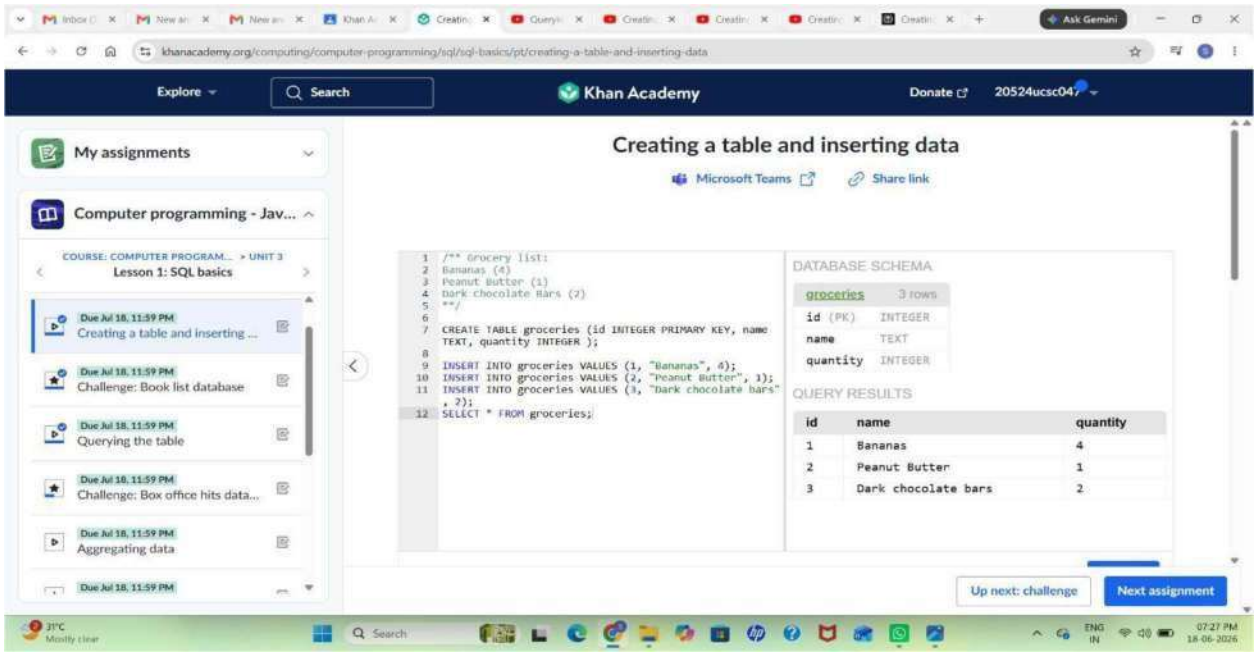
INSERT INTO groceries VALUES (1, "Bananas", 4); INSERT INTO groceries VALUES (2, "Peanut Butter", 1);

INSERT INTO groceries VALUES (3, "Dark

chocolate bars", 2);

SELECT * FROM groceries;

OUTPUT



14. QUERYING THE TABLE

PROGRAM

CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER, aisle INTEGER);

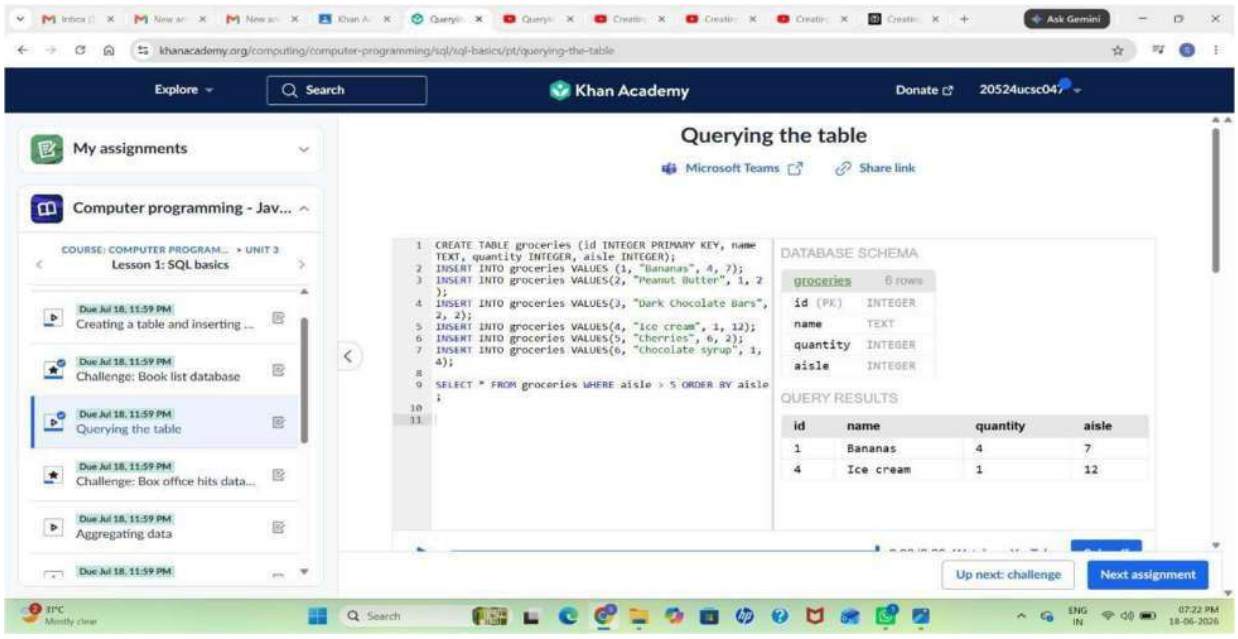
INSERT INTO groceries VALUES (1, "Bananas", 4, 7); INSERT INTO groceries VALUES(2, "Peanut Butter", 1, 2);

INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2); INSERT INTO groceries VALUES(4, "Ice cream", 1, 12);

INSERT INTO groceries VALUES(5, "Cherries", 6, 2); INSERT INTO groceries VALUES(6, "Chocolate syrup", 1, 4);

SELECT * FROM groceries WHERE aisle > 5 ORDER BY aisle;

OUTPUT



15. PROJECT: DESIGN A STORE DATABASE

PROGRAM

CREATE TABLE books (

id INTEGER PRIMARY KEY,

name TEXT, author TEXT, genre TEXT, price REAL, stock

INTEGER

);

INSERT INTO books VALUES (1, "Harry Potter 1", "J.K. Rowling", "Fantasy", 12.99, 50);

INSERT INTO books VALUES (2, "Harry Potter 2", "J.K. Rowling", "Fantasy", 13.99, 45);

INSERT INTO books VALUES (3, "Harry Potter 3", "J.K. Rowling", "Fantasy", 14.99, 40);

INSERT INTO books VALUES (4, "The Hobbit", "J.R.R. Tolkien", "Fantasy", 10.99, 30);

INSERT INTO books VALUES (5, "Lord of the Rings", "J.R.R. Tolkien", "Fantasy", 19.99, 25);

INSERT INTO books VALUES (6, "The Alchemist", "Paulo Coelho", "Fiction", 9.99, 60); INSERT INTO books VALUES (7, "Atomic Habits", "James Clear", "Self-Help", 15.99, 55);

INSERT INTO books VALUES (8, "Rich Dad Poor Dad", "Robert Kiyosaki", "Finance", 11.99, 35);

INSERT INTO books VALUES (9, "Think and Grow Rich", "Napoleon Hill", "Finance", 8.99, 40);

INSERT INTO books VALUES (10, "Deep Work", "Cal Newport", "Self-Help", 13.99, 20);

INSERT INTO books VALUES (11, "1984", "George Orwell", "Dystopian", 7.99, 70);

INSERT INTO books VALUES (12, "Animal Farm", "George Orwell", "Dystopian", 6.99, 65);

INSERT INTO books VALUES (13, "The Great Gatsby", "F. Scott Fitzgerald", "Classic", 8.99, 30);

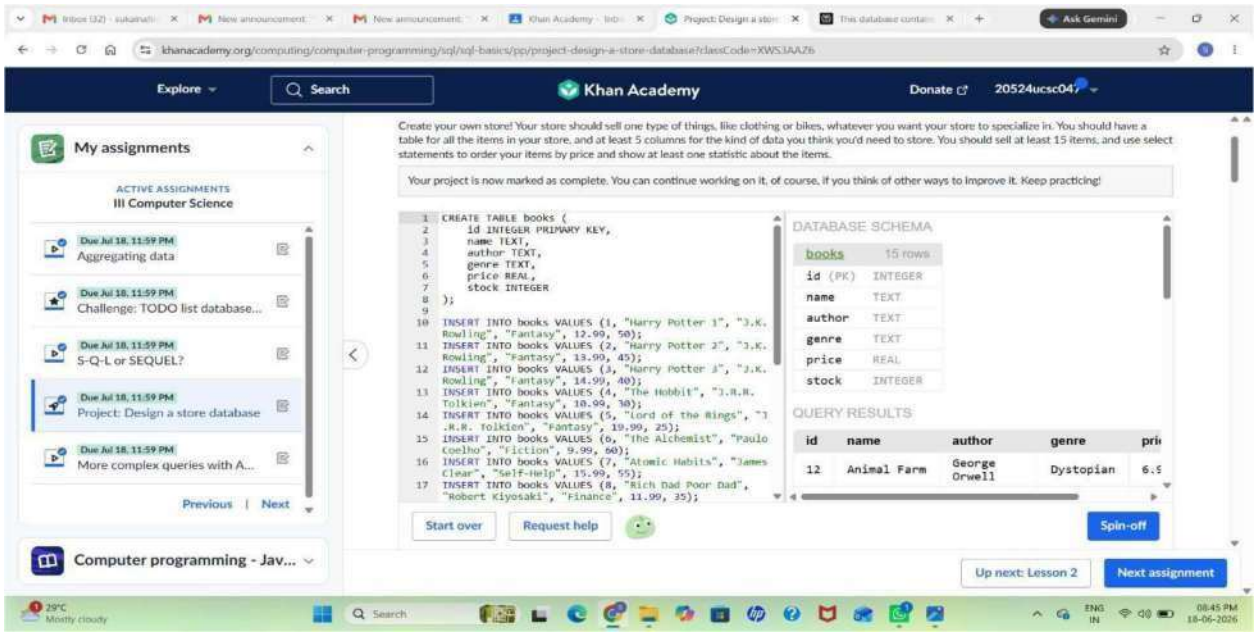
INSERT INTO books VALUES (14, "To Kill a Mockingbird", "Harper Lee", "Classic", 9.99, 25);

INSERT INTO books VALUES (15, "The Diary of a Young Girl", "Anne Frank", "Biography", 10.99, 50);

SELECT * FROM books ORDER BY price ASC;

SELECT MAX(price) FROM books; SELECT MIN(price) FROM books; SELECT AVG(price) FROM books;

OUTPUT



16. MORE COMPLEX QUERIES WITH AND/OR

PROGRAM

```
CREATE TABLE exercise_logs

(id INTEGER PRIMARY KEY AUTOINCREMENT,

type TEXT, minutes INTEGER, calories INTEGER,

heart_rate INTEGER);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200, 120);

SELECT * FROM exercise_logs WHERE calories > 50 ORDER BY calories;

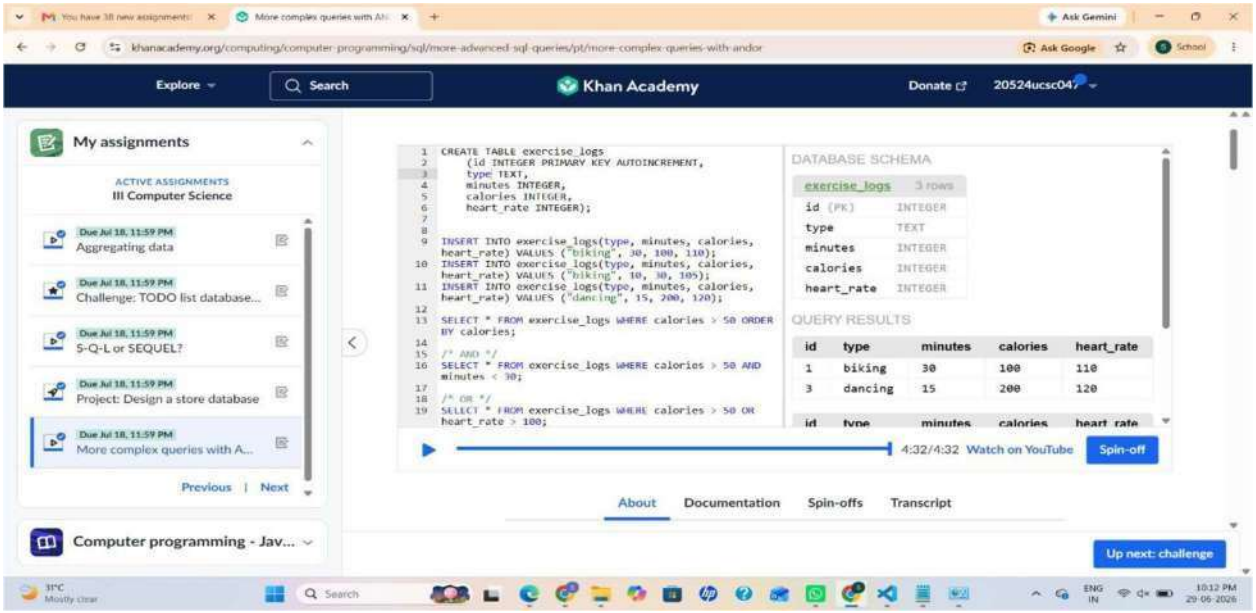
/* AND */

SELECT * FROM exercise_logs WHERE calories > 50 AND minutes < 30;

/* OR */

SELECT * FROM exercise_logs WHERE calories > 50 OR heart_rate > 100;
```

OUTPUT


Explore
Search
Khan Academy
Donate
My assignments
ACTIVE ASSIGNMENTS
III Computer Science
Aggregating data
Challenge: TODO list database...
S-Q-L or SEQUEL?
Project: Design a store database
More complex queries with A...
Previous | Next
Computer programming - Jav...
CREATE TABLE exercise_logs
(id INTEGER PRIMARY KEY AUTOINCREMENT,
type TEXT,
minutes INTEGER,
calories INTEGER,
heart_rate INTEGER);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200, 120);
SELECT * FROM exercise_logs WHERE calories > 50 ORDER BY calories;
/* AND */
SELECT * FROM exercise_logs WHERE calories > 50 AND minutes < 30;
/* OR */
SELECT * FROM exercise_logs WHERE calories > 50 OR heart_rate > 100;
DATABASE SCHEMA
exercise_logs 3 rows
id (PK) INTEGER
type TEXT
minutes INTEGER
calories INTEGER
heart_rate INTEGER
QUERY RESULTS
id type minutes calories heart_rate
1 biking 30 100 110
3 dancing 15 200 120
4:32/4:32 Watch on YouTube
Spin-off
About
Documentation
Spin-offs
Transcript
Up next: challenge

17. RESTRICTING GROUPING RESULTS WITH HAVING

PROGRAM

```
CREATE TABLE exercise_logs

(id INTEGER PRIMARY KEY AUTOINCREMENT,

type TEXT, minutes INTEGER, calories INTEGER,

heart_rate INTEGER);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 115,
110);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 45,
105);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200,
120);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 165,
120);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30,
70, 90);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25,
72,

80);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70,
90);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80,
85);

SELECT * FROM exercise_logs;

SELECT type, SUM(calories) AS total_calories FROM exercise_logs GROUP BY type;

SELECT type, SUM(calories) AS total_calories FROM exercise_logs

GROUP BY type
```

```
HAVING total_calories > 150

;

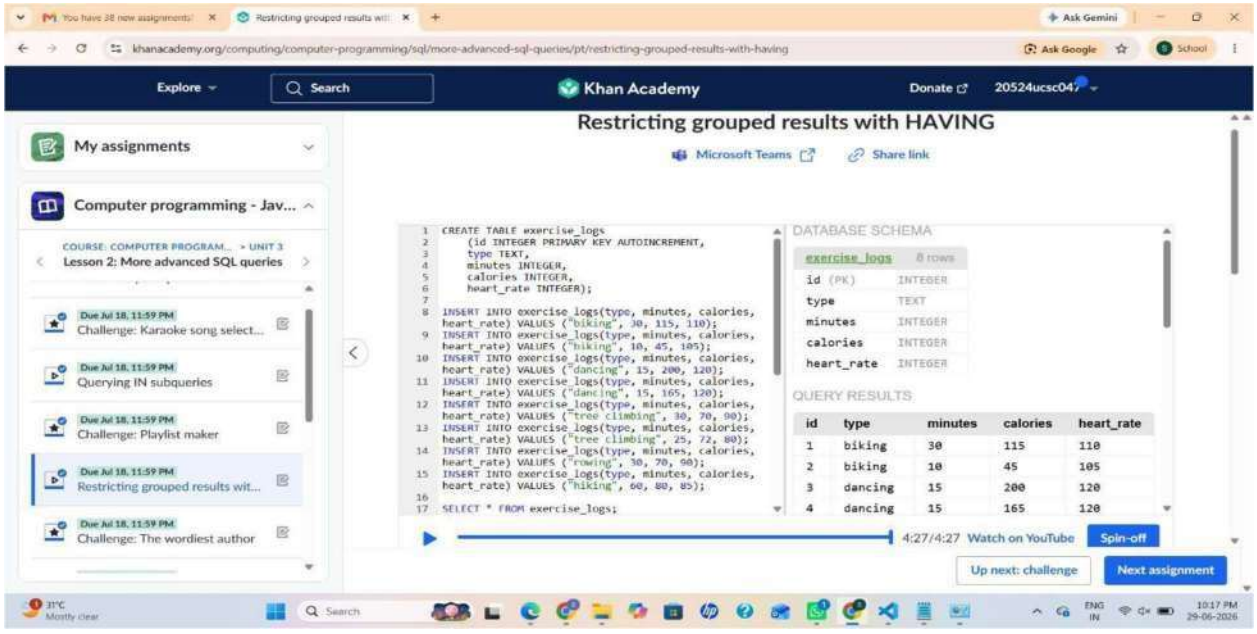
SELECT type, AVG(calories) AS avg_calories FROM exercise_logs GROUP BY type

HAVING avg_calories > 70

;

SELECT type FROM exercise_logs GROUP BY type HAVING COUNT(*) >= 2;
```

OUTPUT



18. THE WORDIEST AUTHOR

PROGRAM

CREATE TABLE books (

id INTEGER PRIMARY KEY AUTOINCREMENT,

author TEXT, title TEXT,

words INTEGER);

INSERT INTO books (author, title, words)

VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944); INSERT INTO books (author, title, words)

VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141); INSERT INTO books (author, title, words)

VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253); INSERT INTO books (author, title, words)

VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637); INSERT INTO books (author, title, words)

VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045); INSERT INTO books (author, title, words)

VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923); INSERT INTO books (author, title, words)

VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);

INSERT INTO books (author, title, words)

VALUES ("Stephenie Meyer", "Twilight", 118501); INSERT INTO books (author, title, words)

VALUES ("Stephenie Meyer", "New Moon", 132807);

INSERT INTO books (author, title, words)

VALUES ("Stephenie Meyer", "Eclipse", 147930); INSERT INTO books (author, title, words)

VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);

INSERT INTO books (author, title, words)

VALUES ("J.R.R. Tolkien", "The Hobbit", 95022); INSERT INTO books (author, title, words)

VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227); INSERT INTO books (author, title, words)

VALUES ("J.R.R. Tolkien", "Two Towers", 143436); INSERT INTO books (author, title, words)

VALUES ("J.R.R. Tolkien", "Return of the King", 134462);

SELECT author, SUM(words) AS total_words FROM books

GROUP BY author

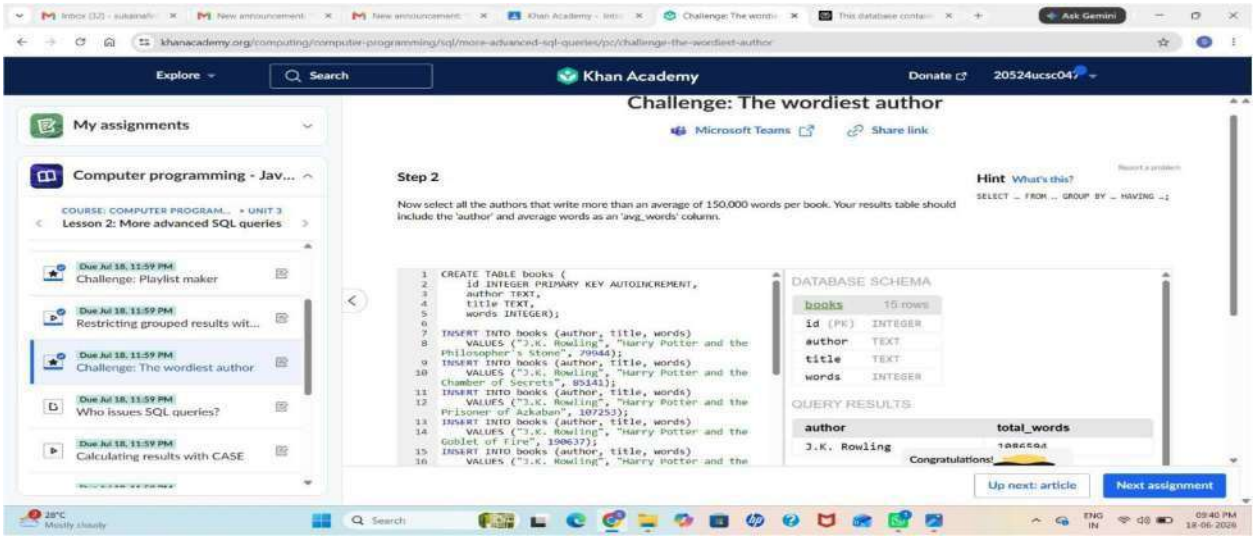
HAVING total_words > 1000000;

SELECT author, AVG(words) AS avg_words FROM books

GROUP BY author

HAVING avg_words > 150000;

OUTPUT



19. CALCULATING RESULTS WITH CASE

PROGRAM

CREATE TABLE exercise_logs

(id INTEGER PRIMARY KEY AUTOINCREMENT,

type TEXT, minutes INTEGER, calories INTEGER,

heart_rate INTEGER);

```
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 30, 100, 110);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("biking", 10, 30, 105);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 200, 120);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("dancing", 15, 165, 120);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 30, 70, 90);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree climbing", 25, 72,80);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("rowing", 30, 70, 90);

INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("hiking", 60, 80, 85);

SELECT * FROM exercise_logs;

SELECT COUNT(*) FROM exercise_logs WHERE heart_rate > 220 - 30;

/* 50-90% of max*/

SELECT COUNT(*) FROM exercise_logs WHERE heart_rate >= ROUND(0.50 * (220-30))

AND heart_rate <= ROUND(0.90 * (220-30));

/* CASE */

SELECT type, heart_rate, CASE

WHEN heart_rate > 220-30 THEN "above max"

WHEN heart_rate > ROUND(0.90 * (220-30)) THEN "above target" WHEN heart_rate > ROUND(0.50 * (220-30)) THEN "within target" ELSE "below target"

END as "hr_zone" FROM exercise_logs;

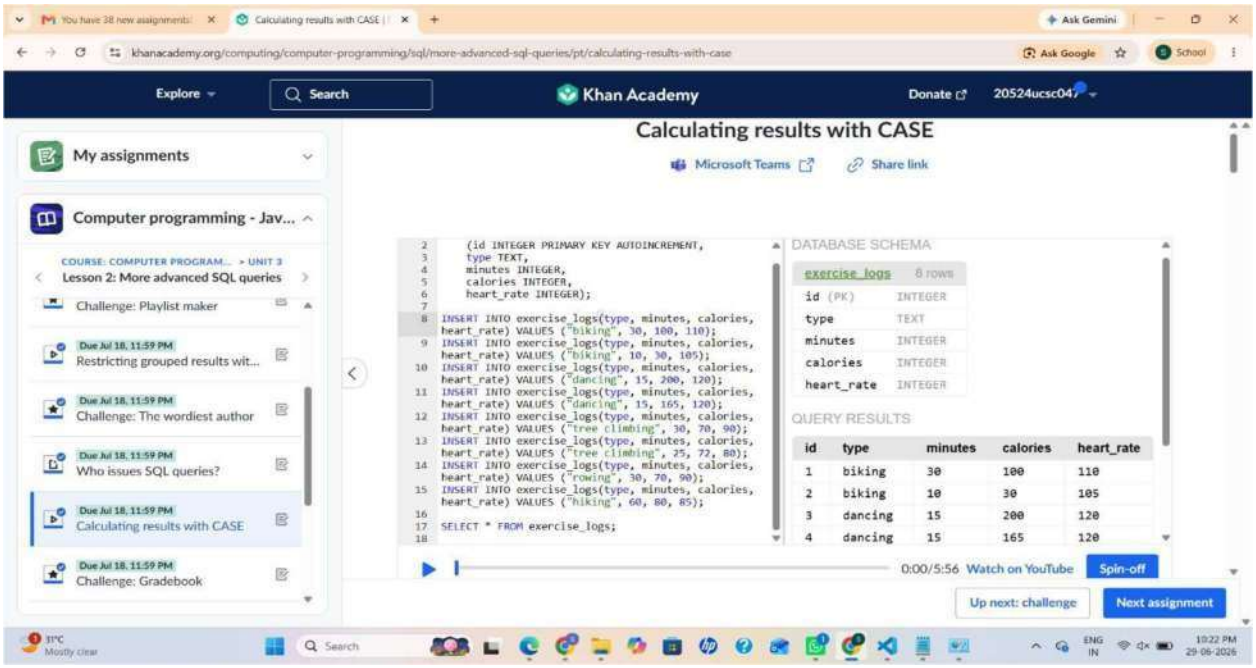
SELECT COUNT(*), CASE
```

WHEN heart_rate > 220-30 THEN "above max"

WHEN heart_rate > ROUND(0.90 * (220-30)) THEN "above target" WHEN heart_rate > ROUND(0.50 * (220-30)) THEN "within target" ELSE "below target"

END as "hr_zone" FROM exercise_logs GROUP BY hr_zone;

OUTPUT



19.PROJECT: DATA DIG

PROGRAM

```
CREATE TABLE astronauts ( Name TEXT PRIMARY KEY, Year INTEGER,

GroupNum INTEGER, Status TEXT, Birth_Date TEXT, Birth_Place TEXT, Gender
TEXT, Alma_Mater TEXT,

Undergraduate_Major TEXT, Graduate_Major TEXT, Military_Rank TEXT,
Military_Branch TEXT, Space_Flights INTEGER, Space_Flight_hr INTEGER,
Space_Walks INTEGER, Space_Walks_hr REAL, Missions TEXT,

Death_Date TEXT, Death_Mission TEXT
);

SELECT

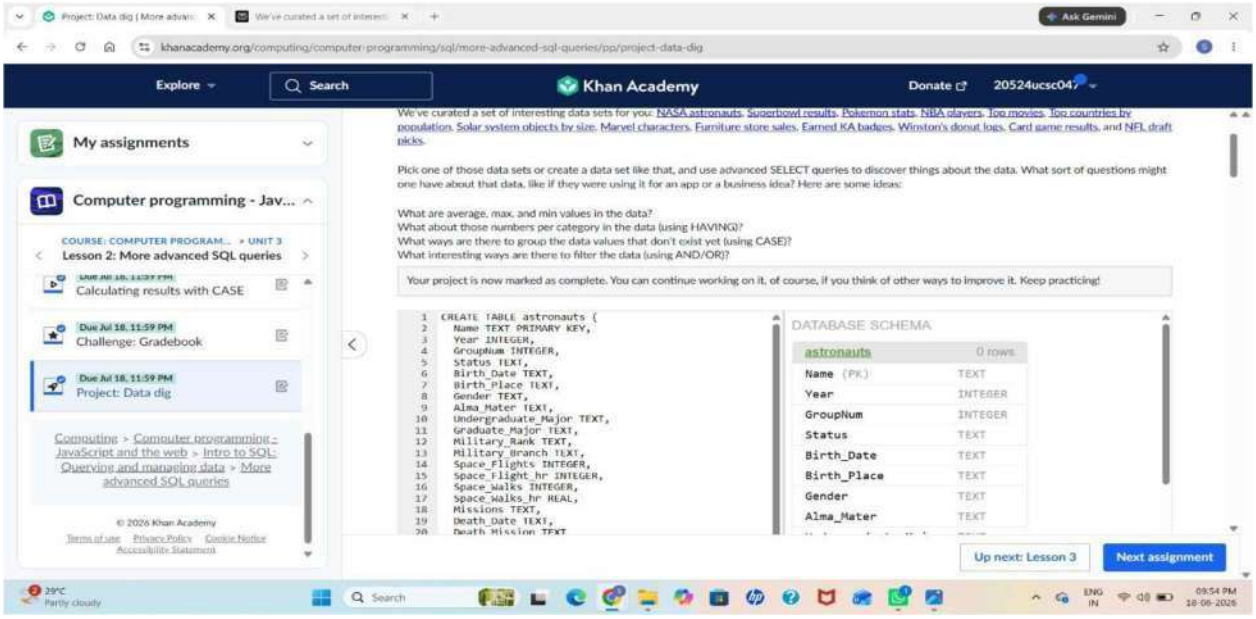
Gender,

AVG(Space_Flights) AS avg_flights, MAX(Space_Flights) AS max_flights,

MIN(Space_Flights) AS min_flights FROM astronauts

GROUP BY Gender;
```

OUTPUT



HACKER RANK

1. Revising the Select Query I

Query all columns for all American cities in the CITY table with populations larger than 100000. The CountryCode for America is USA.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION > 100000;
```

Output:



2. Revising the Select Query II

Query the NAME field for all American cities in the CITY table with populations larger than 120000. The *CountryCode* for America is USA.

The CITY table is described as follows:

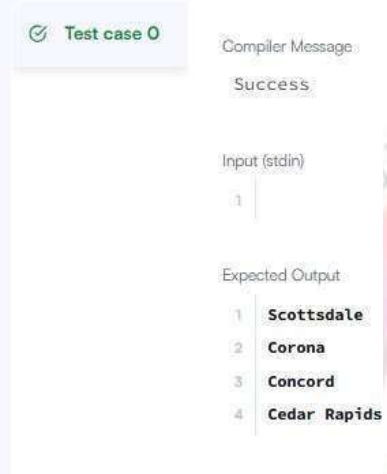
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT NAMEFROM CITY WHERE COUNTRYCODE = 'USA' AND POPULATION > 120000;
```

Output:



3. Select All

Query all columns (attributes) for every row in the CITY table.

The CITY table is described as follows:

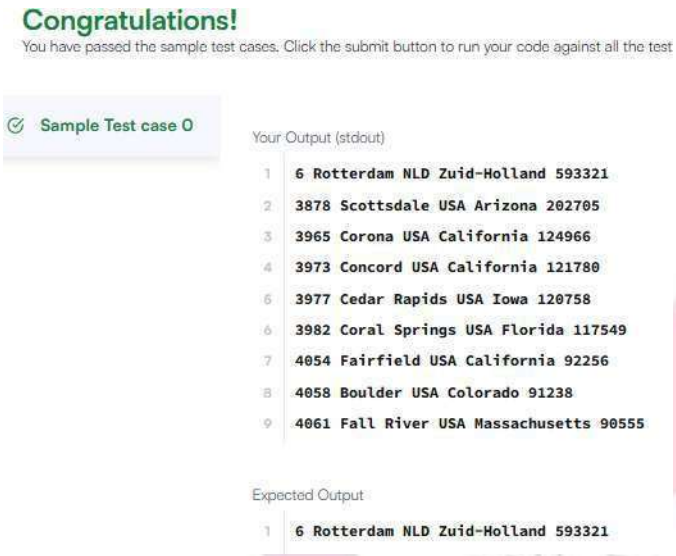
CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY;
```

Output:



4. Select By ID

Query all columns for a city in CITY with the *ID* `1661`.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY WHERE ID = 1661;
```

Output:



5. Japanese Cities' Attributes

Query all attributes of every Japanese city in the CITY table. The COUNTRYCODE for Japan is JPN.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT * FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



6. Japanese Cities' Names

Query the names of all the Japanese cities in the CITY table. The COUNTRYCODE for Japan is JPN.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

Input:

```
SELECT NAME FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



7. Weather Observation Station 1

Query a list of CITY and STATE from the STATION table.

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

Input:

```
SELECT CITY, STATE FROM STATION;
```

Output:



8. Weather Observation Station 3

Query a list of CITY names from STATION for cities that have an even ID number. Print the results in any order, but exclude duplicates from the answer.
The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE MOD(ID, 2) = 0 ORDER BY CITY;
```

Output:



9. Weather Observation Station 4

Find the difference between the total number of CITY entries in the table and the number of distinct CITY entries in the table.
The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

For example, if there are three records in the table with CITY values 'New York', 'New York', 'Bengalaru', there are 2 different city names: 'New York' and 'Bengalaru'.

Input:

```
SELECT COUNT(CITY) - COUNT (DISTINCT City) FROM STATION ;
```

Output:



10. Weather Observation Station 5

Query the two cities in STATION with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.
The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

Sample Input

For example, CITY has four entries: DEF, ABC, PQRS and WXY.

Sample Output

```
ABC 3
PQRS 4
```

Input:

```
SELECT CITY, LENGTH(CITY) FROM STATION ORDER BY LENGTH(CITY), CITY LIMIT 1;
SELECT CITY, LENGTH(CITY) FROM STATION ORDER BY LENGTH(CITY) DESC, CITY LIMIT 1;
```

Output:



11. Weather Observation Station 6

Query the list of *CITY* names starting with vowels (i.e., `a`, `e`, `i`, `o`, or `u`) from STATION. Your result *cannot* contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[aeiouAEIOU]';
```

Output:



12. Weather Observation Station 7

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from STATION. Your result *cannot* contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '[aeiouAEIOU]$';
```

Output:



13. Weather Observation Station 8

Query the list of *CITY* names from STATION which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP
'^[aeiouAEIOU].*[aeiouAEIOU]$';
```

Output:



14. Weather Observation Station 9

Query the list of *CITY* names from STATION that *do not start* with vowels. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[^aeiouAEIOU]';
```

Output:



15. Weather Observation Station 10

Query the list of *CITY* names from STATION that *do not end* with vowels. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE RIGHT(CITY, 1) NOT IN ('A', 'E', 'I',
'O', 'U') ORDER BY CITY;
```

Output:



16. Weather Observation Station 11

Query the list of *CITY* names from STATION that either do not start with vowels or do not end with vowels. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY NOT REGEXP '^[aeiouAEIOU]' OR
CITY NOT REGEXP '[aeiouAEIOU]$';
```

Output:



17. Weather Observation Station 12

Query the list of *CITY* names from STATION that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitud

Input:

```
SELECT DISTINCT CITY FROM STATION WHERE CITY REGEXP '^[^aeiouAEIOU]' AND CITY
REGEXP '[^aeiouAEIOU]$';
```

Output:



18. Higher Than 75 Marks

Query the *Name* of any student in STUDENTS who scored higher than *Marks*. Order your output by the *last three characters* of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending *ID*.

Input Format

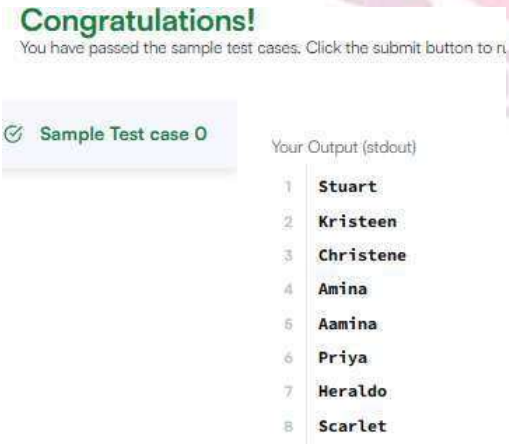
Column	Type
ID	Integer
Name	String
Marks	Integer

The STUDENTS table is described as follows: The *Name* column only contains uppercase (A-Z) and lowercase (a-z) letters.

Input:

```
SELECT NAME FROM STUDENTS WHERE MARKS > 75 ORDER BY RIGHT(NAME, 3), ID ASC;
```

Output:



19. Employee Names

Write a query that prints a list of employee names (i.e.: the *name* attribute) from the Employee table in alphabetical order.

Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is their monthly salary.

Sample Input

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Input:

```
SELECT name FROM Employee ORDER BY name;
```

Output:



20. Employee Salaries

Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in Employee having a salary greater than per month who have been employees for less than months. Sort your result by ascending *employee_id*.

Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

employee_id	name	months	salary
12228	Rose	15	1968
33645	Angela	1	3443
45692	Frank	17	1608
56118	Patrick	7	1345
59725	Lisa	11	2330
74197	Kimberly	16	4372
78454	Bonnie	8	1771
83565	Michael	6	2017
98607	Todd	5	3396
99989	Joe	9	3573

Input:

```
SELECT name FROM Employee WHERE salary > 2000   AND months < 10 ORDER BY
employee_id ASC;
```

Output:



21. Type of Triangle

Write a query identifying the *type* of each record in the TRIANGLES table using its three side lengths. Output one of the following statements for each record in the table:

- Equilateral: It's a triangle with sides of equal length.
- Isosceles: It's a triangle with sides of equal length.
- Scalene: It's a triangle with sides of differing lengths.
- Not A Triangle: The given values of *A*, *B*, and *C* don't form a triangle.

Input Format

The TRIANGLES table is described as follows:

Column	Type
A	Integer
B	Integer
C	Integer

Each row in the table denotes the lengths of each of a triangle's three sides.

Input:

```
SELECT CASE  WHEN A + B <= C OR A + C <= B OR B + C <= A THEN 'Not A Triangle
WHEN A = B AND B = C THEN 'Equilateral' WHEN A = B OR B = C OR A = C THEN
'Isosceles 'ELSE 'Scalene'END FROM TRIANGLES;
```

Output:



23. The PADS

Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in OCCUPATIONS, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: `AnActorName(A)`, `ADoctorName(D)`, `AProfessorName(P)`, and `ASingerName(S)`.
2. Query the number of ocurrences of each occupation in OCCUPATIONS. Sort the occurrences in *ascending order*, and output them in the following format:

   `There are a total of [occupation_count] [occupation]s.`

   where `[occupation_count]` is the number of occurrences of an occupation in OCCUPATIONS and `[occupation]` is the *lowercase* occupation name. If more than one *Occupation* has the same `[occupation_count]`, they should be ordered alphabetically.

Note: There will be at least two entries in the table for each type of occupation.

Input Format

Column	*Type*
Name	*String*
Occupation	*String*

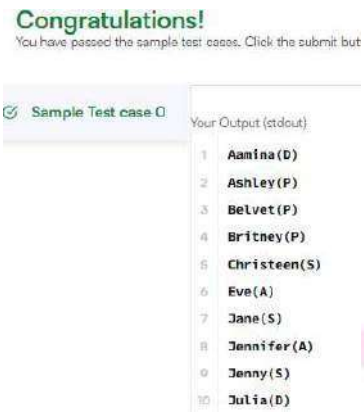
The OCCUPATIONS table is described as follows: *Occupation* will only contain one of the following values: Doctor, Professor, Singer or Actor.

Input:

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')') FROM OCCUPATIONS ORDER BY
Name ASC;
SELECT CONCAT('There are a total of ', COUNT(Occupation), ' ',
LOWER(Occupation), 's.') FROM OCCUPATIONS
```

```
GROUP BY Occupation
ORDER BY COUNT(Occupation) ASC, Occupation ASC;
```

Output:



24. Occupations

Pivot the *Occupation* column in OCCUPATIONS so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Note: Print NULL when there are no more names corresponding to an occupation.

Input Format

The OCCUPATIONS table is described as follows:

Column	*Type*
Name	*String*
Occupation	*String*

Occupation will only contain one of the following values: Doctor, Professor, Singer or Actor.

Input:

```
SELECT
    MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
    MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
    MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
    MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
```

```
    SELECT
        Name,
        Occupation,
        ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS rn
    FROM OCCUPATIONS
) AS sub
GROUP BY rn;
```

Output:



25. Binary Tree Nodes

You are given a table, *BST*, containing two columns: *N* and *P*, where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*.

Column	*Type*
N	*Integer*
P	*Integer*

Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

Input:

```
SELECT N, CASE WHEN P IS NULL THEN 'Root'
WHEN N IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN 'Inner'
 ELSE 'Leaf' END FROM BST ORDER BY N;
```

Output:



26. New Companies

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:



Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

Note:

- The tables may contain duplicate records.
- The *company_code* is string, so the sorting should not be numeric. For example, if the *company_codes* are *C_1*, *C_2*, and *C_10*, then the ascending *company_codes* will be *C_1*, *C_10*, and *C_2*.

Input:

```
SELECT
    c.company_code,
    c.founder,
    COUNT(DISTINCT lm.lead_manager_code),
    COUNT(DISTINCT sm.senior_manager_code),
    COUNT(DISTINCT m.manager_code),
    COUNT(DISTINCT e.employee_code)
FROM Company c
```

```
LEFT JOIN Lead_Manager lm ON c.company_code = lm.company_code
LEFT JOIN Senior_Manager sm ON lm.lead_manager_code = sm.lead_manager_code
LEFT JOIN Manager m ON sm.senior_manager_code = m.senior_manager_code
LEFT JOIN Employee e ON m.manager_code = e.manager_code
GROUP BY c.company_code, c.founder
ORDER BY c.company_code ASC;
```

Output:



27. Revising Aggregations - The Count Function

Query a *count* of the number of cities in CITY having a *Population* larger than 100,000.

Input Format

CITY

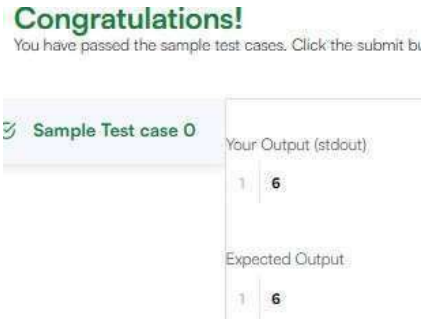
Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The CITY table is described as follows:

Input:

```
SELECT COUNT(*) FROM CITY WHERE POPULATION > 100000;
```

Output:



28. Revising Aggregations - Averages

Query the average population of all cities in CITY where *District* is California.

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The CITY table is described as follows:

Input:

```
SELECT AVG(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```

Output:



29. Average Population

Query the average population for all cities in CITY, rounded *down* to the nearest integer.

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The CITY table is described as follows:

Input:

```
SELECT FLOOR(AVG(POPULATION)) FROM CITY;
```

Output:



30. Japan Population

Query the sum of the populations for all Japanese cities in CITY. The *COUNTRYCODE* for Japan is JPN.

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The CITY table is described as follows:

Input:

```
SELECT SUM(POPULATION) FROM CITY WHERE COUNTRYCODE = 'JPN';
```

Output:



31. Population Density Difference

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The CITY table is described as follows:

Input:

```
SELECT MAX(POPULATION) - MIN(POPULATION) FROM CITY;
```

Output:



32. The Blunder

Samantha was tasked with calculating the average monthly salaries for all employees in the EMPLOYEES table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

The EMPLOYEES table is described as follows:

Column	Type
ID	Integer
Name	String
Salary	Integer

Note: *Salary* is per month.

Input:

```
SELECT CEIL(AVG(SALARY) - AVG(REPLACE(SALARY, '0', ''))) FROM EMPLOYEES;
```

Output:



33. Top Earners

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the Employee table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

Input:

```
SELECT (salary * months) AS earnings, COUNT(*) FROM Employee GROUP BY
earnings ORDER BY earnings DESC LIMIT 1;
```

Output:



34. Weather Observation Station 2

Query the following two values from the STATION table:

1. The sum of all values in *LAT_N* rounded to a scale of decimal places.
2. The sum of all values in *LONG_W* rounded to a scale of decimal places.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SUM(LAT_N), 2), ROUND(SUM(LONG_W), 2) FROM STATION;
```

Output:



35. Weather Observation Station 13

Query the sum of *Northern Latitudes* (*LAT_N*) from STATION having values greater than and less than . Truncate your answer to decimal places.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SUM(LAT_N), 4) FROM STATION WHERE LAT_N > 38.7880 AND LAT_N <
137.2345;
```

Output:



35. Weather Observation Station 14

Query the greatest value of the *Northern Latitudes* (*LAT_N*) from STATION that is less than . Truncate your answer to decimal places.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(MAX(LAT_N), 4) FROM STATION WHERE LAT_N < 137.2345;
```

Output:



36. Weather Observation Station 15

Query the *Western Longitude* (*LONG_W*) for the largest *Northern Latitude* (*LAT_N*) in STATION that is less than . Round your answer to decimal places.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(LONG_W, 4) FROM STATION WHERE LAT_N = (
SELECT MAX(LAT_N) FROM STATION WHERE LAT_N < 137.2345 );
```

Output:



37. Weather Observation Station 16

Query the smallest *Northern Latitude* (*LAT_N*) from STATION that is greater than . Round your answer to decimal places.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(MIN(LAT_N), 4) FROM STATION WHERE LAT_N > 38.7780;
```

Output:



38. Weather Observation Station 17

Query the *Western Longitude* (*LONG_W*)where the smallest *Northern Latitude* (*LAT_N*) in STATION is greater than . Round your answer to decimal places.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(LONG_W, 4) FROM STATION WHERE LAT_N = (
SELECT MIN(LAT_N) FROM STATION WHERE LAT_N > 38.7780
);
```

Output:



39. Weather Observation Station 18

Consider $P_1(a, b)$ and $P_2(c, d)$ to be two points on a 2D plane.

- a happens to equal the minimum value in Northern Latitude (LAT_N in **STATION**).
- b happens to equal the minimum value in Western Longitude (LONG_W in **STATION**).
- c happens to equal the maximum value in Northern Latitude (LAT_N in **STATION**).
- d happens to equal the maximum value in Western Longitude (LONG_W in **STATION**).

Query the Manhattan Distance between points P_1 and P_2 and round it to a scale of 4 decimal places.

Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.

Input:

```
SELECT ROUND(ABS(MAX(LAT_N) - MIN(LAT_N)) + ABS(MAX(LONG_W) - MIN(LONG_W)),4)
FROM STATION;
```

Output:



40. Weather Observation Station 19

Consider and to be two points on a 2D plane where are the respective minimum and maximum values of *Northern Latitude* (*LAT_N*) and are the respective minimum and maximum values of *Western Longitude* (*LONG_W*) in STATION.

Query the Euclidean Distance between points and and *format your answer* to display decimal digits.

Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:

```
SELECT ROUND(SQRT(POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
POWER(MAX(LONG_W)MIN(LONG_W), 2)), 4) FROM STATION;
```

Output:



41. Weather Observation Station 20

A *median* is defined as a number separating the higher half of a data set from the lower half. Query the *median* of the *Northern Latitudes* (*LAT_N*) from STATION and round your answer to decimal places.

Input Format

The STATION table is described as follows:

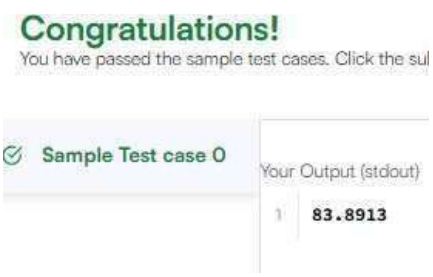
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where *LAT_N* is the northern latitude and *LONG_W* is the western longitude.

Input:
```
SELECT ROUND(S.LAT_N, 4)
FROM STATION AS S
WHERE (SELECT COUNT(LAT_N) FROM STATION WHERE LAT_N < S.LAT_N) = (SELECT
COUNT(LAT_N) FROM STATION WHERE LAT_N > S.LAT_N);
```

Output:



42. Population Census

Given the CITY and COUNTRY tables, query the sum of the populations of all cities where the *CONTINENT* is *'Asia'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The CITY and COUNTRY tables are described as follows:

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Input:

```
SELECT SUM(CITY.POPULATION)
 FROM CITY JOIN COUNTRY
```

```
    ON CITY.COUNTRYCODE = COUNTRY.CODE
WHERE COUNTRY.CONTINENT = 'Asia';
```

Output:



43. African Cities

Given the CITY and COUNTRY tables, query the names of all cities where the *CONTINENT* is *'Africa'*.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The CITY and COUNTRY tables are described as follows:

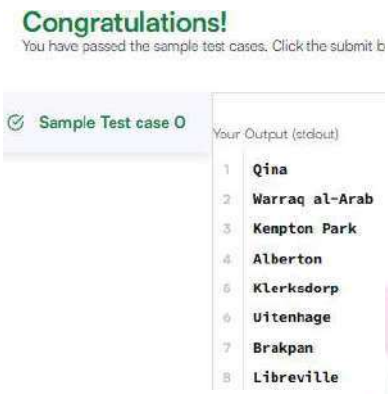
COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Input:

```
SELECT CITY.NAME FROM CITYINNER JOIN COUNTRYON CITY.CountryCode =
COUNTRY.Code WHERE COUNTRY.Continent = 'Africa';
```

Output:



44. Average Population of Each Continent

Given the CITY and COUNTRY tables, query the names of all the continents (*COUNTRY.Continent*) and their respective average city populations (*CITY.Population*) rounded *down* to the nearest integer.

Note: *CITY.CountryCode* and *COUNTRY.Code* are matching key columns.

Input Format

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

The CITY and COUNTRY tables are described as follows:

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Input:

```
SELECT COUNTRY.Continent, FLOOR(AVG(CITY.Population))FROM CITY
INNER JOIN COUNTRY ON CITY.CountryCode = COUNTRY.Code
GROUP BY COUNTRY.Continent;
```

Output:



45. The Report

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

Column	Type
ID	Integer
Name	String
Marks	Integer

Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives *Eve* a task to generate a report containing three columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a grade lower than *8*. The report must be in descending order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

Write a query to help Eve.

Input:

```
SELECT
    CASE
        WHEN g.Grade < 8 THEN NULL
        ELSE s.Name
    END AS Name,
    g.Grade,
    s.Marks
FROM Students s
JOIN Grades g
  ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
    g.Grade DESC,
```

```
    CASE WHEN g.Grade >= 8 THEN s.Name END ASC,
    CASE WHEN g.Grade < 8 THEN s.Marks END ASC;
```

Output:



46. Top Competitors

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Difficulty:* The *difficult_level* is the level of difficulty of the challenge, and *score* is the maximum score that can be achieved for a challenge at that difficulty level.

Column	Type
difficulty_level	Integer
score	Integer

- *Challenges:* The *challenge_id* is the id of the challenge, the *hacker_id* is the id of the hacker who created the challenge, and *difficulty_level* is the level of difficulty of the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer
difficulty_level	Integer

Input:

```
SELECT
    h.hacker_id,
    h.name
FROM submissions s
INNER JOIN hackers h
    ON s.hacker_id = h.hacker_id
INNER JOIN challenges c
    ON s.challenge_id = c.challenge_id
INNER JOIN difficulty d
    ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```

Output:



47. Ollivander's Inventory

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of the wands that Ron's interested in, sorted in order of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

- *Wands_Property:* The *code* is the code of the wand, *age* is the age of the wand, and *is_evil* denotes whether the wand is good for the dark arts. If the value of *is_evil* is *0*, it means that the wand is not evil. The mapping between *code* and *age* is one-one, meaning that if there are two pairs, and , then and .

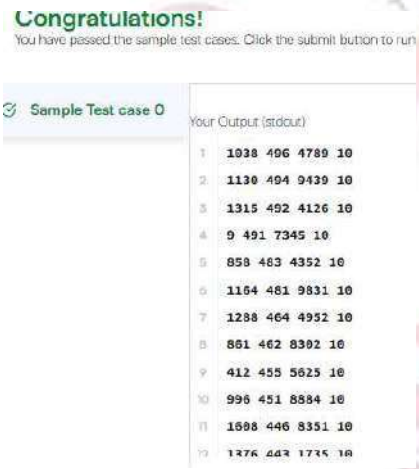
Column	Type
code	Integer
age	Integer
is_evil	Integer

Input:

```
SELECT
    w.id,
    p.age,
    w.coins_needed,
    w.power
FROM
    wands AS w
JOIN
```

```
    wands_property AS p ON w.code = p.code
WHERE
    p.is_evil = 0
    AND w.coins_needed = (
        SELECT
            MIN(w1.coins_needed)
        FROM
            wands AS w1
        JOIN
            wands_property AS p1 ON w1.code = p1.code
        WHERE
            p1.is_evil = 0
            AND w1.power = w.power
            AND p1.age = p.age
    )
ORDER BY
    w.power DESC,
    p.age DESC;
```

Output:



48. Challenges

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the

same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Challenges:* The *challenge_id* is the id of the challenge, and *hacker_id* is the id of the student who created the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer

Input:

```
SELECT H.HACKER_ID,
       H.NAME,
       COUNT(C.CHALLENGE_ID) AS C_COUNT
FROM HACKERS H
JOIN CHALLENGES C ON C.HACKER_ID = H.HACKER_ID
GROUP BY H.HACKER_ID, H.NAME
HAVING C_COUNT =
    (SELECT COUNT(C2.CHALLENGE_ID) AS C_MAX
     FROM CHALLENGES AS C2
     GROUP BY C2.HACKER_ID
     ORDER BY C_MAX DESC LIMIT 1)
OR C_COUNT IN
    (SELECT DISTINCT C_COMPARE AS C_UNIQUE
     FROM (SELECT H2.HACKER_ID,
                  H2.NAME,
                  COUNT(CHALLENGE_ID) AS C_COMPARE
           FROM HACKERS H2
           JOIN CHALLENGES C ON C.HACKER_ID = H2.HACKER_ID
           GROUP BY H2.HACKER_ID, H2.NAME) COUNTS
     GROUP BY C_COMPARE
     HAVING COUNT(C_COMPARE) = 1)
```

```
ORDER BY C_COUNT DESC, H.HACKER_ID;
```

Output:



49. Contest Leaderboard

You did such a great job helping Julia with her last oding contest challenge that she wants you to work on this one, too.The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of from your result.

Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, *challenge_id* is the id of the challenge for which the submission

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

belongs to, and *score* is the score of the submission.

Input:

```
SELECT X.hacker_id,
(SELECT H.NAME FROM HACKERS H
                      WHERE H.HACKER_ID = X.HACKER_ID) NAME,
SUM(X.SCORE) TOTAL_SCORE FROM
  (SELECT HACKER_ID, MAX(SCORE) SCORE FROM SUBMISSIONS S
   GROUP BY 1, S.CHALLENGE_ID) X
GROUP BY 1
HAVING TOTAL_SCORE > 0
ORDER BY TOTAL_SCORE DESC, HACKER_ID;
```

Output:



50. SQL Project Planning

You are given a table, *Projects*, containing three columns: *Task_ID*, *Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	*Type*
Task_ID	*Integer*
Start_Date	*Date*
End_Date	*Date*

If the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

Sample Output

```
2015-10-28  2015-10-29
2015-10-30  2015-10-31
2015-10-13  2015-10-15
2015-10-01  2015-10-04
```

Input:

```
SELECT Start_Date, MIN(End_Date) AS Min_End_Date
FROM (
    SELECT Start_Date
    FROM Projects
    WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)
) AS a
JOIN (
    SELECT End_Date
    FROM Projects
    WHERE End_Date NOT IN (SELECT Start_Date FROM Projects)
) AS b ON a.Start_Date < b.End_Date
GROUP BY Start_Date
ORDER BY DATEDIFF(MIN(End_Date), Start_Date) ASC, Start_Date ASC;
```

Output:



51.Placements

You are given three tables: *Students*, *Friends* and *Packages*. *Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students got same salary offer.

Sample Input

ID	Name
1	Ashley
2	Samantha
3	Julia
4	Scarlet

Students

ID	Friend_ID
1	2
2	3
3	4
4	1

Friends

ID	Salary
1	15.20
2	10.06
3	11.55
4	12.12

Packages

Sample Output

```
Samantha
Julia
Scarlet
```

Explanation

See the following table:

ID	1	2	3	4
Name	Ashley	Samantha	Julia	Scarlet
Salary	15.20	10.06	11.55	12.12
Friend ID	2	3	4	1
Friend Salary	10.06	11.55	12.12	15.20

Now,

- *Samantha's* best friend got offered a higher salary than her at 11.55
- *Julia's* best friend got offered a higher salary than her at 12.12
- *Scarlet's* best friend got offered a higher salary than her at 15.2
- *Ashley's* best friend did NOT get offered a higher salary than her

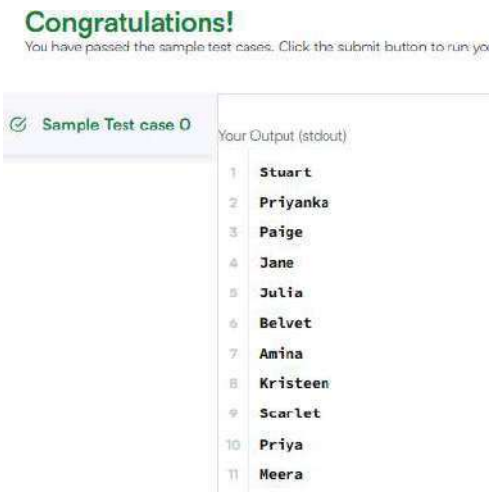
The name output, when ordered by the salary offered to their friends, will be:

- *Samantha*
- *Julia*
- *Scarlet*

Input:

```
SELECT S.NAME
FROM STUDENTS S
JOIN FRIENDS F ON S.ID = F.ID
JOIN PACKAGES P1 ON S.ID = P1.ID
JOIN PACKAGES P2 ON F.FRIEND_ID = P2.ID
WHERE P2.SALARY > P1.SALARY
ORDER BY P2.SALARY;
```

Output:



51. Symmetric Pairs

You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	*Type*
X	*Integer*
Y	*Integer*

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X_1 \leq Y_1$.

Sample Input

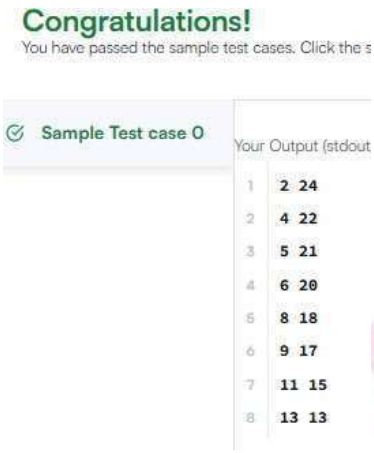
X	Y
20	20
20	20
20	21
23	22
22	23
21	20

Input:

```
SELECT f1.X, f1.Y FROM Functions f1
INNER JOIN Functions f2 ON f1.X = f2.Y AND f1.Y = f2.X
GROUP BY f1.X, f1.Y
```

```
HAVING f1.X < f1.Y OR COUNT(f1.X) > 1
ORDER BY f1.X;
```

Output:



52. 15 Days of Learning SQL

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was *March 06, 2016*.

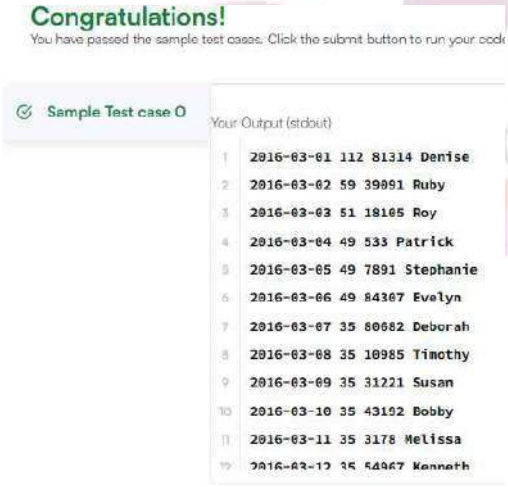
submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

Input:

```
SELECT SUBMISSION_DATE,
(SELECT COUNT(DISTINCT HACKER_ID)
```

```
 FROM SUBMISSIONS S2
 WHERE S2.SUBMISSION_DATE = S1.SUBMISSION_DATE AND
(SELECT COUNT(DISTINCT S3.SUBMISSION_DATE)
 FROM SUBMISSIONS S3 WHERE S3.HACKER_ID = S2.HACKER_ID AND S3.SUBMISSION_DATE
< S1.SUBMISSION_DATE) = DATEDIFF(S1.SUBMISSION_DATE , '2016-03-01')),
(SELECT HACKER_ID FROM SUBMISSIONS S2 WHERE S2.SUBMISSION_DATE =
S1.SUBMISSION_DATE
GROUP BY HACKER_ID ORDER BY COUNT(SUBMISSION_ID) DESC, HACKER_ID LIMIT 1) AS
TMP,
(SELECT NAME FROM HACKERS WHERE HACKER_ID = TMP)
FROM
(SELECT DISTINCT SUBMISSION_DATE FROM SUBMISSIONS) S1
GROUP BY SUBMISSION_DATE;
```

Output:



53. Draw The Triangle 1

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

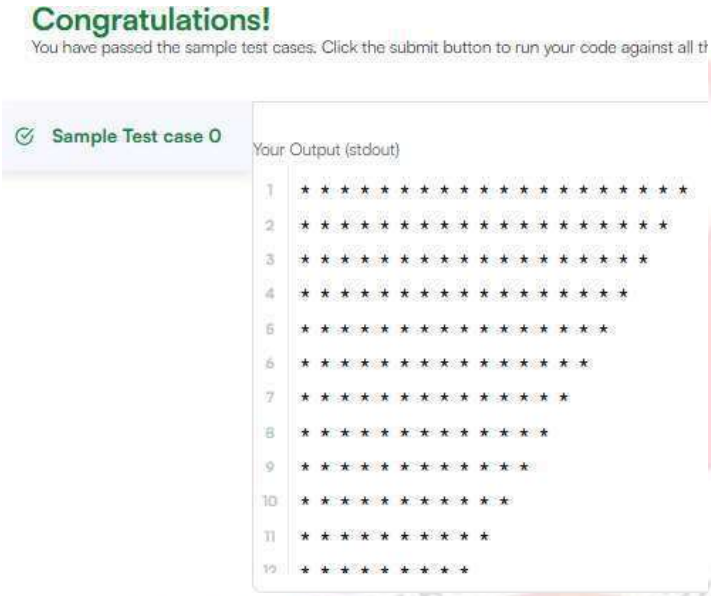
```
* * * * *
* * * *
* * *
* *
*
```

Write a query to print the pattern *P(20)*.

Input:

```
SET @TEMP := 21;
SELECT REPEAT('* ', @TEMP := @TEMP - 1)
FROM INFORMATION_SCHEMA.TABLES;
```

Output:



54. Draw The Triangle 2

P(R) represents a pattern drawn by Julia in *R* rows. The following pattern represents *P(5)*:

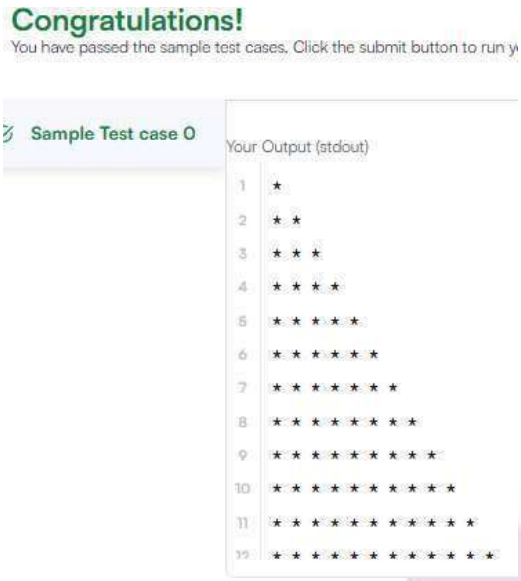
```
*
* *
* * *
* * * *
* * * * *
```

Write a query to print the pattern *P(20)*.

Input:

```
SET @number = 0;
SELECT REPEAT('* ', @number := @number + 1)
FROM information_schema.tables
WHERE @number < 20;
```

Output:



55. Print Prime Numbers

Write a query to print all *prime numbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

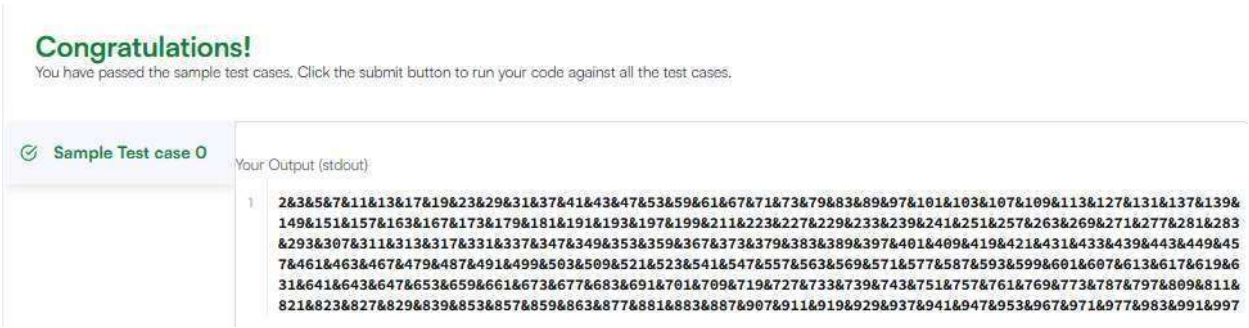
For example, the output for all prime numbers would be:

2&3&5&7

Input:

```
WITH RECURSIVE nums AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n + 1 FROM nums WHERE n < 1000
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
FROM nums p
WHERE NOT EXISTS (
    SELECT 1 FROM nums d WHERE d.n <= SQRT(p.n) AND d.n > 1 AND p.n % d.n = 0
);
```

Output:



56. Interviews

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

  the contest, and *name* is the name of the hacker.
- *Colleges:* The *college_id* is the id of the college, and *contest_id* is the id of the contest that

Column	Type
college_id	Integer
contest_id	Integer

  Samantha used to screen the candidates.
- *Challenges:* The *challenge_id* is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and *college_id* is the id of the college where the challenge was given to candidates.

Column	Type
challenge_id	Integer
college_id	Integer

- *View_Stats:* The *challenge_id* is the id of the challenge, *total_views* is the number of times the challenge was viewed by candidates, and *total_unique_views* is the number of times the challenge was viewed by unique candidates.

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

- *Submission_Stats:* The *challenge_id* is the id of the challenge, *total_submissions* is the number of submissions for the challenge, and *total_accepted_submission* is the number of submissions that achieved full scores.

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Input:

```
SELECT t1.contest_id, t1.hacker_id, t1.name,
       SUM(ts) as ts, SUM(tas) as tas, SUM(tv) as tv, SUM(tuv) as tuv
FROM CONTESTS AS t1
LEFT JOIN COLLEGES AS t2
    ON t1.contest_id = t2.contest_id
LEFT JOIN CHALLENGES AS t3
    ON t2.college_id = t3.college_id
LEFT JOIN (SELECT challenge_id,
           SUM(total_views) AS tv, SUM(total_unique_views) as tuv
           FROM View_Stats
           GROUP BY challenge_id) AS t4
    ON t3.challenge_id = t4.challenge_id
LEFT JOIN (SELECT challenge_id,
           SUM(total_submissions) AS ts, SUM(total_accepted_submissions) as
tas
           FROM Submission_Stats
           GROUP BY challenge_id) as t5
    ON t3.challenge_id = t5.challenge_id
GROUP BY t1.contest_id, t1.hacker_id, t1.name
HAVING (ts + tas + tv + tuv > 0)
ORDER BY t1.contest_id;
```

Output:


Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all
Sample Test case 0
Your Output (stdout)
845 579 Rose 1987 580 1635 566
858 1053 Angela 703 160 1002 384
883 1055 Frank 1121 319 1217 338
1793 2655 Patrick 1337 360 1216 412
2374 2765 Lisa 2733 815 3368 904
2963 2845 Kimberly 4306 1221 3603 1184
3584 2873 Bonnie 2492 652 3019 954
4044 3067 Michael 1323 449 1721 528
4249 3116 Todd 1452 376 1767 463
4269 3256 Joe 1018 372 1766 530
4483 3386 Earl 1911 572 1644 477

1. ALL PRODUCTS

Task:

Write a query to output `Products` table.

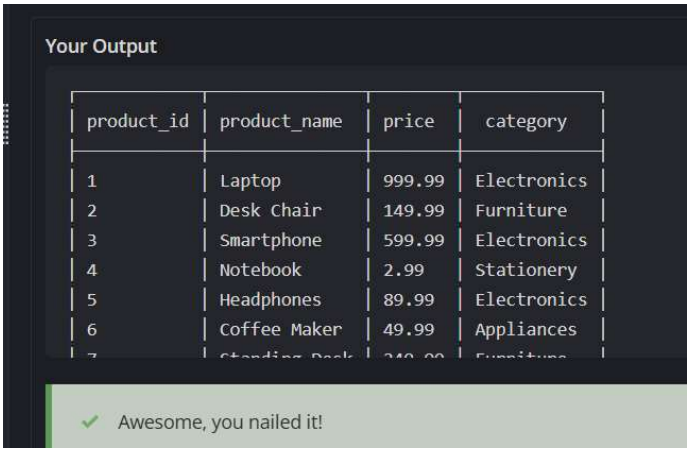
Table Name: `Products`

Product_ID	Product_Name	Price	Category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

Input:

```
SELECT * FROM Products;
```

Output:



2. HIGH PRICE OF PRODUCTS

Task:

Write a query to find all `product_name` and `category` that have a price greater than 100.00 from the **Products** table.

Table Name: `Products`

Product_ID	Product_Name	Price	Category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

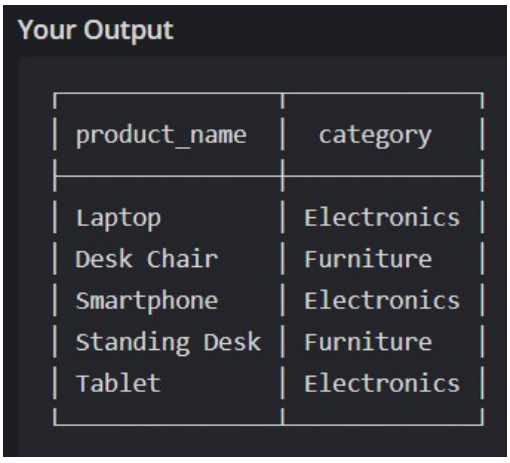
Expected Columns:

- product_name
- category

Input:

```
SELECT product_name, category FROM Products WHERE price > 100.00;
```

Output:



3. AVERAGE SALARY

Task:

Write a query to calculate the average salary across all companies combined. Rename the column as `avg_salary`.

Table Name: `Works`

Employee_ID	Company_Name	Salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

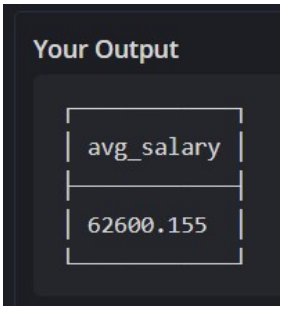
Expected Column:

- avg_salary

Input:

```
SELECT AVG(salary) AS avg_salary FROM Works;
```

Output:



4. LOCATE PEOPLE

Task:

Write a query to retrieve the `department_name` and `location` of departments located in a city that starts with 'S'.

Table Name: `departments`

Department_ID	Department_Name	Location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Bostn

Expected Columns:

- department_name
- location

Input:

```
SELECT department_name, location FROM departments WHERE location LIKE 'S%';
```

Output:

Your Output

department_name	location
Research and Development	San Francisco
IT Support	Seattle
Product Management	San Francisco

5. DISTINCT COMPANIES

Task:

Write a query to select all the distinct companies (`company_name`) in the `Works` table.

Table Name: `Works`

Employee_ID	Company_Name	Salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

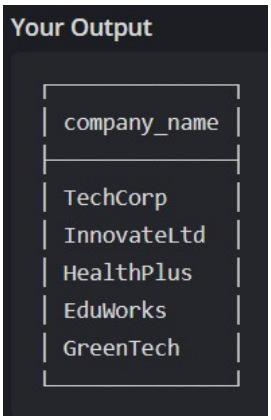
Expected Column:

- company name

Input:

```
SELECT DISTINCT company_name FROM Works;
```

Output:



6. FICTION COLLECTION SIZE

Task:

Write a query to find the total count of books whose genre is **Fiction**.

Note: Output column name should be **fiction_count**.

Table Name: Books

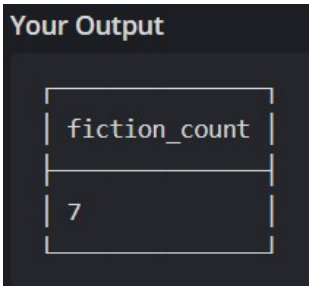
ID	Title	Author	Genre	Price	Published_Year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985
14	A Tale of Two Cities	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

Expected Column:

- fiction_count
- **Input:**

```
SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction';
```

Output:



7. LIST OF MOVIES WITH RATINGS

Task:

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table Name: `Cinema`

Movie_ID	Movie_Name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

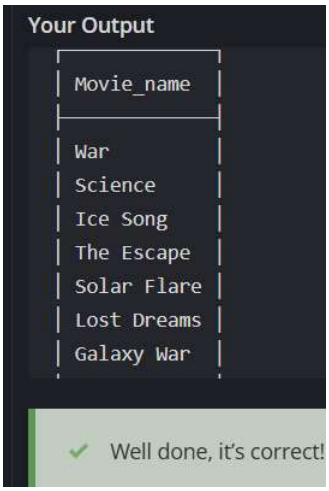
Expected Column:

- Movie_name

Input:

```
SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.0;
```

Output:



8. HANDLING NULL VALUES

Task:

Write a query to retrieve `book_id`, `title`, `author` and `published_year` of the books which have **NULL** rating for their books.

Table Name: `Library`

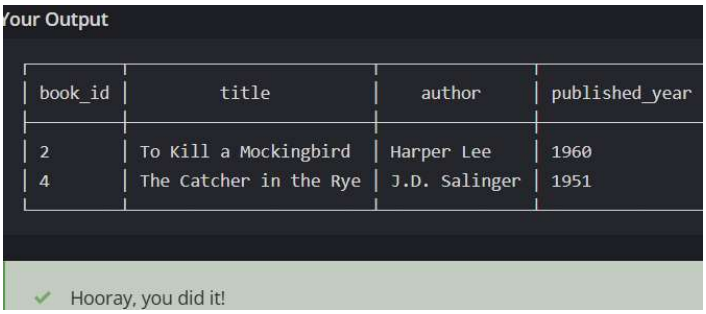
Book_ID	Title	Author	Published_Year	Rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

Expected Columns:

- book_id
- title
- author
- published_year

Input:
```
SELECT book_id, title, author, published_year FROM Library WHERE rating IS NULL;
```

Output:



9. SALARY OF EMPLOYEES

Task:

Create a query to retrieve the `employee_name`, `company`, and `salary` for employees in the full-time category, ordered by salary in **descending** order.

Table Name: `Employees`

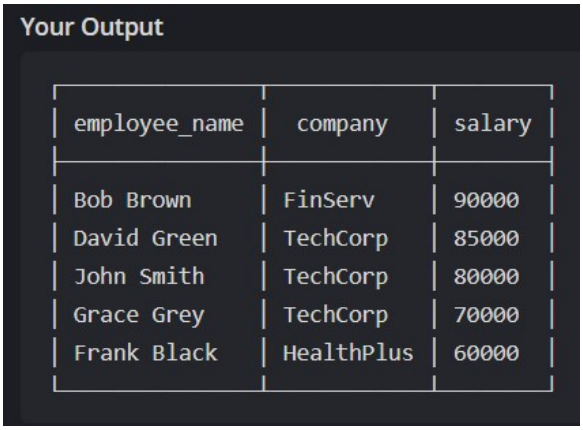
Employee_ID	Employee_Name	Company	Category	Department	Salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

Expected Columns:

- employee_name
- company
- salary

Input: `SELECT employee_name, company, salary FROM Employees WHERE category = 'Full-Time' ORDER BY salary DESC;`

Output:



10. DEPARTMENT OF EACH EMPLOYEE

Task:

Write a query to group the employees by their department and display the total number of employees (as `total_employees`) in each department.

Table Name: `Employees`

Employee_ID	Employee_Name	Company	Category	Department	Salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

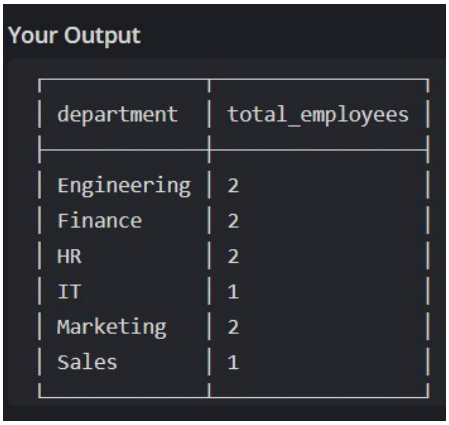
Expected Column:

- department
- total_employees

Input:

```
SELECT department, COUNT(*) AS total_employees FROM Employees GROUP BY department;
```

Output:



11. ARTICLE VIEWS

Task:

Write a query to retrieve the `author_id`, `author_name`, and `publication_name` for authors whose articles got zero views. The result should be sorted by `author_id` in ascending order.

Return the result table sorted by id in **ascending order**.

Table Name: `Views`

Article_ID	Author_ID	Author_Name	Viewer_ID	View_Count	Publication_Name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times
107	2	Jane Smith	2	2	Global News

Expected Columns:

- author_id
- author_name
- publication_name

Input:

```
SELECT DISTINCT author_id, author_name, publication_name FROM Views WHERE view_count = 0
ORDER BY author_id ASC;
```

Output:

Your Output

author_id	author_name	publication_name
1	John Doe	The Daily Times
3	Emily Clark	Tech Monthly
5	Sara White	The Business Journal

12. PLAYER PERFORMANCE INSIGHTS

Task:

Write a query to find the names of the **top 3 distinct players** by highest score who have **won** matches, including their **scores**.

Table 1: `Players`

Player_ID	Player_Name	Score	Rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: `Matches`

Match_ID	Player1	Player2	Winner	Match_Date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

Expected Columns:

- player_name
- score

Input:

```
SELECT DISTINCT player_name, score
FROM Players JOIN Matches WHERE winner=player_name
ORDER BY score DESC
LIMIT 3;
```

Output:

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

13. PLAYER DETAILS

Task:

Write a query to retrieve the details of the **last five matches** played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named **`Players`** and **`Matches`**.

Table 1: `Players`

Player_ID	Player_Name	Score	Rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	Jack	1400	10

Table 2: `Matches`

Match_ID	Player_1	Player_2	Winner	Match_Date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

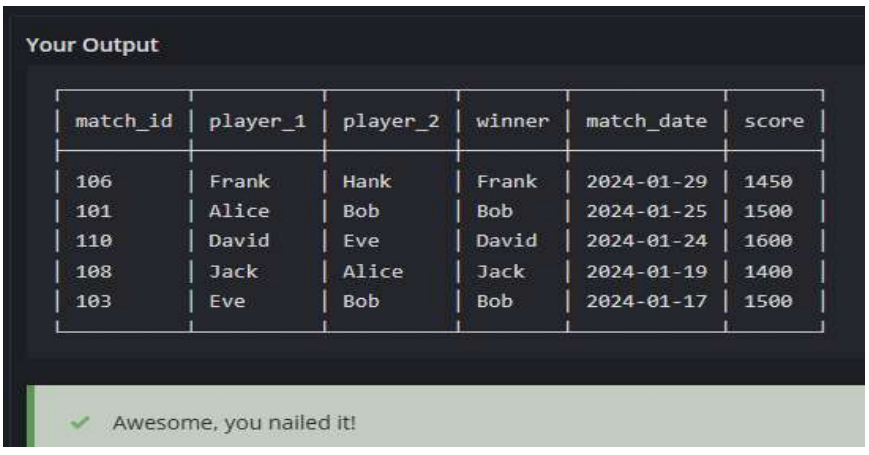
Expected Output Columns:

- match_id
- player_1
- player_2
- winner
- match_date
- score

Input:

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score
FROM Matches m
JOIN Players p ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

Output:



1. Create CABHUB Table

CREATE TABLE CABHUBS3 Vcode INT PRIMARY KEY VehicleName VARCHAR(30),

Make VARCHAR(20), Color VARCHAR(15) Capacity INT, Charges INT);

Results Explain Describe Saved SQL History

Table created.

0.14 seconds

INSERT INTO CABHUBS3 VALUES (100,'Innova','Toyota','WHITE',7,15);

INSERT INTO CABHUBS3 VALUES (102,'SX4','Suzuki','BLUE',4,14);

INSERT INTO CABHUBS3 VALUES (104,'C-Class','Mercedes','RED',4,35);

INSERT INTO CABHUBS3 VALUES (105,'A-Star','Suzuki','WHITE',3,14);

INSERT INTO CABHUBS3 VALUES (108,'Indigo','Tata','SILVER',3,12);

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.03 seconds

Create CUSTOMER Table

CREATE TABLE CUSTOMERS (Code INT PRIMARY KEY, CName VARCHAR(30) VCode INT);

Results Explain Describe Saved SQL History

Table created.

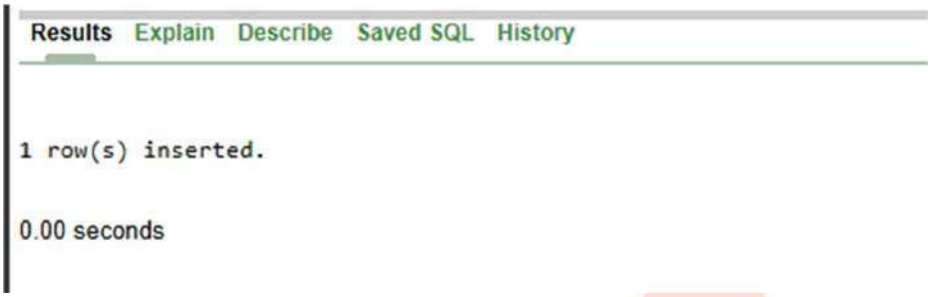
0.06 seconds

INSERT INTO CUSTOMERS VALUES (1,'HemantSahu',101);

INSERT INTO CUSTOMERS VALUES (2,'Raj Lal',108);

INSERT INTO CUSTOMERS VALUES (3,'Feroza Shah',105);

INSERT INTO CUSTOMERS VALUES (4,'Ketan Dhal',104);

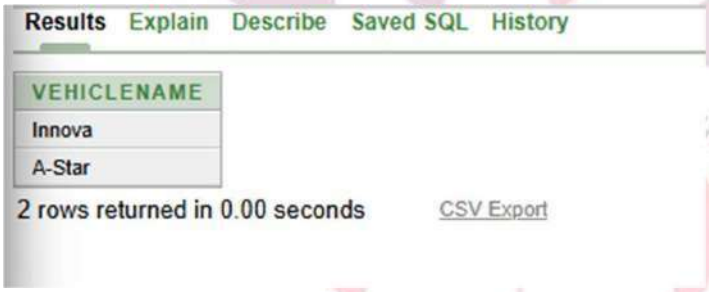


(a) Write SQL commands for the following statements:

(i) To display the names of all the white colored vehicles

SELECT VehicleName FROM CABHUB

WHERE Color = 'WHITE';



(ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.

SELECT VehicleName, Make, Capacity FROM CABHUB

ORDER BY Capacity ASC;

Results Explain Describe Saved SQL History

VEHICLENAME	MAKE	CAPACITY
Indigo	Tata	3
A-Star	Suzuki	3
Swift	Maruti	4
Dzire	Maruti	4
SX4	Suzuki	4
C-Class	Mercedes	4
i20	Hyundai	5
Ertiga	Maruti	6
Innova	Toyota	7

9 rows returned in 0.00 seconds CSV Export

(iii) To display the highest charges at which a vehicle can be hired from CABHUB.

SELECT MAX(Charges) FROM CABHUB;

Results Explain Describe Saved SQL History

MAX(CHARGES)
1800

1 rows returned in 0.00 seconds CSV Export

(iv) To display the customer and the corresponding name of the vehicle hired by them.

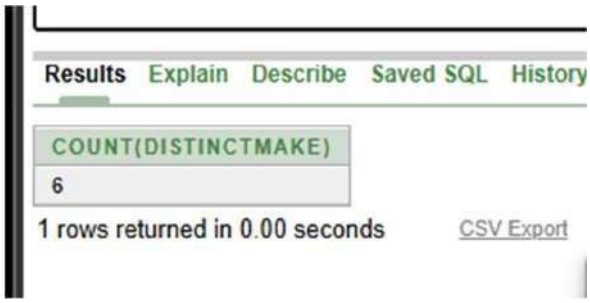
SELECT CUSTOMERS.CName, CABHUBS3.VehicleName FROM CUSTOMERS, CABHUBS3 WHERE CUSTOMERS.VCode = CABHUBS3.VCode;

Results Explain Describe Saved SQL History

CNAME	VEHICLENAME
Ketan Dhal	C-Class
Feroza Shah	A-Star
Raj Lal	Indigo

3 rows returned in 0.00 seconds CSV Export

b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;

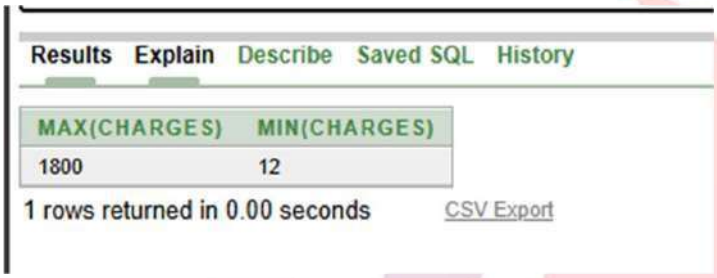


Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
6

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;



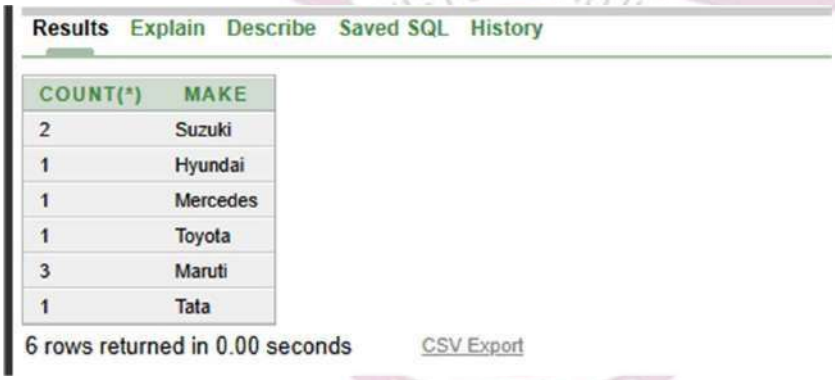
Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
1800	12

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT(*), Make FROM CABHUB;

SELECT COUNT(*), Make FROM CABHUB GROUP BY Make;



Results Explain Describe Saved SQL History

COUNT(*)	MAKE
2	Suzuki
1	Hyundai
1	Mercedes
1	Toyota
3	Maruti
1	Tata

6 rows returned in 0.00 seconds CSV Export

(iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

Results Explain Describe Saved SQL History

VEHICLENAME
Swift
Dzire
SX4
C-Class

4 rows returned in 0.00 seconds CSV Export

2. Create Co2_EMISSIOON Table

CREATE TABLE CO2_EMISSIONS2 VCode INT PRIMARY KEY, Company VARCHAR(30),Model VARCHAR(30), FuelType VARCHAR(15),

EngineSize DECIMAL(3,1),CO2Emission INT);




```
Table created.
```

INSERT INTO CO2_EMISSIONS2 VALUES (101, 'Toyota', 'Corolla', 'Petrol', 1.8, 170);

INSERT INTO CO2_EMISSIONS2 VALUES (102, 'Honda', 'City', 'Petrol', 1.5, 145);

INSERT INTO CO2_EMISSIONS2 VALUES (103, 'Hyundai', 'Creta', 'Diesel', 1.6, 155);

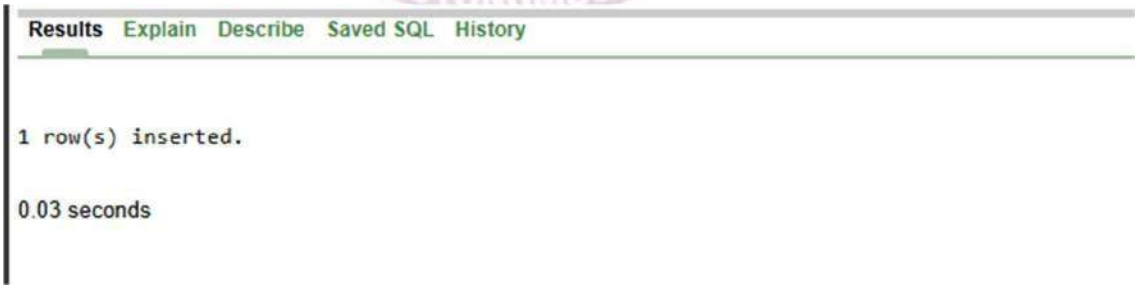
INSERT INTO CO2_EMISSIONS2 VALUES (104, 'Maruti', 'Swift', 'Diesel', 1.3, 120);

INSERT INTO CO2_EMISSIONS2 VALUES (105, 'Tata', 'Nexon', 'Petrol', 1.2, 135);

INSERT INTO CO2_EMISSIONS2 VALUES (106, 'Kia', 'Seltos', 'Diesel', 1.5, 150);

INSERT INTO CO2_EMISSIONS2 VALUES (107, 'Ford', 'EcoSport', 'Petrol', 1.5, 160);

SELECT*FROM CO2_EMISSIONS2



(a) Display all the details of vehicles that use Petrol as the fuel type.
FROM CO2_EMISSION WHERE FuelType = 'Petrol';

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
105	Maruti	Swift	Petrol	1.2	125
108	MG	Hector	Petrol	1.5	160

4 rows returned in 0.05 seconds CSV Export

(b) Display the Company and Model of vehicles whose EngineSize is greater than 1.5 litres.

SELECT Company, Model FROM CO2_EMISSION
WHERE EngineSize > 1.5;

Results Explain Describe Saved SQL History

COMPANY	MODEL
Toyota	Corolla
Mahindra	XUV700

2 rows returned in 0.00 seconds CSV Export

(c) Display the Company, Model, and CO2Emission in ascending order of CO2Emission.

SELECT Company, Model, CO2Emission FROM CO2_EMISSION
ORDER BY CO2Emission ASC;

Results Explain Describe Saved SQL History

COMPANY	MODEL	CO2EMISSION
Maruti	Swift	125
Tata	Nexon	138
Hyundai	Creta	145
Kia	Seltos	150
Honda	City	152
MG	Hector	160
Toyota	Corolla	165
Mahindra	XUV700	185

8 rows returned in 0.00 seconds CSV Export

(d) Display the details of vehicles whose CO2Emission is greater than 150 g/km.

SELECT *FROM CO2_EMISSION WHERE CO2Emission > 150

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
107	Mahindra	XUV700	Diesel	2.2	185
108	MG	Hector	Petrol	1.5	160

4 rows returned in 0.00 seconds CSV Export

(e) Count the number of vehicles that use Diesel as the fuel type.

SELECT COUNT(*)FROM CO2_EMISSION WHERE FuelType = 'Diesel';

Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.00 seconds CSV Export

(f) Insert the following record into the CO2_EMISSION table.

108, 'MG', 'Hector', 'Petrol', 1.5, 160

SELECT * FROM CO2_EMISSIONS2 WHERE VCode = 108;

Results Explain Describe Saved SQL History

VCODE	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
108	MG	Hector	Petrol	1.5	160

1 rows returned in 0.00 seconds CSV Export

(g) Write the output of the following SQL commands.

(i) SELECT MAX(CO2Emission)FROM CO2_EMISSION;

Results Explain Describe Saved SQL History

MAX(CO2EMISSION)
185

rows returned in 0.01 seconds CSV Export

(ii) SELECT MIN(EngineSize)FROM CO2_EMISSION;

SELECT MIN(EngineSize)FROM CO2_EMISSION;



(iii) SELECT AVG(CO2Emission)FROM CO2_EMISSIONWHERE FuelType='Petrol';

SELECT AVG(CO2Emission)FROM CO2_EMISSION WHERE FuelType = 'Petrol';



(iv) SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;

SELECT COUNT(*)FROM CO2_EMISSION WHERE EngineSize >= 1.5;



AUTONOMOUS

3. CREATE TABLE APPLICANTS

CREATE TABLE APPLICANTS (A_ID VARCHAR(5) PRIMARY KEY NAME VARCHAR(30), FEE NUMBER(8,2), GENDER CHAR(1), JOINYEAR NUMBER(4),C_ID VARCHAR(5));




```
Table created.
```

0.14 seconds

INSERT INTO APPLICANTS VALUES ('P01', 'Anu', 25000, 'F', 2008, 'A01');

INSERT INTO APPLICANTS VALUES ('P02', 'Bala', 35000, 'M', 2009, 'A05');

INSERT INTO APPLICANTS VALUES ('P03', 'Divya', 40000, 'F', 2011, '02');

INSERT INTO APPLICANTS VALUES ('P04', 'Kumar', 30000, 'M', 2007, 'A01');

INSERT INTO APPLICANTS VALUES ('P05', 'Priya', 45000, 'F', 2009, 'A05');

INSERT INTO APPLICANTS VALUES ('P06', 'Ravi', 28000, 'M', 2012, '02');




```
1 row(s) inserted.
```

0.00 seconds

CREATE TABLE COURSES

CREATE TABLE COURSES (C_ID VARCHAR(5) PRIMARY KEY, C_NAME VARCHAR(30));




```
Table created.
```

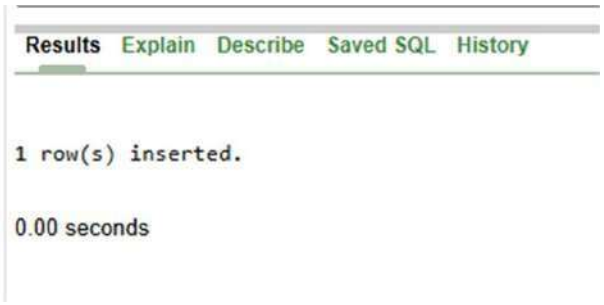
0.02 seconds

INSERT INTO COURSES VALUES ('A01', 'Computer Science');

INSERT INTO COURSES VALUES ('A02', 'Mathematics');

INSERT INTO COURSES VALUES ('A05', 'Commerce');

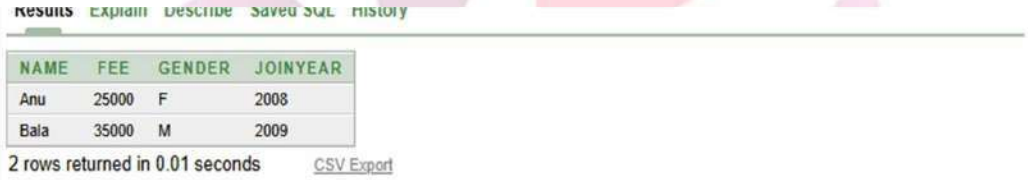
INSERT INTO COURSES VALUES ('02', 'Physics');



(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.

SELECT NAME, FEE, GENDER, JOINYEAR FROM APPLICANTS

WHERE JOINYEAR < 2010;



NAME	FEE	GENDER	JOINYEAR
Anu	25000	F	2008
Bala	35000	M	2009

2 rows returned in 0.01 seconds CSV Export

b)To display the names of applicants, who are paying fee more than 30000.

SELECT NAME FROM APPLICANTS WHERE FEE > 30000;



(c) To display name of all applicants in ascending order of their joinyear.

SELECT NAMEz FROM APPLICANTS
ORDER BY JOINYEAR ASC;

Results Explain Describe Saved SQL History

NAME
Anu
Bala
Divya

3 rows returned in 0.00 seconds CSV Export

(d) To display the year and the total number of applicants joined in each YEAR from the table APPLICANTS.

```
SELECT JOINYEAR, COUNT(*) AS TOTAL_APPLICANTS
FROM APPLICANTS GROUP BY JOINYEAR;
```

Results Explain Describe Saved SQL History

JOINYEAR	TOTAL_APPLICANTS
2009	1
2011	1
2008	1

3 rows returned in 0.00 seconds CSV Export

(e) To display the C_ID (i.e. Course ID) and the number of applicants registered in the course from the APPLICANTS table.

```
SELECT C_ID, COUNT(*) AS TOTAL_APPLICANTS
FROM APPLICANTS GROUP BY C_ID;
```

Results Explain Describe Saved SQL History

C_ID	TOTAL_APPLICANTS
02	1
A05	1
A01	1

3 rows returned in 0.00 seconds CSV Export

(f) To display the applicant's name with their respective course's name from the tables APPLICANTS and COURSES.

```
SELECT APPLICANTS.NAME, COURSES.C_NAME
FROM APPLICANTS JOIN COURSES ON APPLICANTS.C_ID =
COURSES.C_ID;
```

Results Explain Describe Saved SQL History

NAME	C_NAME
Divya	Physics

1 rows returned in 0.00 seconds CSV Export

(g) Give the output of following SQL statements:

(g1) SELECT NAME, JOIN YEAR FROM APPLICANTS WHERE GENDER= 'F' AND C_ID= '02';

Results Explain Describe Saved SQL History

NAME	JOINYEAR
Divya	2011

1 rows returned in 0.00 seconds CSV Export

(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= 'M'

Results Explain Describe Saved SQL History

MIN(JOINYEAR)
2009

1 rows returned in 0.00 seconds CSV Export

(g3) SELE CT AVG(FEE) FROM APPLICANTS WHERE C_ID= 'A01' OR C_ID= 'A05';

Results Explain Describe Saved SQL History

AVG(FEE)
30000

1 rows returned in 0.00 seconds CSV Export

(g4) SELECT SUM (FEE), C_ID FROM APPLICATIONS GROUP BY C_ID HAVING COUNT (*)=2;

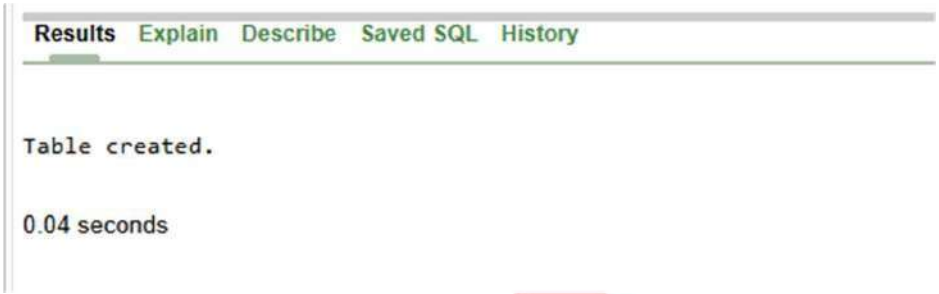
Results Explain Describe Saved SQL History

SUM(FEE)	C_ID
55000	A01

1 rows returned in 0.02 seconds CSV Export

4. Create PRODUCTS table

CREATE TABLE PRODUCTS (PID NUMBER(5) PNAME VARCHAR2(30), PRICE NUMBER(10,2), QTY NUMBER(5), SUPCODE VARCHAR2(5))**;**



INSERT INTO PRODUCTS VALUES (101,'Laptop',12000,150,'S01');
INSERT INTO PRODUCTS VALUES (102,'Mobile',15000,80,'S01');
INSERT INTO PRODUCTS VALUES (103,'Printer',9000,120,'S02');
INSERT INTO PRODUCTS VALUE S(104,'Monitor',18000,150,'S02');
INSERT INTO PRODUCTS VALUES(105,'Keyboard',10000,90,'S03');



Create SUPPLIERS table

CREATE TABLE SUPPLIERS (SUPCODE VARCHAR2(5),

SNAME VARCHAR2(30), CITY VARCHAR2(20));



INSERT INTO SUPPLIERS VALUES ('S01','ABC COMPANY','DELHI');

INSERT INTO SUPPLIERS VALUES ('S02','XYZ COMPANY','CHENNAI');

INSERT INTO SUPPLIERS VALUES ('S03','PQR COMPANY','MUMBAI');

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.00 seconds

a) To display the details of all the products in ascending order of product names (i.e. PNAME).

SELECT *FROM PRODUCTS ORDER BY PNAME ASC;

Results Explain Describe Saved SQL History

PID	PNAME	PRICE	QTY	SUPCODE
105	Keyboard	10000	90	S03
101	Laptop	12000	150	S01
102	Mobile	15000	80	S01
104	Monitor	18000	150	S02
103	Printer	9000	120	S02

5 rows returned in 0.02 seconds CSV Export

(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).

SELECT PNAME, PRICE FROM PRODUCTS WHERE PRICE BETWEEN 10000 AND 15000;

Results Explain Describe Saved SQL History

PNAME	PRICE
Laptop	12000
Mobile	15000
Keyboard	10000

3 rows returned in 0.00 seconds CSV Export

(c) To display the number of products which are supplied by each supplier i.e. the expected output should be

SELECT SUPCODE, COUNT(*) AS TOTAL_PRODUCT FROM PRODUCTS

GROUP BY SUPCODE;

Results Explain Describe Saved SQL History

SUPCODE	TOTAL_PRODUCTS
S01	2
S02	2
S03	1

3 rows returned in 0.02 seconds CSV Export

(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.

SELECT PRICE, PNAME, QTY FROM PRODUCTS WHERE QTY > 100;

Results Explain Describe Saved SQL History

PRICE	PNAME	QTY
12000	Laptop	150
9000	Printer	120
18000	Monitor	150

3 rows returned in 0.00 seconds CSV Export

(e) To display the names of those suppliers, who are either from DELHI or from CHENNAI.

SELECT SNAME FROM SUPPLIERS WHERE CITY = 'DELHI' OR CITY = 'CHENNAI';

Results Explain Describe Saved SQL History

SNAME
ABC COMPANY
XYZ COMPANY

2 rows returned in 0.00 seconds CSV Export

(f) To display the name of the companies and the name of the products in descending order of company names.

SELECT S.SNAME, P.PNAME FROM PRODUCTS P, SUPPLIERS S

WHERE P.SUPCODE = S.SUPCODE ORDER BY S.SNAME DESC;

Results Explain Describe Saved SQL History

SNAME	PNAME
XYZ COMPANY	Printer
XYZ COMPANY	Monitor
PQR COMPANY	Keyboard
ABC COMPANY	Mobile
ABC COMPANY	Laptop

5 rows returned in 0.00 seconds CSV Export

(g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.

(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

Results Explain Describe Saved SQL History

SUPCODE
S01
S02
S03

3 rows returned in 0.00 seconds CSV Export

(g2) SELECT MAX(PRICE), MIN (PRICE) FROM PRODUCTS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
18000	9000

1 rows returned in 0.00 seconds CSV Export

(g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

Results Explain Describe Saved SQL History

AMOUNT
2700000

1 rows returned in 0.00 seconds CSV Export

(g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P. SUPCODE = S. SUPCODE AND QTY>100;

Results Explain Describe Saved SQL History

PNAME	SNAME
Laptop	ABC COMPANY
Monitor	XYZ COMPANY
Printer	XYZ COMPANY

3 rows returned in 0.02 seconds CSV Export

5.Create SHOPPE table

CREATE TABLE SHOPPE (SNo VARCHAR2(5), SName VARCHAR2(30), Area VARCHAR2(30));

Results Explain Describe Saved SQL History

Table created.

0.04 seconds

INSERT INTO SHOPPE VALUES ('S01','ABC SHOPPE','Nehru Place');

INSERT INTO SHOPPE VALUES ('S02','XYZ SHOPPE','Nehru Place');

INSERT INTO SHOPPE VALUES ('S03','PQR SHOPPE','Karol Bagh');

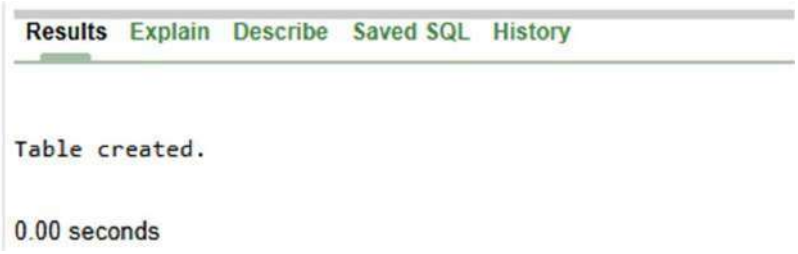
Results Explain Describe Saved SQL History

1 row(s) inserted.

0.00 seconds

Create ACCESSORIES table

CREATE TABLE ACCESSORIES (ANo VARCHAR2(5), Name VARCHAR2(30), Price NUMBER(10,2),SNo VARCHAR2(5));



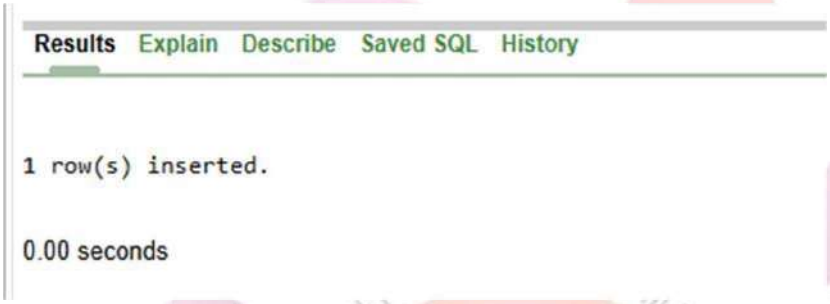
INSERT INTO ACCESSORIES VALUES ('A01','Keyboard',800,'S01');

INSERT INTO ACCESSORIES VALUES ('A02','Mouse',400,'S02');

INSERT INTO ACCESSORIES VALUES ('A03','Printer',5000,'S03');

INSERT INTO ACCESSORIES VALUES ('A04','Speaker',1200,'S02');

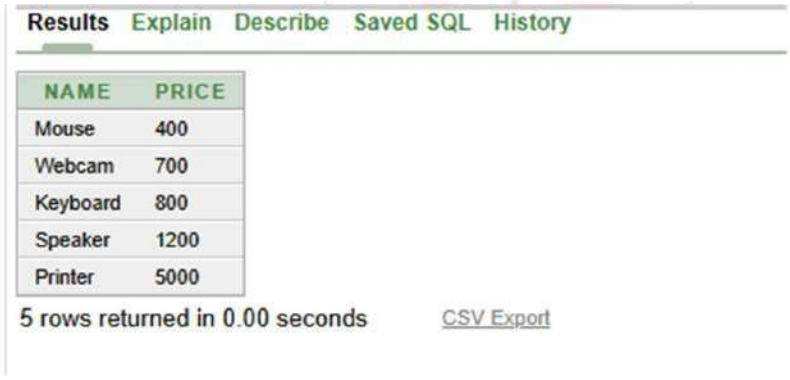
INSERT INTO ACCESSORIES VALUES ('A05','Webcam',700,'S01');



(a) Write the SQL queries:

(i) To display Name and Price of all the accessories in ascending order of their Price.

SELECT Name, Price FROM ACCESSORIES ORDER BY Price ASC;



(ii) To display Id and SName of all Shoppe in Nehru Place.

SELECT SNo, SName FROM SHOPPE WHERE Area = 'Nehru Place';

Results Explain Describe Saved SQL History

SNO	SNAME
S01	ABC SHOPPE
S02	XYZ SHOPPE

2 rows returned in 0.01 seconds CSV Export

(iii) To display Minimum and Maximum Price of each Name of accessories

SELECT NAME, MIN(PRICE), MAX(PRICE)FROM ACCESSORIES

GROUP BY NAME;

Results Explain Describe Saved SQL History

NAME	MIN(PRICE)	MAX(PRICE)
Printer	5000	5000
Mouse	400	400
Speaker	1200	1200
Keyboard	800	800
Webcam	700	700

5 rows returned in 0.02 seconds CSV Export

(iv) To display Name, Price of all accessories and their respective SName where they are available.

SELECT A.NAME, A.PRICE, S.SNAME FROM ACCESSORIES A, SHOPPE S WHERE A.SNO = S.SNO;

Results Explain Describe Saved SQL History

NAME	PRICE	SNAME
Keyboard	800	ABC SHOPPE
Mouse	400	XYZ SHOPPE
Printer	5000	PQR SHOPPE
Speaker	1200	XYZ SHOPPE
Webcam	700	ABC SHOPPE

5 rows returned in 0.01 seconds CSV Export

(b) (i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

Results Explain Describe Saved SQL History

NAME	PRICE	SNAME
Keyboard	800	ABC SHOPPE
Mouse	400	XYZ SHOPPE
Printer	5000	PQR SHOPPE
Speaker	1200	XYZ SHOPPE
Webcam	700	ABC SHOPPE

5 rows returned in 0.01 seconds CSV Export

(ii) SELECT Area, COUNT (*) FROM GROUP BY Area;

Results Explain Describe Saved SQL History

AREA	COUNT(*)
Karol Bagh	1
Nehru Place	2

2 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT (DISTINCT Area) FROM SHOPPE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
2

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT Name, Price*0.05 DISCOUNT FROM ACCESSORIES WHERE SNo IN ('S02, 'S03');

Results Explain Describe Saved SQL History

NAME	DISCOUNT
Mouse	20
Printer	250
Speaker	60

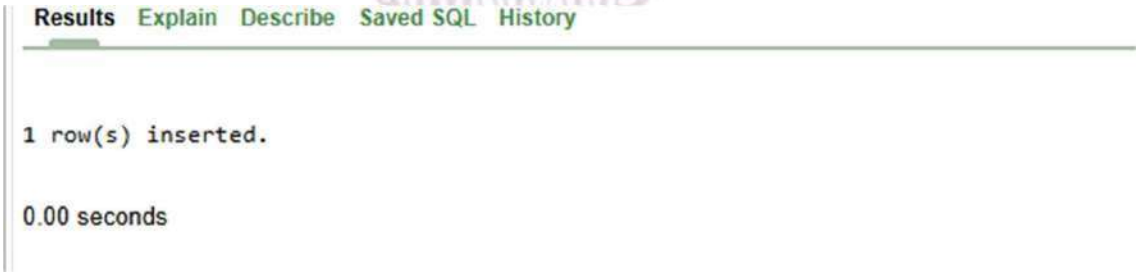
3 rows returned in 0.00 seconds CSV Export

6. HOSPITAL – CREATE TABLE

CREATE TABLE HOSPITAL (PID NUMBER(5), PNAME VARCHAR2(30), AGE NUMBER(3), DEPARTMENT VARCHAR2(20), DATOFADM DATE, CHARGES NUMBER(10,2), GENDER CHAR(1));



INSERT INTO HOSPITAL VALUES
(1, 'Asha', 25, 'Cardiology', TO_DATE('10/02/98','DD/MM/YY'), 500, 'F');
INSERT INTO HOSPITAL VALUES
(2, 'Ravi', 35, 'ENT', TO_DATE('15/03/98','DD/MM/YY'), 700, 'M');
INSERT INTO HOSPITAL VALUES
(3, 'Priya', 28, 'ENT', TO_DATE('20/04/98','DD/MM/YY'), 600, 'F');
INSERT INTO HOSPITAL VALUES
(4, 'Kumar', 45, 'Cardiology', TO_DATE('05/06/98','DD/MM/YY'), 900, 'M');
INSERT INTO HOSPITAL VALUES
(5, 'Meena', 22, 'Surgery', TO_DATE('12/07/98','DD/MM/YY'), 400, 'F');
INSERT INTO HOSPITAL VALUES
(6, 'Arun', 29, 'ENT', TO_DATE('01/08/98','DD/MM/YY'), 600, 'M');



(a) To select all the information of patients of cardiology department.

SELECT *FROM HOSPITAL WHERE DEPARTMENT = 'Cardiology';

Results Explain Describe Saved SQL History

PID	PNAME	AGE	DEPARTMENT	DATOFADM	CHARGES	GENDER
1	Asha	25	Cardiology	10-FEB-98	500	F
4	Kumar	45	Cardiology	05-JUN-98	900	M
4	Kumar	45	Cardiology	05-JUN-98	900	M

3 rows returned in 0.01 seconds CSV Export

(b) To list the names of female patients who are in ENT department

SELECT PNAME FROM HOSPITAL WHERE GENDER = 'F'

AND DEPARTMENT = 'ENT';

Results Explain Describe Saved SQL History

PNAME
Priya

1 rows returned in 0.00 seconds CSV Export

(c) To list name of all patients with their date of admission in ascending order.

SELECT PNAME, DATOFADM FROM HOSPITAL ORDER BY DATOFADM ASC;

Results Explain Describe Saved SQL History

PNAME	DATOFADM
Asha	10-FEB-98
Ravi	15-MAR-98
Priya	20-APR-98
Kumar	05-JUN-98
Kumar	05-JUN-98
Meena	12-JUL-98
Arun	01-AUG-98

7 rows returned in 0.00 seconds CSV Export

(d) To display Patient's Name, Charges, Age for only female patients.

SELECT PNAME, CHARGES, AGE FROM HOSPITAL

WHERE GENDER = 'F';

Results Explain Describe Saved SQL History

PNAME	CHARGES	AGE
Asha	500	25
Priya	600	28
Meena	400	22

3 rows returned in 0.00 seconds CSV Export

(e) To count the number of patients with Age<30.

SELECT COUNT(*)FROM HOSPITAL WHERE AGE < 30;

Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.00 seconds CSV Export

(f) To inset in a new row in the HOSPITAL table with the following data: 11, “Aftab”, 24, “Surgery”, {25/02/98}, 300, “M”

INSERT INTO HOSPITAL VALUES (11, 'Aftab', 24, 'Surgery', TO_DATE('25/02/98','DD/MM/YY'), 300, 'M');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(g) Give the output of following SQL statements:

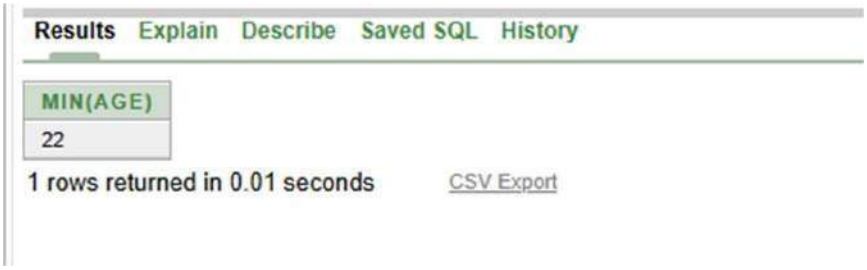
(i) Select COUNT (DISTINCT charges) from HOSPITAL;

Results Explain Describe Saved SQL History

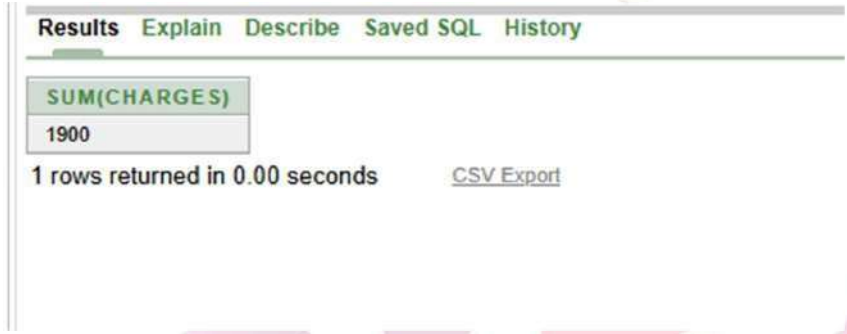
COUNT(DISTINCTCHARGES)
6

1 rows returned in 0.02 seconds CSV Export

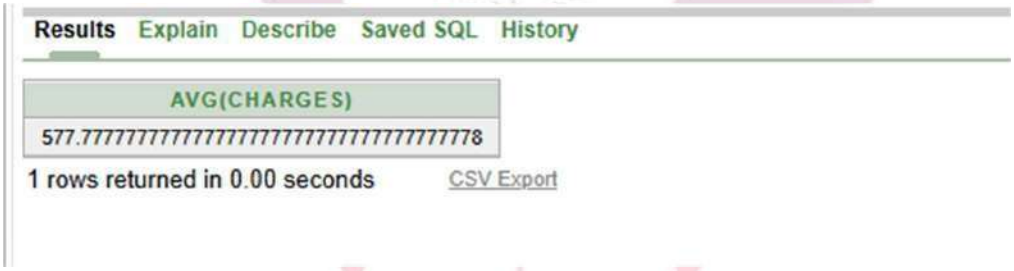
(ii) Select MIN (Age) from HOSPITAL where Gender = “F”



(iii) Select SUM (Charges) from HOSPITAL where Department = "ENT"



(iv) Select AVG (Charges) from HOSPITAL where Datofadm< {12/08/98}



7. Create STORE table

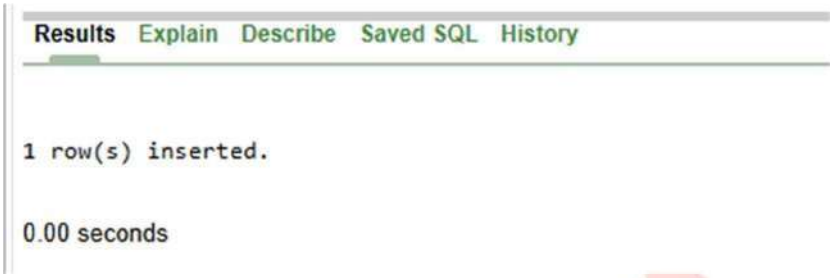
CREATE TABLE STORE (SNo VARCHAR2(5), SName VARCHAR2(30), Area VARCHAR2(30));



INSERT INTO STORE VALUES ('S01', 'ABC STORE', 'CP');

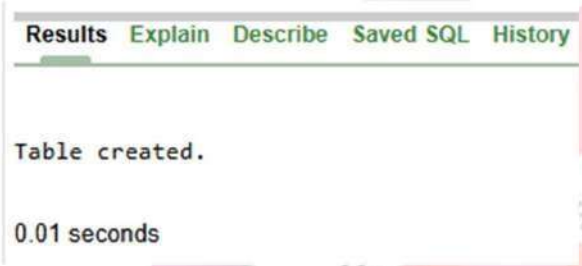
INSERT INTO STORE VALUES ('S02', 'XYZ STORE', 'CP');

INSERT INTO STORE VALUES ('S03', 'PQR STORE', 'Karol Bagh');



Create ITEM table

CREATE TABLE ITEM (INo VARCHAR2(5), IName VARCHAR2(30), Price NUMBER(10,2), SNo VARCHAR2(5));



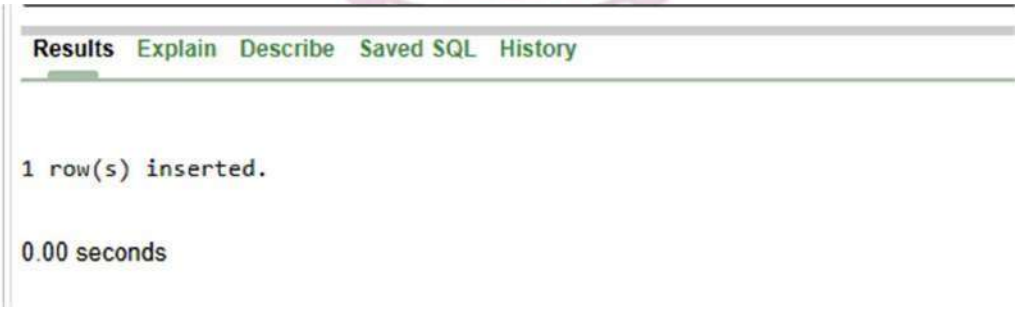
INSERT INTO ITEM VALUES ('I01', 'Laptop', 45000, 'S01');

INSERT INTO ITEM VALUES ('I02', 'Mobile', 15000, 'S02');

INSERT INTO ITEM VALUES ('I03', 'Printer', 8000, 'S03');

INSERT INTO ITEM VALUES ('I04', 'Keyboard', 3000, 'S02');

INSERT INTO ITEM VALUES ('I05', 'Monitor', 12000, 'S01');



(a) Write the SQL queries (i) to (iv):

(i) To display IName and Price of all the items in ascending order of their Price.

SELECT IName, Price FROM ITEM ORDER BY Price ASC;

Results Explain Describe Saved SQL History

INAME	PRICE
Keyboard	3000
Printer	8000
Monitor	12000
Mobile	15000
Laptop	45000

5 rows returned in 0.00 seconds CSV Export

(ii) To display SNo and SName of all store location in CP.

SELECT SNo, SName FROM STORE WHERE Area = 'CP';

Results Explain Describe Saved SQL History

SNO	SNAME
S01	ABC STORE
S02	XYZ STORE

2 rows returned in 0.00 seconds CSV Export

(iii) To display Minimum and maximum Price of each IName from the table ITEM.

SELECT IName, MIN(Price) AS MIN_PRICE, MAX(Price) AS MAX_PRICE

FROM ITEM GROUP BY IName;

Results Explain Describe Saved SQL History

INAME	MIN_PRICE	MAX_PRICE
Mobile	15000	15000
Printer	8000	8000
Laptop	45000	45000
Keyboard	3000	3000
Monitor	12000	12000

5 rows returned in 0.00 seconds CSV Export

(iv) To display IName, Price of all items and their respective SName where they are available.

SELECT I.IName, I.Price, S.SName FROM ITEM I, STORE S

WHERE I.SNo = S.SNo;

Results Explain Describe Saved SQL History

INAME	PRICE	SNAME
Laptop	45000	ABC STORE
Mobile	15000	XYZ STORE
Printer	8000	PQR STORE
Keyboard	3000	XYZ STORE
Monitor	12000	ABC STORE

5 rows returned in 0.00 seconds CSV Export

(b) Write the output of the following SQL commands (i) to (iv):

(i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;

Results Explain Describe Saved SQL History

INAME
Mobile
Printer
Laptop
Monitor

4 rows returned in 0.00 seconds CSV Export

(ii) SELECT Area, COUNT (*) FROM STORE GROUP BY Area;

Results Explain Describe Saved SQL History

AREA	COUNT(*)
CP	2
Karol Bagh	1

2 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT (DISTINCT Area) FROM STORE:

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
2

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

Results Explain Describe Saved SQL History

INAME	DISCOUNT
Mobile	750
Printer	400
Keyboard	150

3 rows returned in 0.00 seconds CSV Export

8. Create TEACHER Table

CREATE TABLE TEACHER (TNo NUMBER(3), Name VARCHAR2(30), Age NUMBER(3),Department VARCHAR2(20), Date_of_join DATE, Salary NUMBER(8,2),Gender CHAR(1));

Results Explain Describe Saved SQL History

```
Table created.

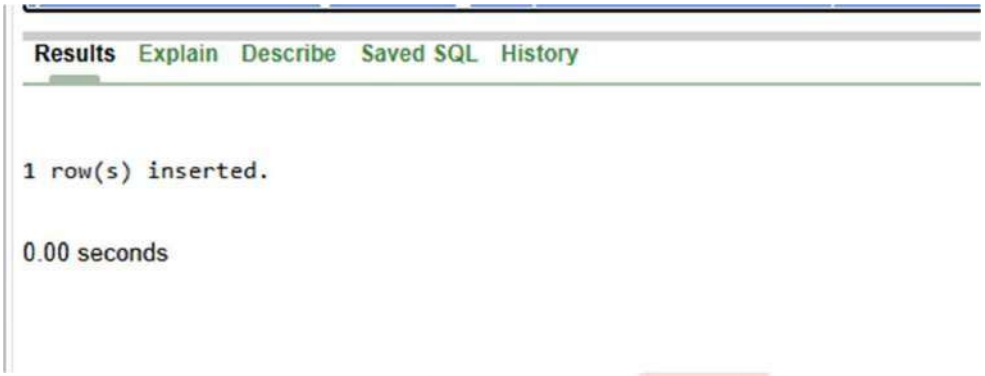
0.01 seconds
```

INSERT INTO TEACHER VALUES
(1, 'Anita', 25, 'History', TO_DATE('10/02/95','DD/MM/YY'), 2500, 'F');
INSERT INTO TEACHER VALUES
(2, 'Ravi', 30, 'Hindi', TO_DATE('15/03/95','DD/MM/YY'), 3000, 'M');
INSERT INTO TEACHER VALUES
(3, 'Priya', 24, 'Hindi', TO_DATE('20/04/95','DD/MM/YY'), 2800, 'F');
INSERT INTO TEACHER VALUES
(4, 'Kumar', 35, 'History', TO_DATE('05/06/95','DD/MM/YY'), 3500, 'M');
INSERT INTO TEACHER VALUES
(5, 'Meena', 22, 'English', TO_DATE('12/07/95','DD/MM/YY'), 2200, 'F');
INSERT INTO TEACHER VALUES
(6, 'Arun', 28, 'Hindi', TO_DATE('01/08/95','DD/MM/YY'), 2700, 'M');
INSERT INTO TEACHER VALUES

(7, 'Divya', 26, 'History', TO_DATE('10/09/95','DD/MM/YY'), 2600, 'F');

INSERT INTO TEACHER VALUES

8, 'Suresh', 32, 'English', TO_DATE('15/10/95','DD/MM/YY'), 3200, 'M');



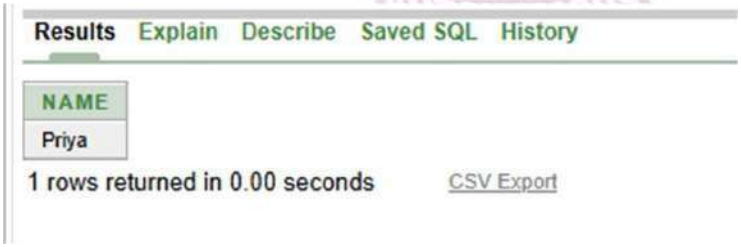
(a) To show all information about the teacher of history department

SELECT *FROM TEACHER WHERE Department = 'History';



(b) To list the names of female teacher who are in Hindi department

SELECT Name FROM TEACHER WHERE Gender = 'F' AND Department = 'Hindi';



(c) To list names of all teachers with their date of joining in ascending order.

SELECT Name, Date_of_join FROM TEACHER ORDER BY Date_of_join ASC;

Results Explain Describe Saved SQL History

NAME	DATE_OF_JOIN
Anita	10-FEB-95
Ravi	15-MAR-95
Priya	20-APR-95
Kumar	05-JUN-95
Meena	12-JUL-95
Arun	01-AUG-95
Divya	10-SEP-95
Suresh	15-OCT-95

8 rows returned in 0.00 seconds CSV Export

(d) To display teacher's Name, Salary, Age for male teacher only

SELECT Name, Salary, Age FROM TEACHER WHERE Gender = 'M';

Results Explain Describe Saved SQL History

NAME	SALARY	AGE
Ravi	3000	30
Kumar	3500	35
Arun	2700	28
Suresh	3200	32

4 rows returned in 0.02 seconds CSV Export

(e) To count the number of teachers with Age>23.

SELECT COUNT(*)FROM TEACHER WHERE Age > 23;

Results Explain Describe Saved SQL History

COUNT(*)
7

1 rows returned in 0.00 seconds CSV Export

(f) To inset a new row in the TEACHER table with the following data: 9, "Raja", 26, "Computer", {13/05/95}, 2300, "M"

INSERT INTO TEACHERVALUES (9,Raja', 26, 'Computer'
TO_DATE('13/05/95','DD/MM/YY'),2300, 'M');

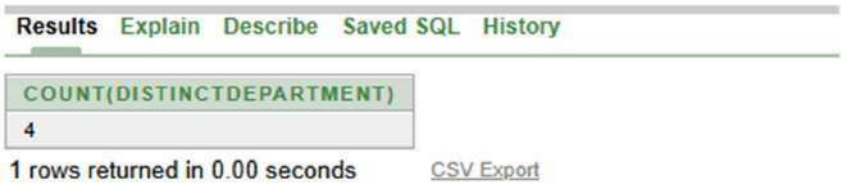
Results Explain Describe Saved SQL Histo

1 row(s) inserted.

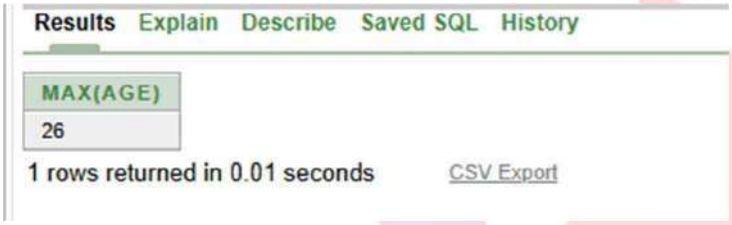
0.05 seconds

(g) Give the output of following SQL statements:

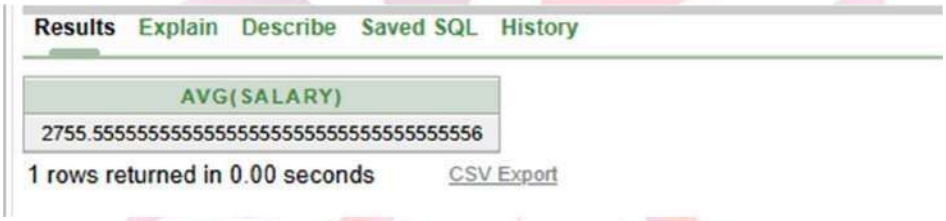
(i) Select COUNT (distinct department) from TEACHER;



(II) Select MAX (Age) from TEACHER where Gender= "F"



(iii) Select AVG (Salary) from TEACHER where Date of join < {12/07/96};



(iv) Select SUM (Salary) from TEACHER where Date of join < {12/07/96};



9. Create Books Table

CREATE TABLE Books (Book_Id VARCHAR(10), Book_Name VARCHAR(50), Author_Name VARCHAR(50), Publishers VARCHAR(30), Price INT,Type VARCHAR(20), Quantity INT);

Results Explain Describe Saved SQL History

```
Table created.

0.06 seconds
```

INSERT INTO Books VALUES ('F0001', 'Book Name', 'Author Name', 'EPB', 450, 'Fiction', 10);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) To show Book name, Author name and Price of books of EPB publishers.

SELECT Book_Name, Author_Name, Price FROM Books WHERE Publishers = 'EPB';

Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME	PRICE
Book Name	Author Name	450

1 rows returned in 0.00 seconds CSV Export

(b) To list the names of the books of Fiction Type.

SELECT Book_Name FROM Books WHERE Type = 'Fiction';

Results Explain Describe Saved SQL History

BOOK_NAME
Book Name

1 rows returned in 0.01 seconds CSV Export

(c) To display the names and price of the books in descending order of their price.

SELECT Book_Name, Price FROM Books ORDER BY Price DESC;



Results Explain Describe Saved SQL History

BOOK_NAME	PRICE
Book Name	450

1 rows returned in 0.00 seconds CSV Export

(d) To increase the price of all books of first publisher by 50.

UPDATE Books SET Price = Price + 50 WHERE Publishers = 'EPB';



Results Explain Describe Saved SQL History

1 row(s) updated.

0.00 seconds

(e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued. (The query will require contents from both the tables).

SELECT TABLE_NAME FROM USER_TABLES WHERE TABLE_NAME IN ('BOOKS', 'ISSUED');



Results Explain Describe Saved SQL History

TABLE_NAME
BOOKS

1 rows returned in 0.03 seconds CSV Export

(f) To insert a new row in the table Issued following the data: "F0002",4

CREATE TABLE Issueds(Book_Id VARCHAR2(10), Quantity_Issued NUMBER);



Results Explain Describe Saved SQL History

Table created.

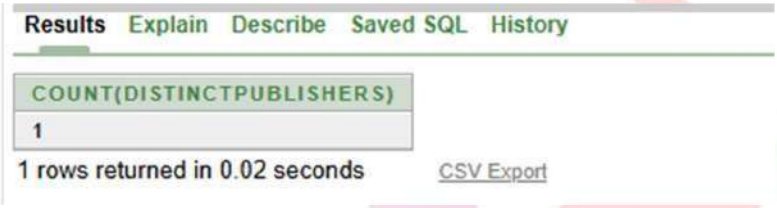
0.02 seconds

INSERT INTO Issued (Book_Id, Quantity_Issued)VALUES ('F0002', 4);

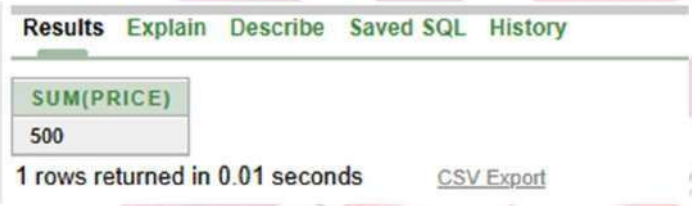


(g) Give the output of the following queries based on the above tables:

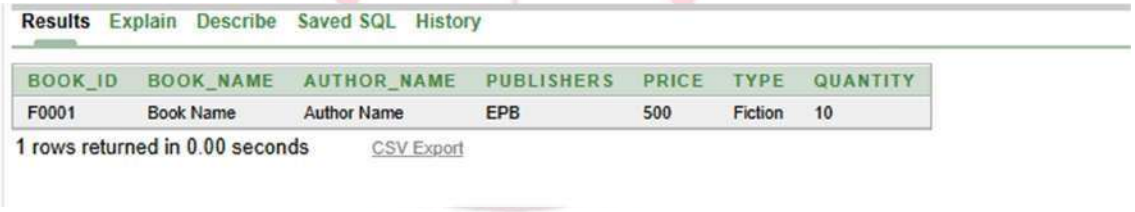
(i) SELECT COUNT(DISTINCT Publishers) FROM Books;



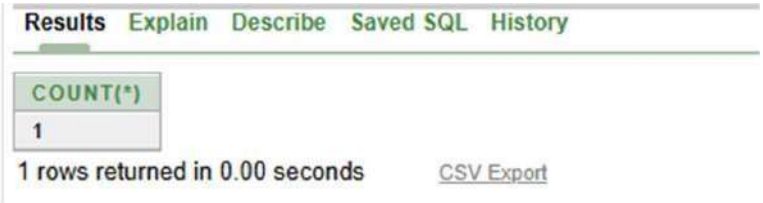
(ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;



(iii) SELECT BOOK_NAME,AUTHOR_NAME FROM Books WHERE Price < 500;



(iv) SELECT COUNT (*) FROM Books;



10. CARDEN Table

CREATE TABLE CARDEN (CarNo VARCHAR2(10),

CarName VARCHAR2(30), Make VARCHAR2(30), Color VARCHAR2(20), Capacity NUMBER, Charges NUMBER);



INSERT INTO CARDEN VALUES ('C001', 'CarName', 'Make', 'SILVER', 4, 500);



CUSTOMER Table

CREATE TABLE CUSTOMERS6 (Customer_Id VARCHAR2(10), Customer_Name VARCHAR2(30),CarNo VARCHAR2(10));



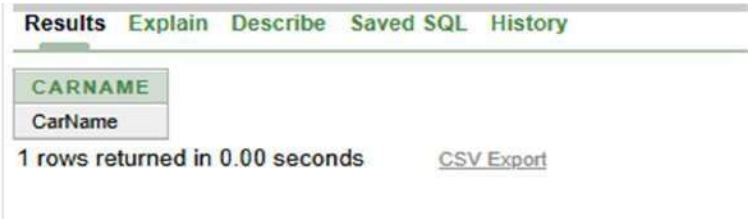
CREATE TABLE CUSTOMERS6 (Customer_Id VARCHAR2(10), Customer_Name VARCHAR2(30), CarNo VARCHAR2(10));



(a) Write SQL commands for the following statements:

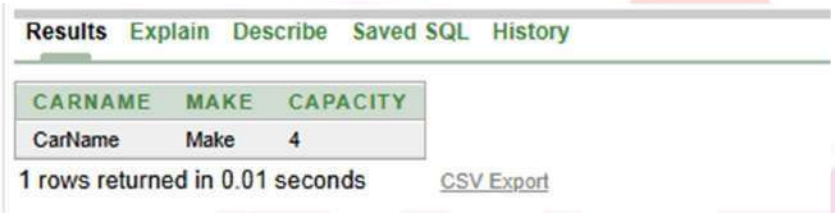
(i) To display the name of all the SILVER colored cars.

SELECT CarName FROM CARDEN WHERE Color = 'SILVER';



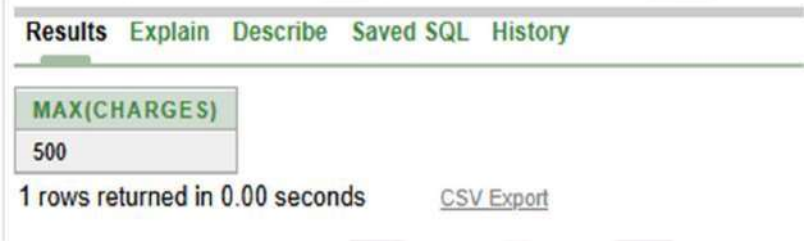
(ii) To display name of car, make and capacity of cars in descending order of their sittingcapacity.

SELECT CarName, Make, Capacity FROM CARDEN ORDER BY Capacity DESC;



(iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

SELECT MAX(Charges)FROM CARDEN;



(iv) To display the customer name and the corresponding name of the cards hired by them.

SELECT CUSTOMER.CName, CARDEN.CarName FROM CUSTOMER, CARDEN WHERE CUSTOMER.CCode = CARDEN.CCode;

DESC CUSTOMER;

Results Explain Describe Saved SQL History

Object Type TABLE Object CUSTOMER

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CUSTOMER	CUSTOMERID	Number	-	-	0	1	-	-	-
	CUSTOMERNAME	Varchar2	50	-	-	-	✓	-	-
	VEHICLENO	Varchar2	10	-	-	-	✓	-	-

1 - 3

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Make) FROM CARDEN;

Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
1

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MAX (Charges), MIN (Charges) FROM CARDEN;

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
500	500

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT (*), Make FROM CARDEN;

Results Explain Describe Saved SQL History

COUNT(*)	MAKE
1	Make

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT CarName FROM CARDEN WHERE Capacity = 4;

Results Explain Describe Saved SQL History

CARNAME
CarName

1 rows returned in 0.01 seconds CSV Export

11. EMPLOYEE table:

CREATE TABLE EMPLOYEE (ECODE NUMBER, NAME VARCHAR2(30),DESIGN VARCHAR2(30), SGRADE VARCHAR2(10),

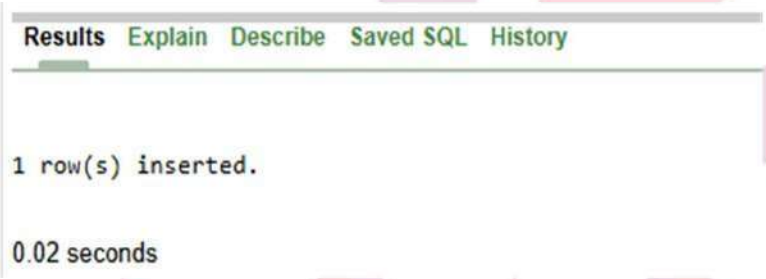
DOJ DATE,DOB DATE);



INSERT INTO EMPLOYEE VALUES (109, 'Harish Roy', 'HEAD-IT', 'S02',

TO_DATE('09-SEP-2007','DD-MON-YYYY'),

TO_DATE('21-APR-1983','DD-MON-YYYY'));



SALGRADE table:

CREATE TABLE SALGRADE (SGRADE VARCHAR2(10), SALARY NUMBER, HRA NUMBER);



INSERT INTO SALGRADE VALUES ('S01', 56000, 18000);

INSERT INTO SALGRADE VALUES ('S02', 32000, 12000);

INSERT INTO SALGRADE VALUES ('S03', 24000, 8000);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) Write SQL commands for the following statements:

(i) To display the detail of all the EMPLOYEE in descending order of DOJ.

SELECT *FROM EMPLOYEE ORDER BY DOJ DESC;

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	SGRADE	DOJ	DOB
109	Harish Roy	HEAD-IT	S02	09-SEP-07	21-APR-83

1 rows returned in 0.02 seconds CSV Export

(ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

SELECT NAME, DESIGN FROM EMPLOYEE WHERE SGRADE IN ('S02', 'S03');

Results Explain Describe Saved SQL History

NAME	DESIGN
Harish Roy	HEAD-IT

1 rows returned in 0.00 seconds CSV Export

(iii) To display the content of all the EMPLOYEE table, whose DOJ is in between '09-FEB-2006' and '08-AUG-2009'.

SELECT *FROM EMPLOYEE WHERE DOJ BETWEEN TO_DATE('09-FEB-2006','DD-MON-YYYY') AND TO_DATE('08-AUG-2009','DD-MON-YYYY');

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	SGRADE	DOJ	DOB
109	Harish Roy	HEAD-IT	S02	09-SEP-07	21-APR-83

1 rows returned in 0.02 seconds CSV Export

(iv) TO add a new row in the EMPLOYEE table with the following data:

109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983'.

INSERT INTO EMPLOYEE VALUES (109,'Harish Roy', 'HEAD-IT', 'S02', TO_DATE('09-SEP-2007','DD-MON-YYYY'),

TO_DATE('21-APR-1983','DD-MON-YYYY'));

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

Results Explain Describe Saved SQL History

COUNT(SGRADE)	SGRADE
2	S02

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

Results Explain Describe Saved SQL History

MIN(DOB)	MAX(DOJ)
21-APR-83	09-SEP-07

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S WHERE E. SGRADE = S. SGRADE AND E. ECODE<103;

Results Explain Describe Saved SQL History

ECODE	NAME	SGRADE
109	Harish Roy	S02
109	Harish Roy	S02

2 rows returned in 0.00 seconds CSV Export

(iv) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = 'S02';

Results Explain Describe Saved SQL History

SGRADE	SALARY+HRA
S02	44000

1 rows returned in 0.01 seconds CSV Export

12.Create WORKER table

CREATE TABLE WORKER (ECODE NUMBER, NAME VARCHAR2(30), DESIGN VARCHAR2(30),PLEVEL VARCHAR2(10), DOJ DATE,

DOB DATE);

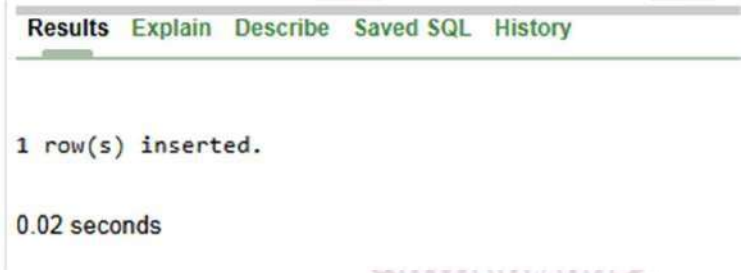
Results Explain Describe Saved SQL History

```
Table created.

0.04 seconds
```

```
INSERT INTO WORKER VALUES(11, 'Radhe Shyam', 'Supervisor', 'P001',
TO_DATE('13-SEP-2004','DD-MON-YYYY'),
TO_DATE('23-AUG-1981','DD-MON-YYYY'));

INSERT INTO WORKER VALUES(12, 'Chander Nath', 'Operator', 'P003',
TO_DATE('22-FEB-2010','DD-MON-YYYY'),
TO_DATE('12-JUL-1987','DD-MON-YYYY'));
INSERT INTO WORKER VALUES(13, 'Fizza', 'Operator', 'P003',
 TO_DATE('14-JUN-2009','DD-MON-YYYY'),
 TO_DATE('14-OCT-1983','DD-MON-YYYY'));
INSERT INTO WORKER VALUES(15, 'Ameen Ahmed', 'Mechanic', 'P002',
 TO_DATE('21-AUG-2006','DD-MON-YYYY'),
 TO_DATE('13-MAR-1984','DD-MON-YYYY'));
INSERT INTO WORKER VALUES(18, 'Sanya', 'Clerk', 'P002',
 TO_DATE('19-DEC-2005','DD-MON-YYYY'),
 TO_DATE('09-JUN-1983','DD-MON-YYYY'));
```



Create PAYLEVEL table

```
CREATE TABLE PAYLEVEL ( PLEVEL VARCHAR2(10), PAY NUMBER,
ALLOWANCE NUMBER);
```



INSERT INTO PAYLEVEL VALUES ('P001', 25000, 5000);

INSERT INTO PAYLEVEL VALUES ('P002', 35000, 7000);

INSERT INTO PAYLEVEL VALUES ('P003', 45000, 9000);

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.00 seconds

(a) Write SQL commands for the following statements:

(i) To display the detail of all WORKER in descending order of DOB.

SELECT *FROM WORKER ORDER BY DOB DESC;

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
12	Chander Nath	Operator	P003	22-FEB-10	12-JUL-87
15	Ameen Ahmed	Mechanic	P002	21-AUG-06	13-MAR-84
13	Fizza	Operator	P003	14-JUN-09	14-OCT-83
18	Sanya	Clerk	P002	19-DEC-05	09-JUN-83
11	Radhe Shyam	Supervisor	P001	13-SEP-04	23-AUG-81

5 rows returned in 0.00 seconds CSV Export

(ii) To display name and design of those WORKER, whoseplevel is either P001 to P002.

SELECT NAME, DESIGN FROM WORKER WHERE PLEVEL IN ('P001', 'P002');

Results Explain Describe Saved SQL History

NAME	DESIGN
Radhe Shyam	Supervisor
Ameen Ahmed	Mechanic
Sanya	Clerk

3 rows returned in 0.01 seconds CSV Export

(iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.

SELECT *FROM WORKER WHERE DOB BETWEEN TO_DATE('19-JAN-1984','DD-MON-YYYY') AND TO_DATE('18-JAN-1987','DD-MON-YYYY');

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
15	Ameen Ahmed	Mechanic	P002	21-AUG-06	13-MAR-84

1 rows returned in 0.00 seconds CSV Export

(iv) To add a new row with the following:

19, 'Daya Kishore', 'Operator', 'P003', '19-JUN-2008', '11-JUL-1984'.

INSERT INTO WORKER VALUES (19, 'Daya Kishore', 'Operator', 'P003',
TO_DATE('19-JUN-2008','DD-MON-YYYY'),
TO_DATE('11-JUL-1984','DD-MON-YYYY'));

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.00 seconds

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

Results Explain Describe Saved SQL History

COUNT(PLEVEL)	PLEVEL
1	P001
3	P003
2	P002

3 rows returned in 0.01 seconds CSV Export

(ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;

Results Explain Describe Saved SQL History

MAX(DOB)	MIN(DOJ)
12-JUL-87	13-SEP-04

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W. PLEVEL= P.LEVEL AND W. ECODE<13;

Results Explain Describe Saved SQL History

NAME	PAY
Radhe Shyam	25000
Chander Nath	45000

2 rows returned in 0.00 seconds CSV Export

(iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PLEVEL WHERE PLEVEL = 'P003';

Results Explain Describe Saved SQL History

PLEVEL	PAY+ALLOWANCE
P003	54000

1 rows returned in 0.00 seconds CSV Export

13.Create STORE table

CREATE TABLE STORE (ItemNo NUMBER(4) PRIMARY KEY, Item VARCHAR2(30), Scode NUMBER(3),Qty NUMBER(5), Rate NUMBER(10,2), LastBuy DATE

Results Explain Describe Saved SQL History

```
Table created.

0.03 seconds
);
```

INSERT INTO STORE VALUES(2001, 'Pen', 21, 100, 10, TO_DATE('15-01-2025','DD-MM-YYYY'));

INSERT INTO STORE VALUES(2002, 'Pencil', 22, 120, 8, TO_DATE('20-02-2025','DD-MM-YYYY'));

INSERT INTO STORE VALUES(2003, 'Notebook', 23, 150, 25, TO_DATE('10-03-2025','DD-MM-YYYY'));

INSERT INTO STORE VALUES(2004, 'Eraser', 22, 130, 12, TO_DATE('05-04-2025','DD-MM-YYYY'));

INSERT INTO STORE VALUES(2005, 'Marker', 24, 90, 20, TO_DATE('12-05-2025','DD-MM-YYYY'));

INSERT INTO STORE VALUES(2006, 'Scale', 23, 110, 18, TO_DATE('18-06-2025','DD-MM-YYYY'));



CREATE TABLE SUPPLIERS

CREATE TABLE SUPPLIERS8 (Scode NUMBER(3) PRIMARY KEY,

Sname VARCHAR2(30));



INSERT INTO SUPPLIERS8 VALUES (21, 'ABC Suppliers');

INSERT INTO SUPPLIERS8 VALUES (22, 'MKS Suppliers');

INSERT INTO SUPPLIERS8 VALUES (23, 'Ravi Traders');

INSERT INTO SUPPLIERS8 VALUES (24, 'Sri Suppliers');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) Write SQL commands for the following statements:

(i) To display details of all the items in the Store table in ascending order of LastBuy.

SELECT * FROM STORE ORDER BY LastBuy ASC;

Results Explain Describe Saved SQL History

ITEMNO	ITEM
2003	Notebook
2005	Marker
2006	Scale

3 rows returned in 0.00 seconds CSV Export

(ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than15 Rupees.

SELECT ItemNo, Item FROM STORE WHERE Rate > 15;

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2002	Pencil	22	120	8	20-FEB-25
2003	Notebook	23	150	25	10-MAR-25
2004	Eraser	22	130	12	05-APR-25

3 rows returned in 0.00 seconds CSV Export

(iii) To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

SELECT *FROM STORE WHERE Scode = 22 OR Qty > 110;

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Pen	21	100	10	15-JAN-25
2002	Pencil	22	120	8	20-FEB-25
2003	Notebook	23	150	25	10-MAR-25
2004	Eraser	22	130	12	05-APR-25
2005	Marker	24	90	20	12-MAY-25
2006	Scale	23	110	18	18-JUN-25

6 rows returned in 0.02 seconds CSV Export

(iv)To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

SELECT Scode, MIN(Rate)FROM STORE GROUP BY Scode;

Results Explain Describe Saved SQL History

SCODE	MIN(RATE)
22	8
21	10
24	20
23	18

4 rows returned in 0.00 seconds CSV Export

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Scode) FROM STORE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTSCODE)
4

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT Rate * Qty FROM STORE WHERE ItemNo=2004;

Results Explain Describe Saved SQL History

RATE*QTY
1560

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.ScodeAND ItemNo=2006;

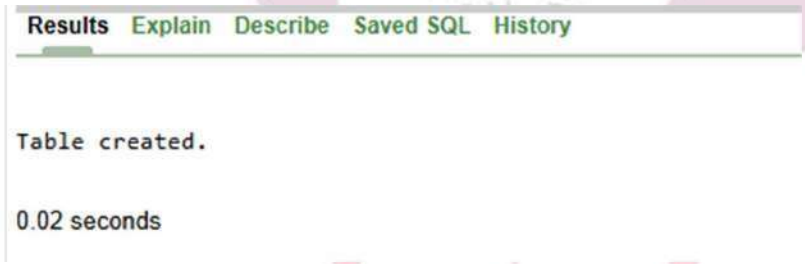


(iv) SELECT MAX(LastBuy) FROM STORE;



14. GARMENT Table

CREATE TABLE GARMENT(GCODE NUMBER(4) PRIMARY KEY, DESCRIPTION VARCHAR2(30), TYPE VARCHAR2(20), FCODE VARCHAR2(5), PRICE NUMBER(8,2), READYDATE DATE);



INSERT INTO GARMENT VALUES(1001, 'Cotton Shirt', 'Shirt', 'F01', 1200, TO_DATE('10-12-2007','DD-MM-YYYY'));

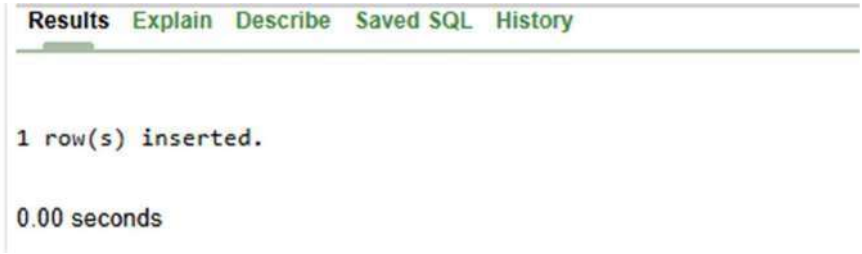
INSERT INTO GARMENT VALUES(1002, 'Silk Saree', 'Saree', 'F02', 1500, TO_DATE('15-01-2008','DD-MM-YYYY'));

INSERT INTO GARMENT VALUES(1003, 'Denim Jeans', 'Jeans', 'F03', 1800, TO_DATE('20-03-2008','DD-MM-YYYY'));

INSERT INTO GARMENT VALUES(1004, 'Cotton Pant', 'Pant', 'F01', 1100, TO_DATE('08-06-2008','DD-MM-YYYY'));

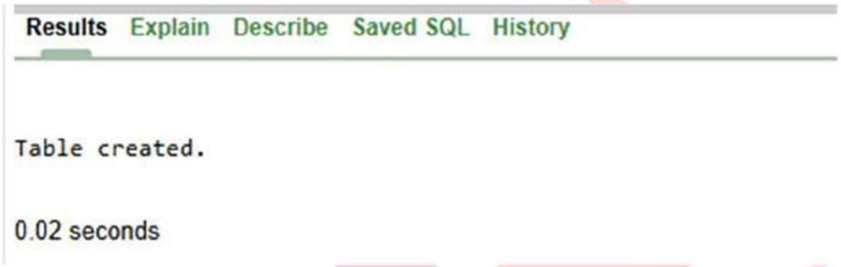
INSERT INTO GARMENT VALUES(1005, 'Wool Sweater', 'Sweater', 'F04', 2000, TO_DATE('16-06-2008','DD-MM-YYYY'));

INSERT INTO GARMENT VALUES(1006, 'Linen Shirt', 'Shirt', 'F03', 1600, TO_DATE('25-06-2008','DD-MM-YYYY'));



FABRIC Table

CREATE TABLE FABRIC(FCODE VARCHAR2(5) PRIMARY KEY, TYPE VARCHAR2(20));



INSERT INTO FABRIC VALUES('F01', 'Cotton');

INSERT INTO FABRIC VALUES('F02', 'Silk');

INSERT INTO FABRIC VALUES('F03', 'Denim');

INSERT INTO FABRIC VALUES('F04', 'Wool');



(i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.

SELECT GCODE, DESCRIPTION FROM GARMENT ORDER BY GCODE DESC;

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION
1006	Linen Shirt
1005	Wool Sweater
1004	Cotton Pant
1003	Denim Jeans
1002	Silk Saree
1001	Cotton Shirt

6 rows returned in 0.01 seconds CSV Export

(ii) To display the details of all the GARMENT, which have READYDATE in between 08-DEC-07 and 16-JUN-08 (inclusive if both the dates).

SELECT *FROM GARMENT WHERE READYDATE BETWEEN

TO_DATE('08-12-2007','DD-MM-YYYY')AND

TO_DATE('16-06-2008','DD-MM-YYYY');

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	TYPE	FCODE	PRICE	READYDATE
1001	Cotton Shirt	Shirt	F01	1200	10-DEC-07
1002	Silk Saree	Saree	F02	1500	15-JAN-08
1003	Denim Jeans	Jeans	F03	1800	20-MAR-08
1004	Cotton Pant	Pant	F01	1100	08-JUN-08
1005	Wool Sweater	Sweater	F04	2000	16-JUN-08

5 rows returned in 0.00 seconds CSV Export

(iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.

SELECT AVG(PRICE)FROM GARMENT WHERE FCODE = 'F03';

Results Explain Describe Saved SQL History

AVG(PRICE)
1700

1 rows returned in 0.00 seconds CSV Export

(iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).

SELECT FCODE, MAX(PRICE) AS HIGHEST_PRICE, MIN(PRICE) AS LOWEST_PRICE FROM GARMENT GROUP BY FCODE;

Results Explain Describe Saved SQL History

FCODE	HIGHEST_PRICE	LOWEST_PRICE
F02	1500	1500
F04	2000	2000
F03	1800	1600
F01	1200	1100

4 rows returned in 0.00 seconds CSV Export

(v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = 'F01';

SELECT SUM(PRICE)FROM GARMENT WHERE FCODE = 'F01';

Results Explain Describe Saved SQL History

SUM(PRICE)
2300

1 rows returned in 0.00 seconds CSV Export

(vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT, FCODE = FABRIC.FCODE AND GARMENT.PRICE >=1260;

SELECT G.DESCRIPTION, F.TYPE FROM GARMENT G, FABRIC F

WHERE G.FCODE = F.FCODE AND G.PRICE >= 1260;

Results Explain Describe Saved SQL History

DESCRIPTION	TYPE
Silk Saree	Silk
Linen Shirt	Denim
Denim Jeans	Denim
Wool Sweater	Wool

4 rows returned in 0.00 seconds CSV Export

(vii) SELECT MAX (FCODE) FROM FABRIC;

SELECT MAX(FCODE)FROM FABRIC;

Results Explain Describe Saved SQL History

MAX(FCODE)
F04

1 rows returned in 0.00 seconds CSV Export

(viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

SELECT COUNT(DISTINCT PRICE)FROM GARMENT;

Results Explain Describe Saved SQL History

COUNT(DISTINCTPRICE)
6

1 rows returned in 0.00 seconds CSV Export

15.CREATE TABLE FURNITURE

CREATE TABLE FURNITURE(ITEMNO NUMBER(4) PRIMARY KEY,ITEMNAME VARCHAR2(30), TYPE VARCHAR2(20), DATEOFSTOCK DATE, PRICE NUMBER(10,2),DISCOUNT NUMBER(5,2));

Results Explain Describe Saved SQL History

```
Table created.

0.08 seconds
```

INSERT INTO FURNITURE VALUES(1, 'White lotus', 'Double Bed',TO DATE('23-02-2002','DD-MM-YYYY'), 30000, 25);

INSERT INTO FURNITURE VALUES(2, 'Pink feather', 'Baby cot',TO_DATE('20-01-2002','DD-MM-YYYY'), 7000, 20);

INSERT INTO FURNITURE VALUES(3, 'Dolphin', 'Baby cot',TO_DATE('19-02-2002','DD-MM-YYYY'), 9500, 20);

INSERT INTO FURNITURE VALUES(4, 'Decent', 'Office Table',TO_DATE('01-01-2002','DD-MM-YYYY'), 25000, 30);

INSERT INTO FURNITURE VALUES(5, 'Comfort zone', 'Double Bed', TO_DATE('12-01-2002','DD-MM-YYYY'), 25000, 25);

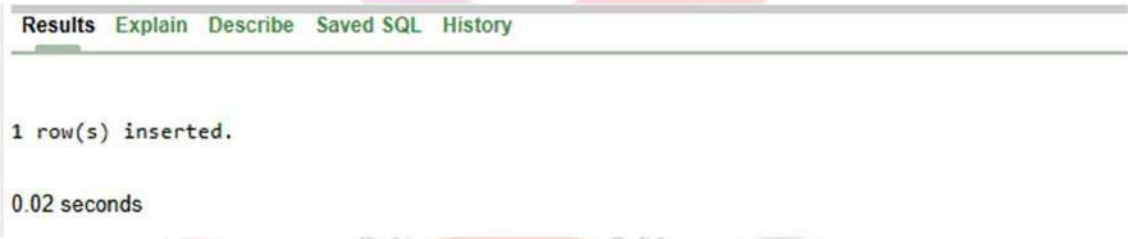
INSERT INTO FURNITURE VALUES(6, 'Donald', 'Baby cot',TO_DATE('24-02-2002','DD-MM-YYYY'), 6500, 15);

INSERT INTO FURNITURE VALUES(7, 'Royal Finish', 'Office Table',TO_DATE('20-02-2002','DD-MM-YYYY'), 18000, 30);

INSERT INTO FURNITURE VALUES(8, 'Royal tiger', 'Sofa', TO_DATE('22-02-2002','DD-MM-YYYY'), 31000, 30);

INSERT INTO FURNITURE VALUES(9, 'Econo sitting', 'Sofa',TO_DATE('13-12-2001','DD-MM-YYYY'), 9500, 25);

INSERT INTO FURNITURE VALUES(10, 'Eating Paradise', 'Dining Table',TO_DATE('19-02-2002','DD-MM-YYYY'), 11500, 25);



CREATE TABLE ARRIVALS

CREATE TABLE ARRIVALS(ITEMNO NUMBER(4) PRIMARY KEY,ITEMNAME VARCHAR2(30), TYPE VARCHAR2(20),DATEOFSTOCK DATE, PRICE NUMBER(10,2),DISCOUNT NUMBER(5,2));



INSERT INTO ARRIVALS VALUES(11, 'Elegant', 'Sofa',TO_DATE('10-01-2002','DD-MM-YYYY'), 18000, 20);

INSERT INTO ARRIVALS VALUES(12, 'Comfort', 'Double Bed', TO_DATE('15-01-2002','DD-MM-YYYY'), 22000, 25);

INSERT INTO ARRIVALS VALUES(13, 'Royal', 'Dining Table',TO_DATE('20-02-2002','DD-MM-YYYY'), 15000, 20);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) To show all information about the baby cots from the FURNITURE table.

SELECT *FROM FURNITUREWHERE TYPE = 'Baby cot';

Results Explain Describe Saved SQL History

ITEMNO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
2	Pink feather	Baby cot	20-JAN-02	7000	20
3	Dolphin	Baby cot	19-FEB-02	9500	20
6	Donald	Baby cot	24-FEB-02	6500	15

3 rows returned in 0.01 seconds CSV Export

(b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.

SELECT ITEMNAME FROM FURNITURE WHERE PRICE > 15000;

Results Explain Describe Saved SQL History

ITEMNAME
White lotus
Decent
Comfort zone
Royal Finish
Royal tiger

5 rows returned in 0.00 seconds CSV Export

(c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.

SELECT ITEMNAME, TYPE FROM FURNITURE WHERE DATEOFSTOCK < TO_DATE('22-01-2002','DD-MM-YYYY')ORDER BY ITEMNAME DESC;

Results Explain Describe Saved SQL History

ITEMNAME	TYPE
Pink feather	Baby cot
Econo sitting	Sofa
Decent	Office Table
Comfort zone	Double Bed

4 rows returned in 0.00 seconds CSV Export

(d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.

SELECT ITEMNAME, DATEOFSTOCK FROM FURNITURE WHERE DISCOUNT > 25;

Results Explain Describe Saved SQL History

ITEMNAME	DATEOFSTOCK
Decent	01-JAN-02
Royal Finish	20-FEB-02
Royal tiger	22-FEB-02

3 rows returned in 0.01 seconds CSV Export

(e) To count the number of items, whose TYPE is "Sofa" from FURNITURE table.

SELECT COUNT(*)FROM FURNITURE WHERE TYPE = 'Sofa';

Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

(f) To insert a new row in the ARRIVALS table with the following data: 14, "Velvet touch", Double bed", {25/03/03}, 25000, 30

INSERT INTO ARRIVALS VALUES(14, 'Velvet touch', 'Double bed', TO_DATE('25-03-2003','DD-MM-YYYY'), 25000, 30);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(g) Give the output of following SQL statement:

Note: outputs of the below mentioned queries should be based on original data given in both the tables i.e., without considering the insertion done in (g) part of this question.

(i) Select COUNT(distinct TYPE) from FURNITURE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTTYPE)
5

1 rows returned in 0.00 seconds CSV Export

(ii) Select MAX(DISCOUNT) from FURNITURE, ARRIVALS;

Results Explain Describe Saved SQL History

MAX(DISCOUNT)
30

1 rows returned in 0.00 seconds CSV Export

(iii) Select AVG(DISCOUNT) from FURNITURE where TYPE = "Baby cot";

Results Explain Describe Saved SQL History

AVG(DISCOUNT)
18.333333333333333333333333333333333333

1 rows returned in 0.00 seconds CSV Export

(iv) Select SUM(PRICE) from FURNITURE where DATEOFSTOCK < {12/02/02};

Results Explain Describe Saved SQL History

SUM(PRICE)
66500

1 rows returned in 0.00 seconds CSV Export

16.CREATE TABLE – COVID19

CREATE TABLE COVID19(PatientID NUMBER(3) PRIMARY KEY,PatientName VARCHAR2(30), State VARCHAR2(20),Age NUMBER(3), Status VARCHAR2(20), Vaccinated VARCHAR2(3));



INSERT INTO COVID19 VALUES(101, 'Arun', 'Tamil Nadu', 25, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES(102, 'Bala', 'Kerala', 32, 'Active', 'Yes');

INSERT INTO COVID19 VALUES(103, 'Divya', 'Tamil Nadu', 45, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES(104, 'Kumar', 'Karnataka', 51, 'Active', 'No');

INSERT INTO COVID19 VALUES(105, 'Priya', 'Tamil Nadu', 28, 'Recovered', 'No');

INSERT INTO COVID19 VALUES(106, 'Ravi', 'Andhra Pradesh', 36, 'Active', 'Yes');

INSERT INTO COVID19 VALUES(107, 'Meena', 'Tamil Nadu', 60, 'Active', 'Yes');



a) Display all details of patients from Tamil Nadu.

SELECT *FROM COVID19 WHERE State = 'Tamil Nadu';

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	Arun	Tamil Nadu	25	Recovered	Yes
103	Divya	Tamil Nadu	45	Recovered	Yes
105	Priya	Tamil Nadu	28	Recovered	No
107	Meena	Tamil Nadu	60	Active	Yes

4 rows returned in 0.00 seconds CSV Export

b) Display the names of patients whose Status is Recovered.

SELECT PatientName FROM COVID19 WHERE Status = 'Recovered';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Divya
Priya

3 rows returned in 0.00 seconds CSV Export

c) Display PatientName and Age in ascending order of Age.

SELECT PatientName, Age FROM COVID19ORDER BY Age ASC;

Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Arun	25
Priya	28
Bala	32
Ravi	36
Divya	45
Kumar	51
Meena	60

7 rows returned in 0.02 seconds CSV Export

d) Display the names of patients who are Vaccinated = 'Yes'.

SELECT PatientName FROM COVID19 WHERE Vaccinated = 'Yes';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Bala
Divya
Ravi
Meena

5 rows returned in 0.00 seconds CSV Export

e) Count the number of patients whose status is Active.

SELECT COUNT(*)FROM COVID19 WHERE Status = 'Active';

Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.01 seconds CSV Export

f) Insert the following record into the COVID19 table:

108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

INSERT INTO COVID19 VALUES (108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.03 seconds
```

(g) Write the output of the following SQL commands.

a. SELECT COUNT(*) FROM COVID19;



b. SELECT MAX(Age) FROM COVID19;



c. SELECT COUNT(*) FROM COVID19 WHERE Vaccinated='Yes';



d. SELECT MIN(Age)FROM COVID19 WHERE Status='Recovered';



17.CREATE TABLE – HOUSEPRICE

CREATE TABLE HOUSEPRICE(HouseID NUMBER(3) PRIMARY KEY, Location VARCHAR2(30),Bedrooms NUMBER(2),AreaSqft NUMBER(6),

Price NUMBER(10),HouseType VARCHAR2(20));



INSERT INTO HOUSEPRICE VALUES(201, 'Chennai', 2, 1200, 5500000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES(202, 'Coimbatore', 3, 1500, 6800000, 'Villa');

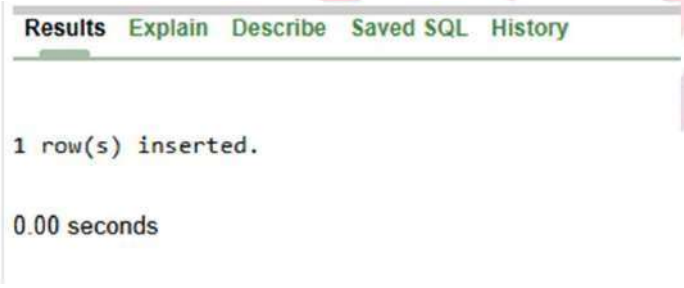
INSERT INTO HOUSEPRICE VALUES(203, 'Chennai', 3, 1800, 7500000, 'Villa');

INSERT INTO HOUSEPRICE VALUES(204, 'Madurai', 2, 1000, 4500000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES(205, 'Chennai', 4, 2200, 9000000, 'Independent');

INSERT INTO HOUSEPRICE VALUES(206, 'Salem', 3, 1600, 6200000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES(207, 'Trichy', 2, 1100, 5000000, 'Apartment');



a) Display all houses located in Chennai.

SELECT *FROM HOUSEPRICE WHERE Location = 'Chennai';

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1200	5500000	Apartment
203	Chennai	3	1800	7500000	Villa
205	Chennai	4	2200	9000000	Independent

3 rows returned in 0.01 seconds CSV Export

b) Display the Location and Price of houses having 3 Bedrooms.

SELECT Location, Price FROM HOUSEPRICE WHERE Bedrooms = 3;

Results Explain Describe Saved SQL History

LOCATION	PRICE
Chennai	7500000
Coimbatore	6800000
Salem	6200000

3 rows returned in 0.00 seconds CSV Export

c) Display Location, AreaSqft, and Price in descending order of Price.

SELECT Location, AreaSqft, Price FROM HOUSEPRICE ORDER BY Price DESC;

Results Explain Describe Saved SQL History

LOCATION	AREASQFT	PRICE
Chennai	2200	9000000
Chennai	1800	7500000
Coimbatore	1500	6800000
Salem	1600	6200000
Chennai	1200	5500000
Trichy	1100	5000000
Madurai	1000	4500000

7 rows returned in 0.00 seconds CSV Export

d) Display the details of houses costing more than ₹70,00,000.

SELECT *FROM HOUSEPRICE WHERE Price > 7000000;

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
203	Chennai	3	1800	7500000	Villa
205	Chennai	4	2200	9000000	Independent

2 rows returned in 0.00 seconds CSV Export

e) Count the number of Apartments.

SELECT COUNT(*)FROM HOUSEPRICE

Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.00 seconds CSV Export

f)WHERE HouseType = 'Apartment';f) Insert the following record:

208, 'Vellore', 3, 1700, 7300000, 'Villa'

INSERT INTO HOUSEPRICE VALUES (208, 'Vellore', 3, 1700, 7300000, 'Villa');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(g) Write the output of the following SQL commands.

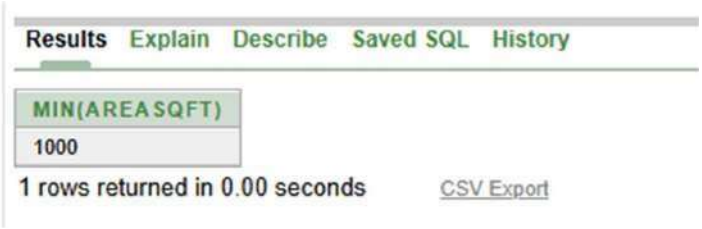
(i) SELECT MAX(Price)FROM HOUSEPRICE;

Results Explain Describe Saved SQL History

MAX(PRICE)
9000000

1 rows returned in 0.00 seconds CSV Export

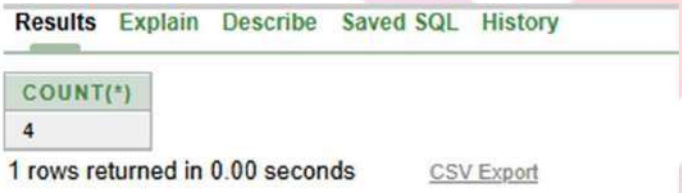
(ii) SELECT MIN(AreaSqft)FROM HOUSEPRICE;



(iii) SELECT AVG(Price)FROM HOUSEPRICE WHERE HouseType='Apartment';



(iv) SELECT COUNT(*)FROM HOUSEPRICE WHERE Bedrooms=3;



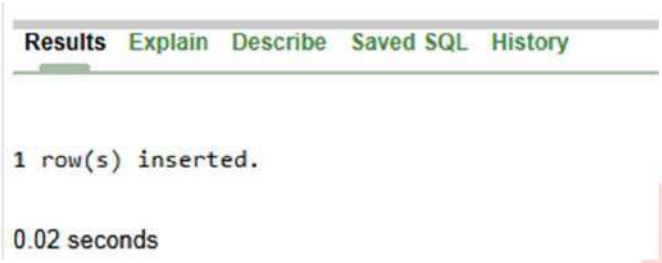
18.CREATE TABLE – CANCERDATA

CREATE TABLE CANCERDATA(PatientID NUMBER(3) PRIMARY KEY,
PatientName VARCHAR2(30), CancerType VARCHAR2(30),
Age NUMBER(3),Stage VARCHAR2(5),SurvivalYears NUMBER(3));



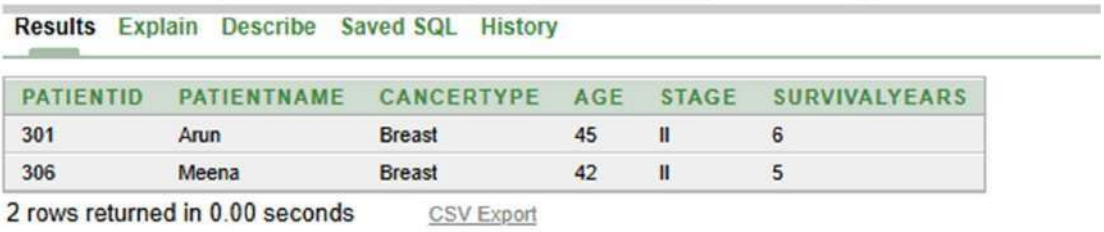
INSERT INTO CANCERDATA VALUES(301, 'Arun', 'Breast', 45, 'II', 6);
INSERT INTO CANCERDATA VALUES(302, 'Priya', 'Lung', 52, 'III', 4);
INSERT INTO CANCERDATA VALUES(303, 'Kumar', 'Breast', 38, 'I', 8);

```
INSERT INTO CANCERDATA VALUES(304, 'Divya', 'Lung', 60, 'II', 3);
INSERT INTO CANCERDATA VALUES(305, 'Ravi', 'Colon', 55, 'III', 7);
INSERT INTO CANCERDATA VALUES(306, 'Meena', 'Breast', 42, 'II', 5);
INSERT INTO CANCERDATA VALUES(307, 'Suresh', 'Lung', 48, 'I', 9);
```



a) Display all patients diagnosed with Breast Cancer.

```
SELECT *FROM CANCERDATA WHERE CancerType = 'Breast';
```



b) Display the names of patients whose Stage is III.

```
SELECT PatientNameFROM CANCERDATA  WHERE Stage = 'III';
```



c) Display PatientName and Age in ascending order of Age.

```
SELECT PatientName, Age FROM CANCERDATA ORDER BY Age ASC;
```

Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Meena	42
Arun	45
Suresh	48
Priya	52
Ravi	55
Divya	60

6 rows returned in 0.00 seconds CSV Export

d) Display the names of patients having SurvivalYears greater than 5.

SELECT PatientName FROM CANCERDATA WHERE SurvivalYears > 5;

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Priya
Divya
Ravi
Suresh
Meena

6 rows returned in 0.00 seconds CSV Export

e) Count the number of patients diagnosed with Lung Cancer.

SELECT COUNT(*)FROM CANCERDATA WHERE CancerType = 'Lung';

Results Explain Describe Saved SQL History

COUNT(*)
3

1 rows returned in 0.00 seconds CSV Export

f) Insert the following record:308, 'Anand', 'Leukemia', 50, 'II', 7

INSERT INTO CANCERDATA VALUES (308, 'Anand', 'Leukemia', 50, 'II', 7);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(g) Write the output of the following SQL commands.

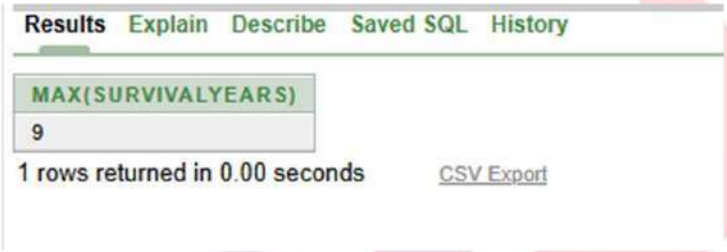
(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;


Results Explain Describe Saved SQL History

COUNT(DISTINCTCANCERTYPE)
4

1 rows returned in 0.02 seconds CSV Export

(ii) SELECT MAX(SurvivalYears) FROM CANCERDATA;


Results Explain Describe Saved SQL History

MAX(SURVIVALYEARS)
9

1 rows returned in 0.00 seconds CSV Export

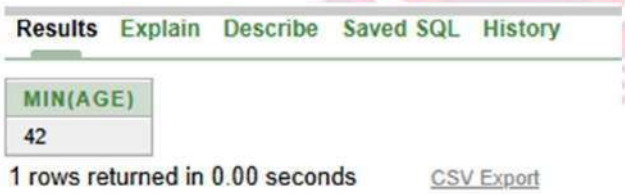
(iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';


Results Explain Describe Saved SQL History

AVG(AGE)
43.5

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';


Results Explain Describe Saved SQL History

MIN(AGE)
42

1 rows returned in 0.00 seconds CSV Export

19.CREATE TABLE – ITEMS

CREATE TABLE ITEMS(CODE NUMBER(4) PRIMARY KEY, INAME VARCHAR2(30),TCODE VARCHAR2(5),PRICE NUMBER(10,2), QTY NUMBER(5), COMPANY VARCHAR2(30));



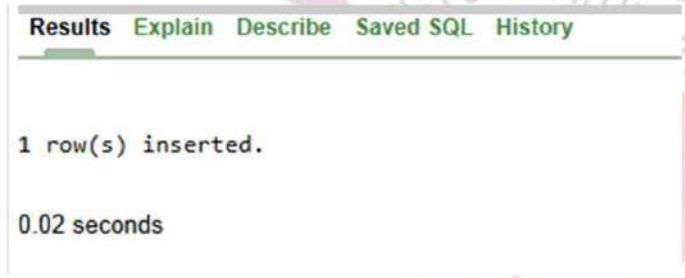
INSERT INTO ITEMS VALUES(1001, 'Computer', 'T01', 25000, 100, 'Dell');

INSERT INTO ITEMS VALUES(1002, 'Printer', 'T01', 15000, 200, 'HP');

INSERT INTO ITEMS VALUES(1003, 'Keyboard', 'T02', 12000, 180, 'Logitech');

INSERT INTO ITEMS VALUES(1004, 'Monitor', 'T02', 20000, 80, 'Samsung');

INSERT INTO ITEMS VALUES(1005, 'Mouse', 'T03', 8000, 50, 'Dell');



CREATE TABLE – TRADERS

CREATE TABLE TRADERS(TCODE VARCHAR2(5) PRIMARY KEY,

TNAME VARCHAR2(30), CITY VARCHAR2(20));



INSERT INTO TRADERS VALUES('T01', 'Rajesh', 'DELHI');

INSERT INTO TRADERS VALUES('T02', 'Suresh', 'MUMBAI');

INSERT INTO TRADERS VALUES('T03', 'Kumar', 'CHENNAI');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) To display the details of all the items in ascending order of item names (i.e. INAME).

SELECT *FROM ITEMS ORDER BY INAME ASC;

Results Explain Describe Saved SQL History

CODE	INAME	TCODE	PRICE	QTY	COMPANY
1001	Computer	T01	25000	100	Dell
1003	Keyboard	T02	12000	180	Logitech
1004	Monitor	T02	20000	80	Samsung
1005	Mouse	T03	8000	50	Dell
1002	Printer	T01	15000	200	HP

5 rows returned in 0.00 seconds CSV Export

(b) To display item name and price of all those items, whose price is in the range of 10000 and 22000 (both values inclusive).

SELECT INAME, PRICE FROM ITEMS WHERE PRICE BETWEEN 10000 AND 22000;

Results Explain Describe Saved SQL History

INAME	PRICE
Printer	15000
Keyboard	12000
Monitor	20000

3 rows returned in 0.00 seconds CSV Export

(c) To display the number of items, which are traded by each trader. The expected output of this query should be

T01 2

T02 2

T03 1

SELECT TCODE, COUNT(*)FROM ITEMS GROUP BY TCODE;

Results Explain Describe Saved SQL History

TCODE	COUNT(*)
T01	2
T03	1
T02	2

3 rows returned in 0.00 seconds CSV Export

(d) To display the price, item name (i.e. INAME) and quantity (i.e. QTY) of those items which have quantity more than 150.

SELECT PRICE, INAME, QTY FROM ITEMS WHERE QTY > 150;

Results Explain Describe Saved SQL History

PRICE	INAME	QTY
15000	Printer	200
12000	Keyboard	180

2 rows returned in 0.00 seconds CSV Export

(e) To display the names of those traders, who are either from DELHI or from MUMBAI.

SELECT TNAME FROM TRADERSWHERE CITY IN ('DELHI', 'MUMBAI');

Results Explain Describe Saved SQL History

TNAME
Rajesh
Suresh

2 rows returned in 0.00 seconds CSV Export

(f) To display the name of the companies and the bane of the items in descending order of company names.

SELECT COMPANY, INAME FROM ITEMS ORDER BY COMPANY DESC;

Results Explain Describe Saved SQL History

COMPANY	INAME
Samsung	Monitor
Logitech	Keyboard
HP	Printer
Dell	Mouse
Dell	Computer

5 rows returned in 0.00 seconds CSV Export

(g) Obtain the outputs of the following SQL queries based on the data given in tables ITEMS and TRADERS.

(g1) SELECT MAX (PRICE), MIN (PRICE) FROM ITEMS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
25000	8000

1 rows returned in 0.00 seconds CSV Export

(g2) SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

Results Explain Describe Saved SQL History

AMOUNT
1600000

1 rows returned in 0.00 seconds CSV Export

(g3) SELE CT DISTINCT TCODE FROM ITEMS;

Results Explain Describe Saved SQL History

TCODE
T01
T03
T02

3 rows returned in 0.02 seconds CSV Export

(g4) SELECT INAME, TNAME FROM ITEMS I, TRASERS T WHERE I, TCODE AND QTY<100;

Results Explain Describe Saved SQL History

INAME	TNAME
Monitor	Suresh
Mouse	Kumar

2 rows returned in 0.00 seconds CSV Export